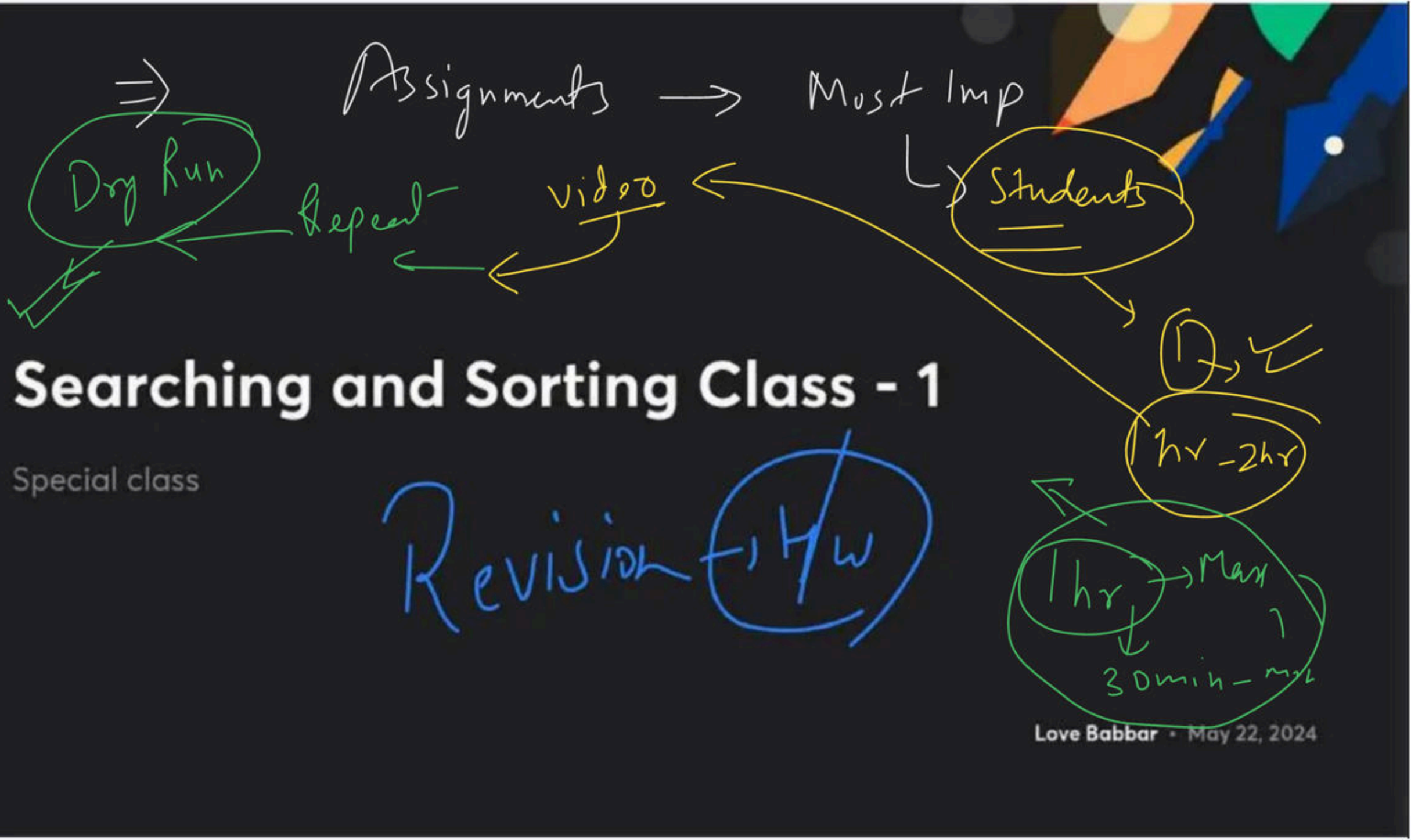
Doubt Class With Lakshay

Special class

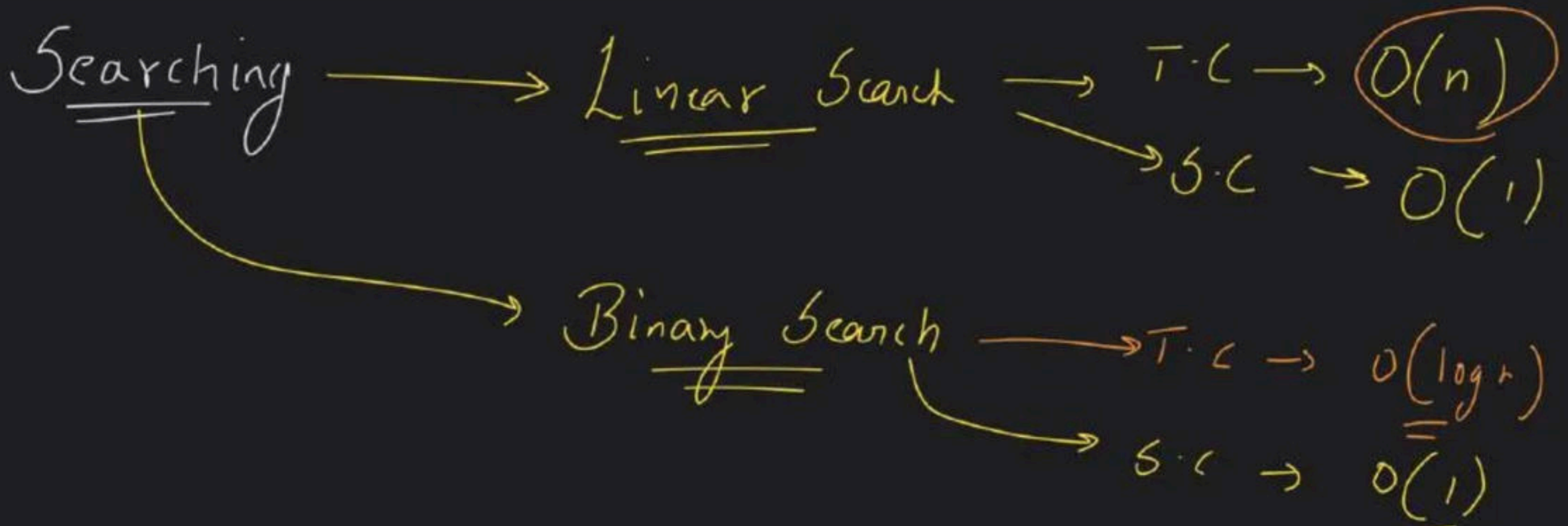
Love Babbar • June 1, 2024



Searching and Sorting Class - 1

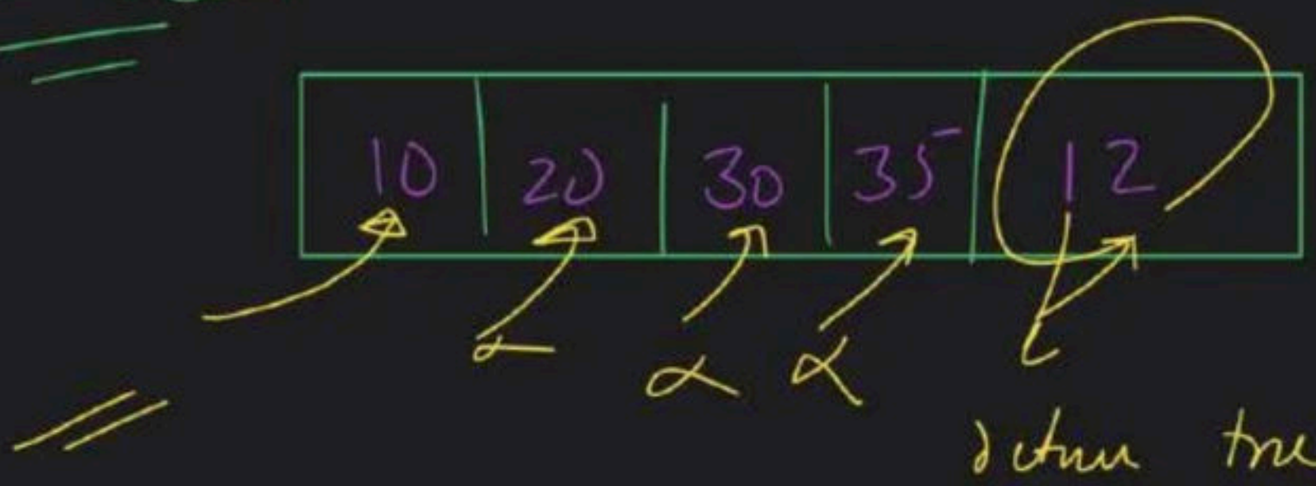
Special class

Love Babbar • May 22, 2024

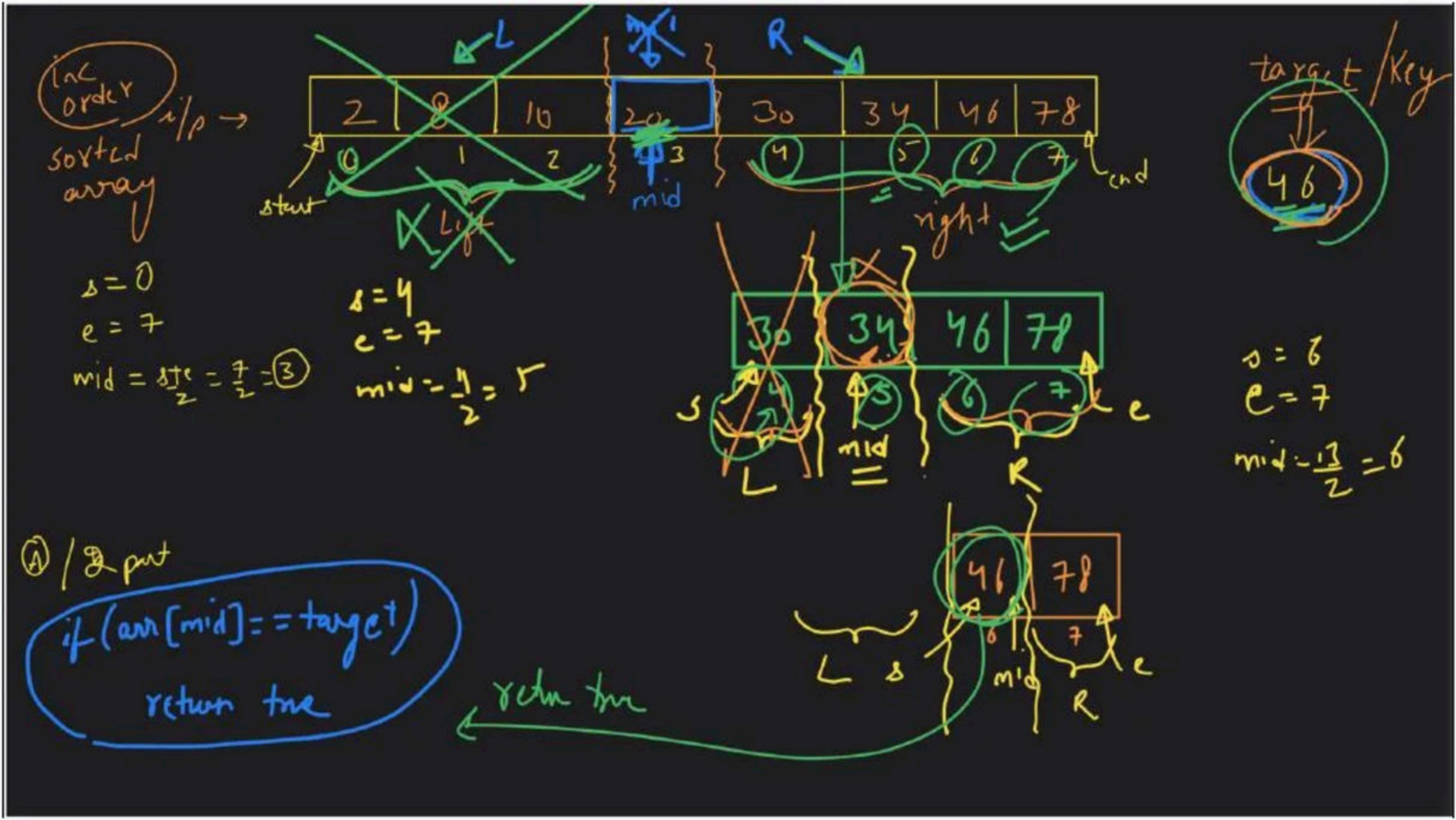

Searching
Linear Search
T.C → O(n)
S.C → O(1)
Binary Search
T.C → O(log n)
S.C → O(1)

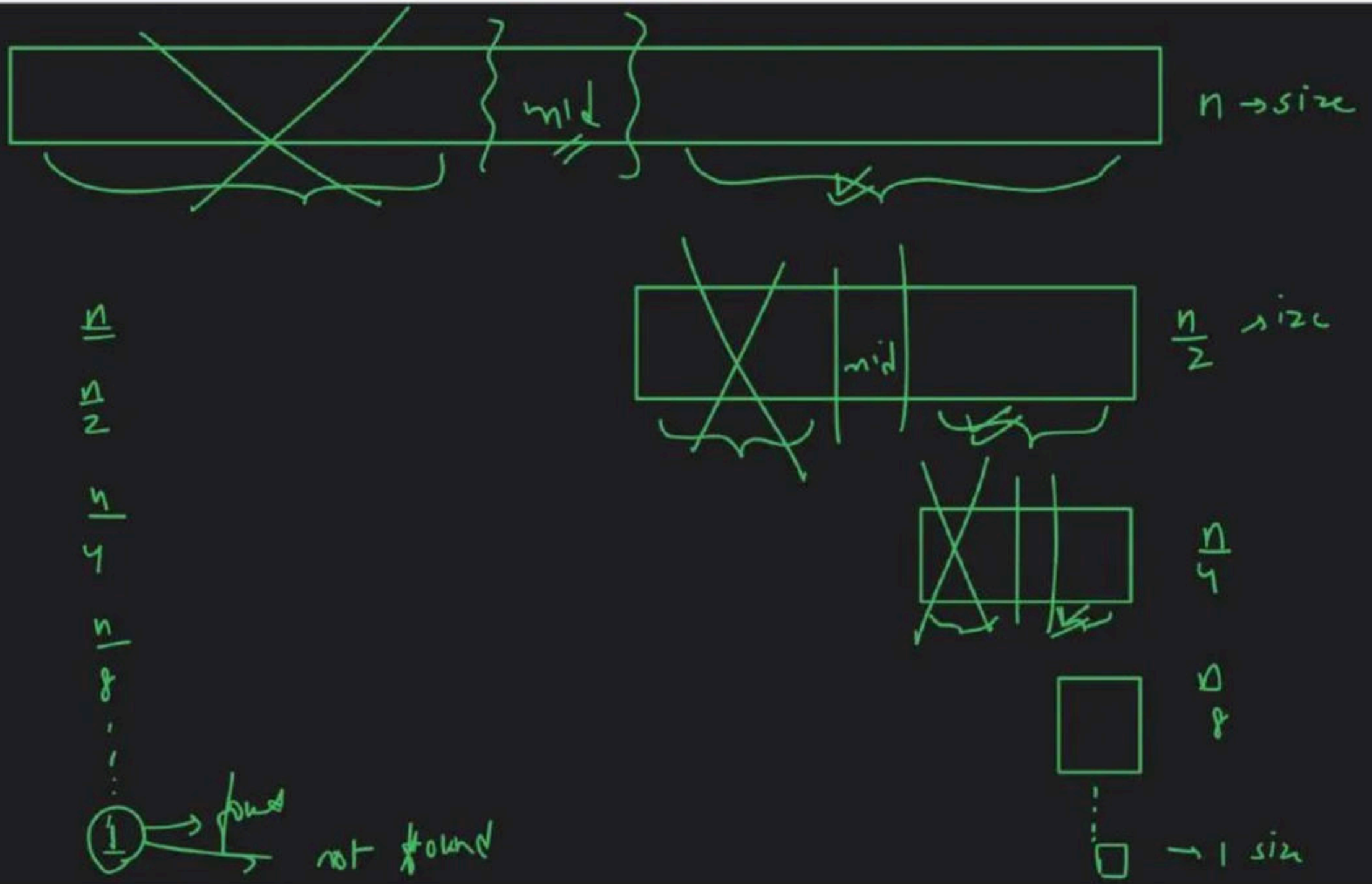
Binary Search → condition → monotonic function

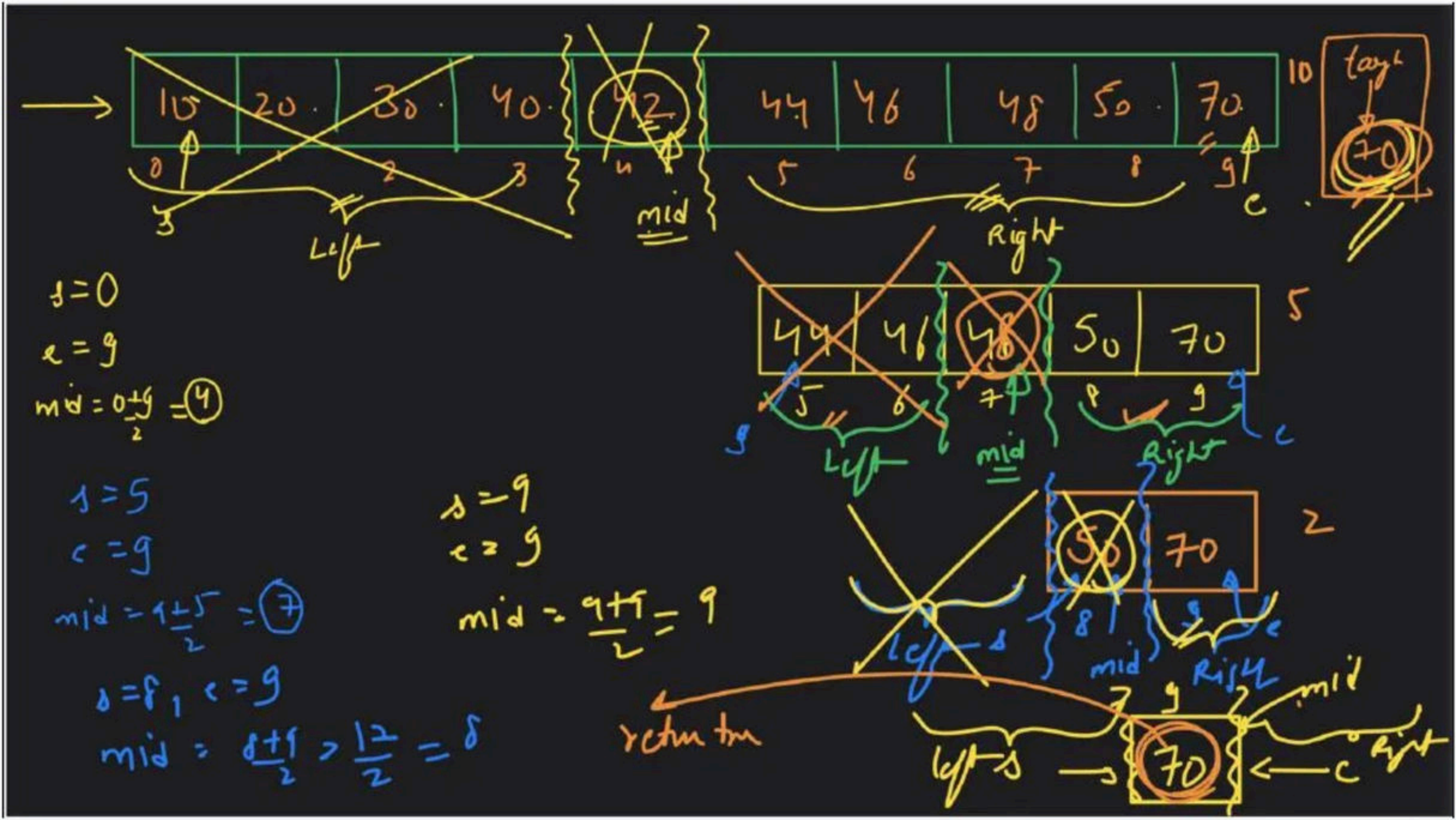
- Linear Search

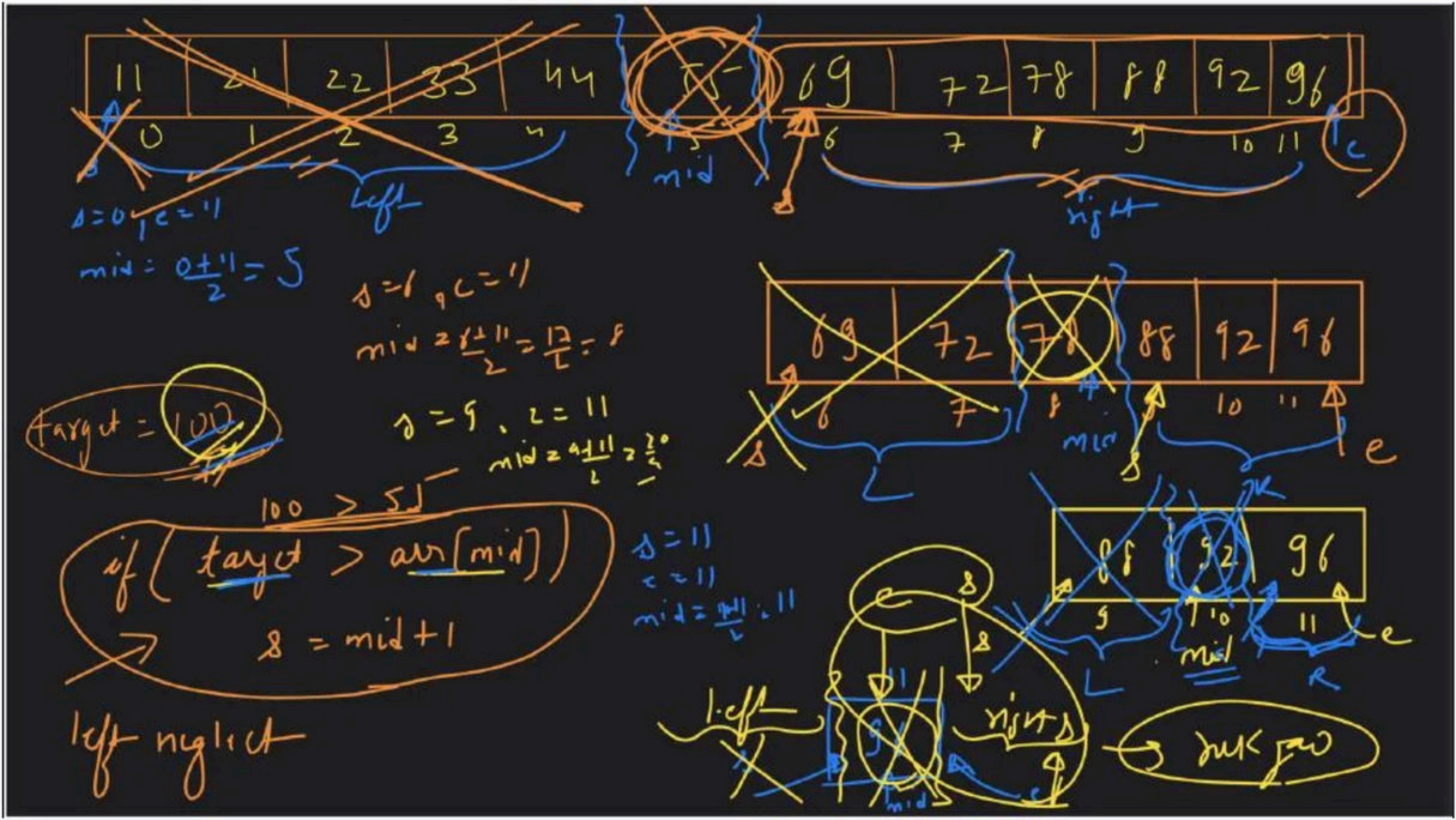





Inc order
sorted array
i/p →
2
8
10
20
30
34
46
78
0
1
2
3
4
5
6
7
start
end
mid
L
R
Left
right
target/key
46
s = 0
e = 7
mid = s+e/2 = 7/2 = 3
s = 4
e = 7
mid = 11/2 = 5
s = 6
e = 7
mid = 13/2 = 6
s
e
mid
L
R
return true
if (arr[mid] == target)
return true


mid
n → size
mid
n/2 size
n/4
n/8
1
found
not found
1 size


10
20
30
40
42
44
46
48
50
70
0
1
2
3
4
5
6
7
8
9
target
70
mid
Left
Right
s=0
e=9
mid = 0+9/2 = 4
s=5
e=9
mid = 5+9/2 = 7
s=8, e=9
mid = 8+9/2 = 8
s=9
e=9
mid = 9+9/2 = 9
return true
5
2

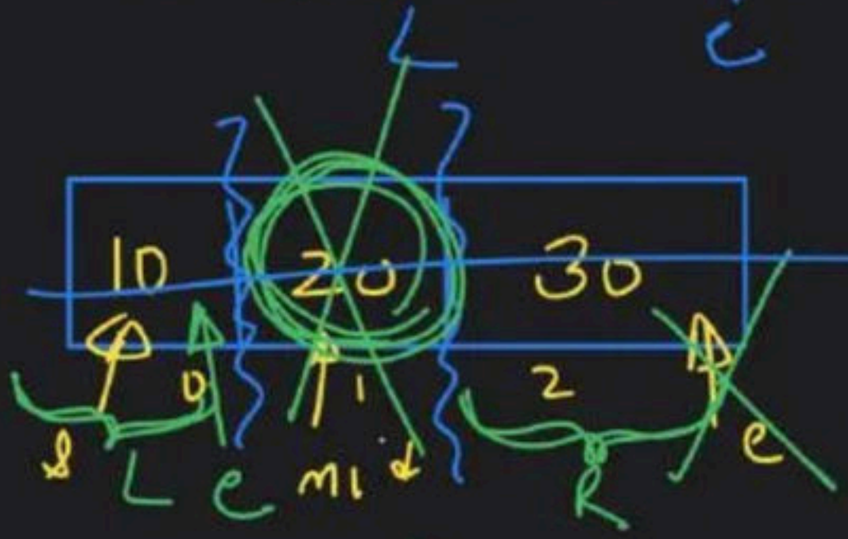

11 21 22 33 44 55 69 72 78 88 92 96
0 1 2 3 4 5 6 7 8 9 10 11
s=0, e=11
mid = 0+11/2 = 5
left
right
s=1, e=11
mid = 1+11/2 = 12/2 = 6
s=9, e=11
mid = 9+11/2 = 20/2
s=11
e=11
mid = 11+11/2
target = 100
if (target > arr[mid])
s = mid+1
left right
69 72 78 88 92 96
6 7 8 9 10 11
mid
88 92 96
9 10 11
mid
L
R
left
right
mid
dekho

```
if (arr[mid] == target)
    return true;

if (target > arr[mid])
{
    // Right me jao
    s = mid + 1;
}
```

```
if (target < arr[mid])
{
    // Left me jao
    e = mid - 1;
}
```





s = 0 , e = 2

$mid = \frac{0+2}{2} = 1$

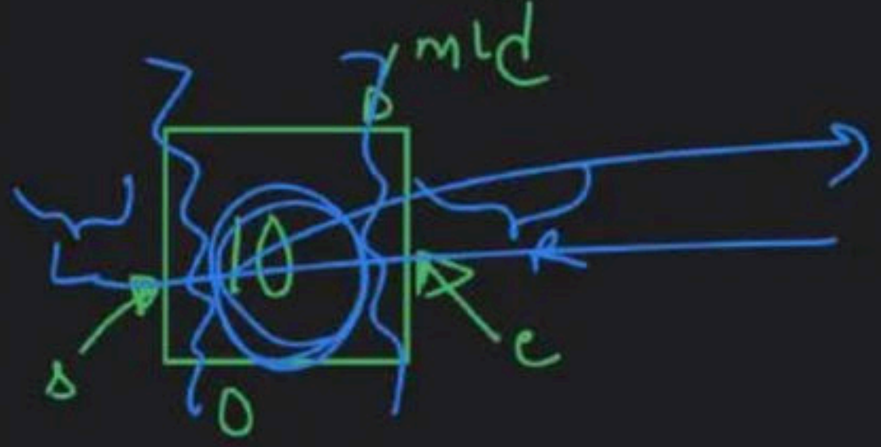
s = 0 , e = 0

$mid = \frac{0+0}{2} = 0$

target = 10

s = 0 , e = 7

$mid = \frac{0+7}{2} = 3$



return true

invalid arr → s > e

return kar

if (target < arr[mid])

e = mid - 1

s <= e

left me jao

s > e → invalid array → ruka hai

s <= e → valid array → chalna hai

if (arr[mid] == target) → return true;

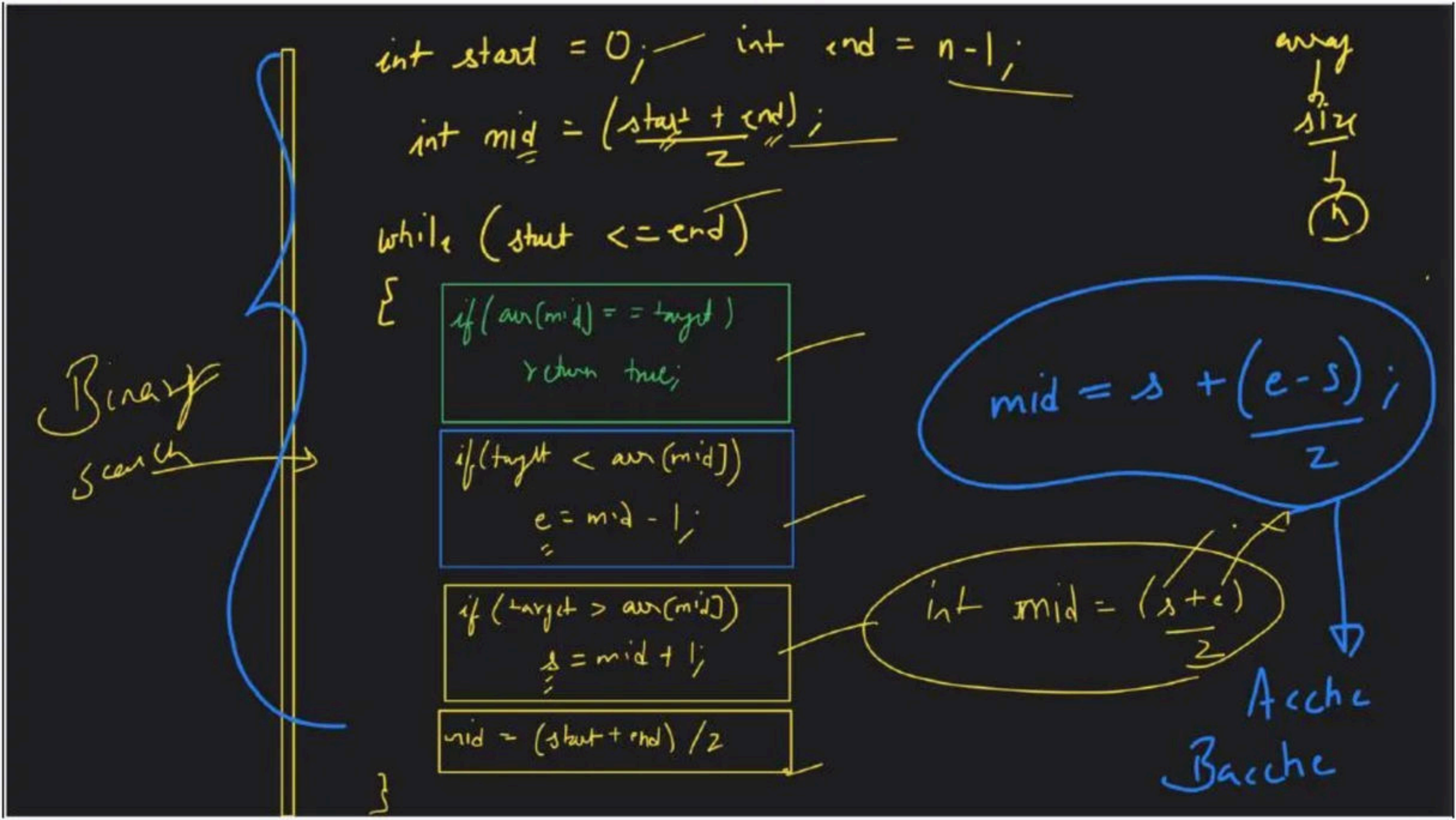
if (target > arr[mid]) → right me jao

s = mid + 1

if (target < arr[mid]) → left me jao

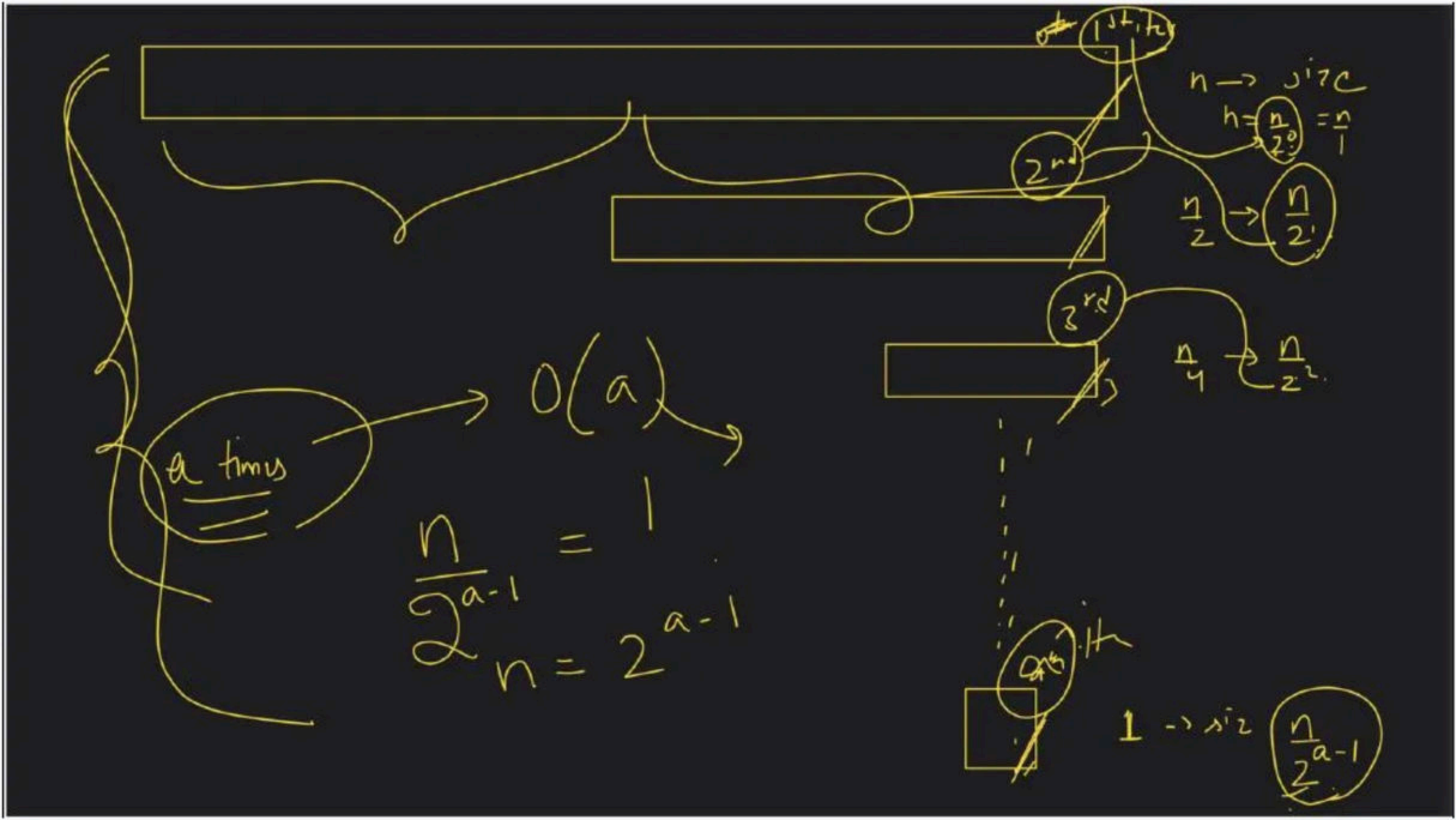
e = mid - 1

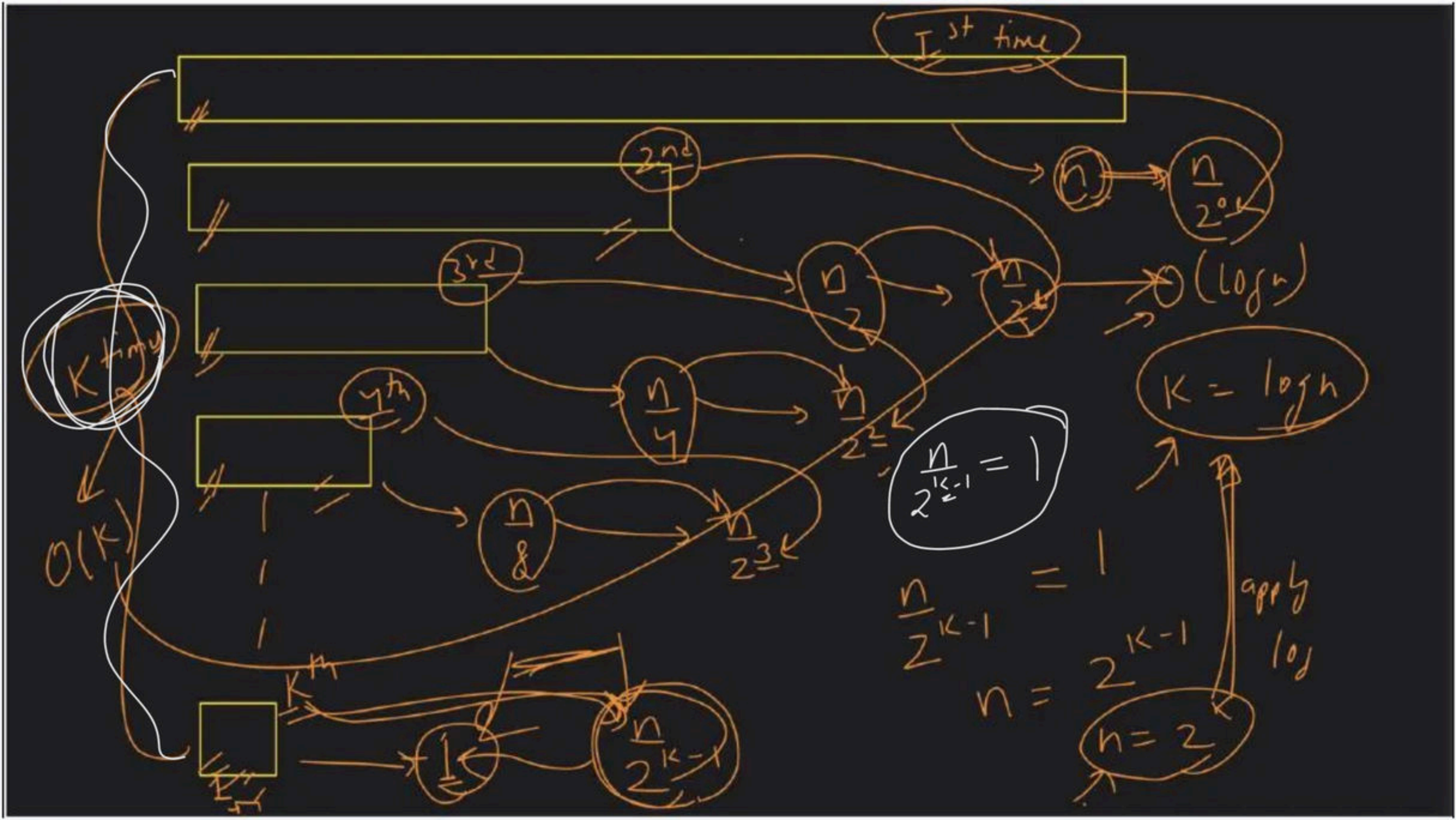
assumed → inc order array

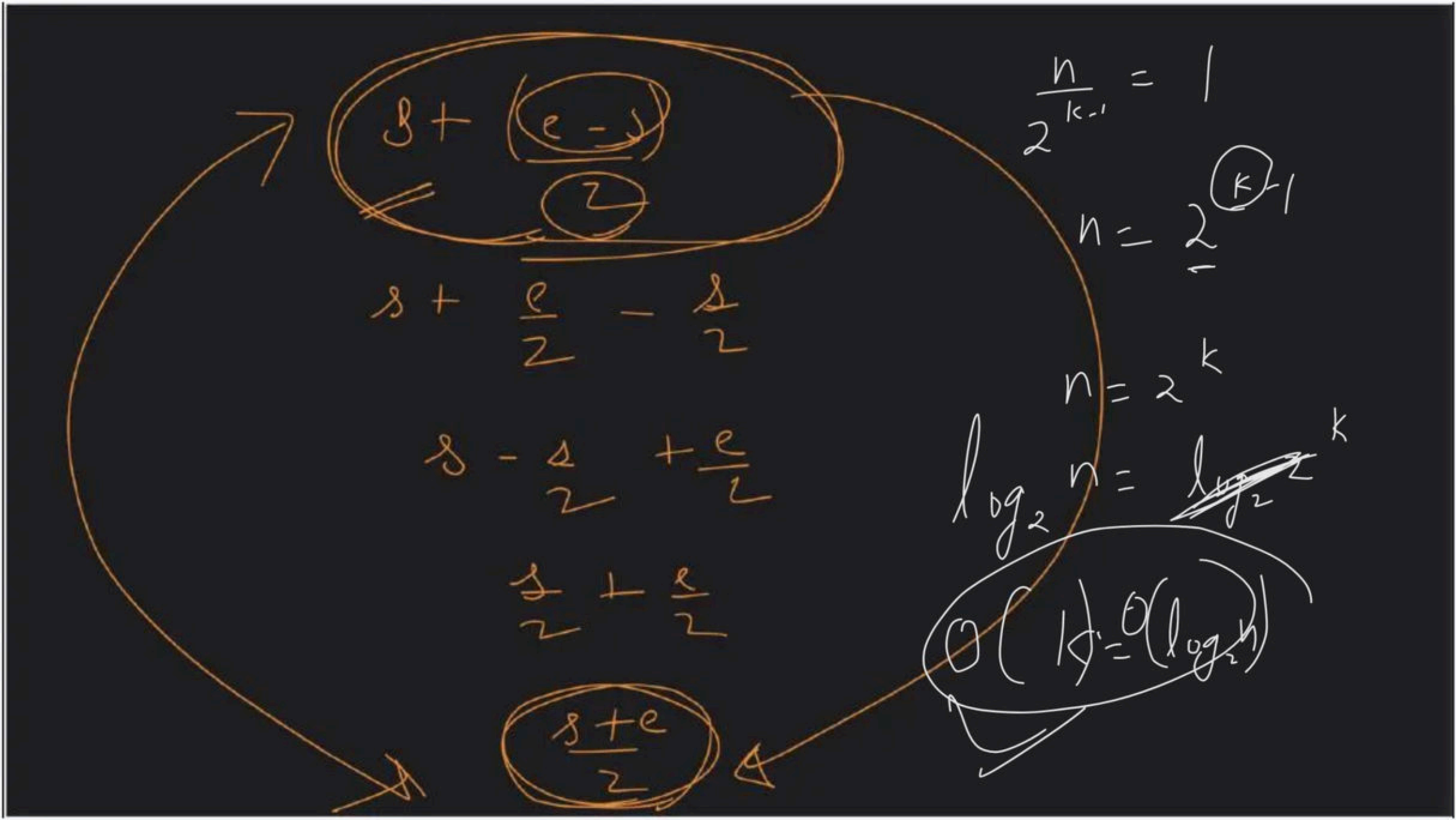

int start = 0; int end = n-1;
int mid = (start + end)/2;
while (start <= end)
{
if (arr[mid] == target)
return true;
if (target < arr[mid])
e = mid - 1;
if (target > arr[mid])
s = mid + 1;
mid = (start + end) /2
}
Binary search
array
size
n
mid = s + (e-s)/2;
int mid = (s+e)/2
Accha
Bacche

int mid = (s + e)

int mid = s + (e - s) / 2


1st th
n -> size
h = n/2^0 = n/1
2nd
n/2 -> n/2^1
3rd
n/4 -> n/2^2
a times
O(a)
1
n/2^(a-1) = 1
n = 2^(a-1)
ath
1 -> size
n/2^(a-1)


1st time
2nd
3rd
kth
n
n/2
n/4
n/8
n/2^3
n/2^k
(log n)
O(k)
k = log n
n/2^(k-1) = 1
n/2^(k-1) = 1
n = 2^(k-1)
apply log
n = 2
k
1
n/2^(k-1)


s + (e - s)/2
s + e/2 - s/2
s - s/2 + e/2
s/2 + e/2
(s+e)/2
n/2^(k-1) = 1
n = 2^(k-1)
n = 2^k
log2 n = log2 2^k
O(k) = O(log2 n)

$n = 2^{a-1}$

$n = 2^{a-1}$

$\rightarrow n = 2^{\uparrow}$

apply log

$\log_2 n = \log_2(2^a)$

$\log n = a$

$O(a)$

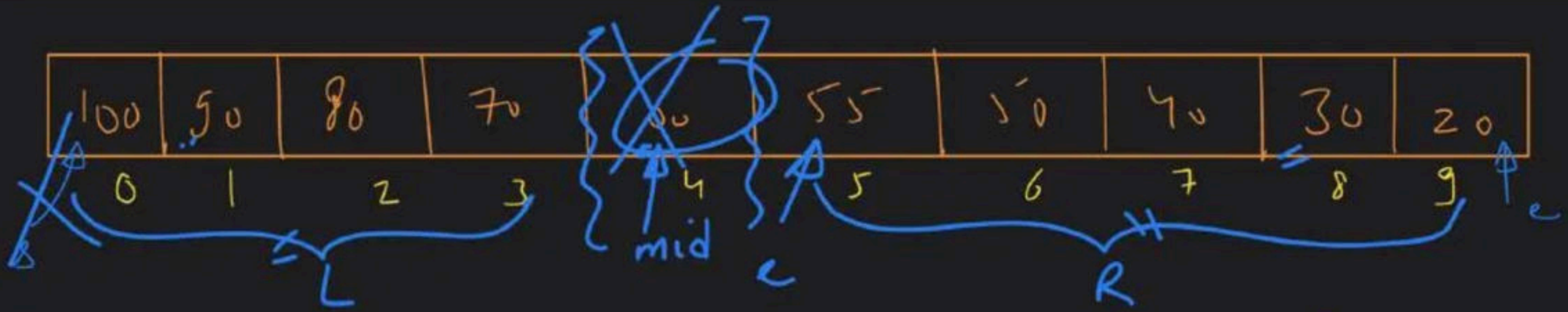
$O(\lg n)$

$\Gamma \cdot (\rightarrow$



10	$n = 10$
5	$\frac{n}{2} = 5$
2	$\frac{n}{4} = 2$
1	$\frac{n}{8} = 1$

invalid array → $s > c$



target

30

s = 0

e = 9

$mid = \frac{9}{2} = 4$

decreasing order

dec order

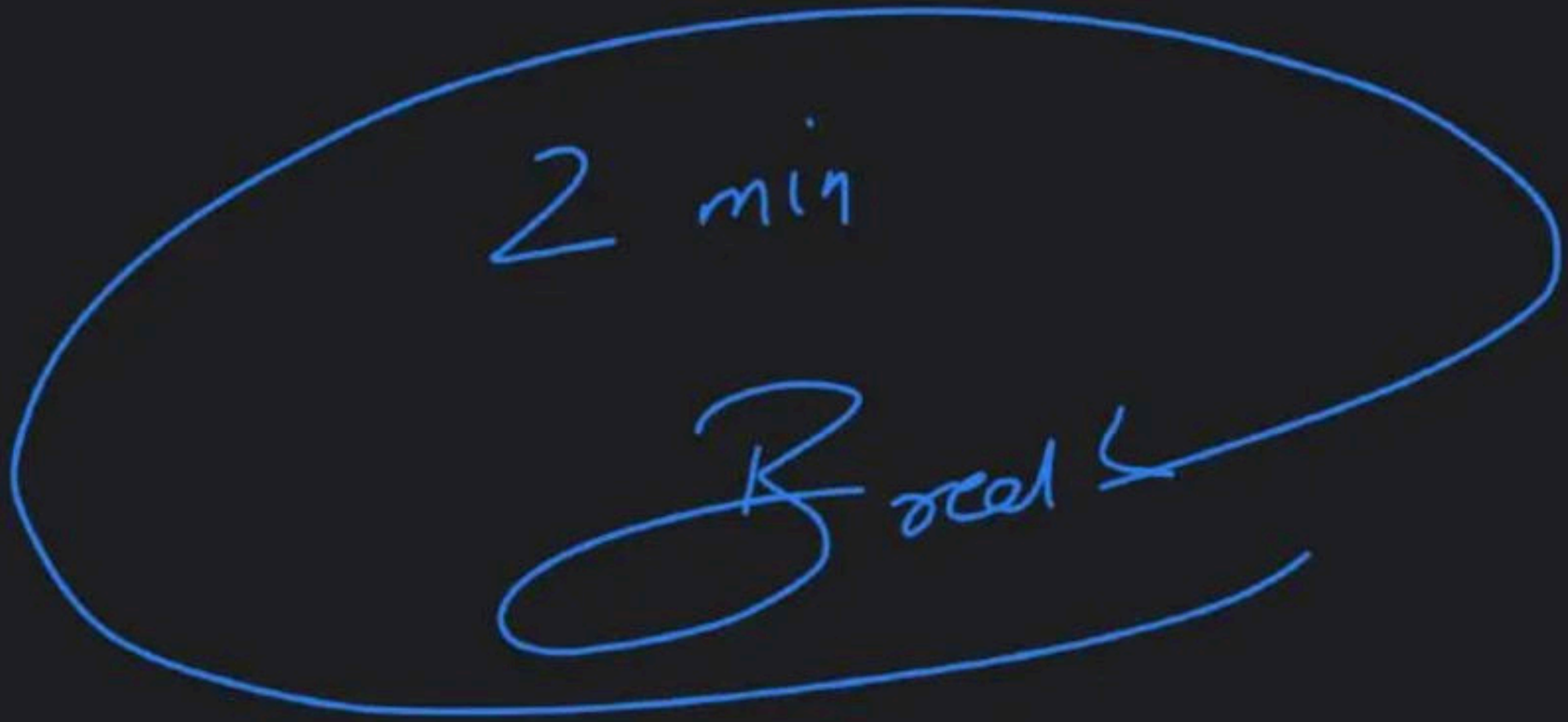
if (target > arr[mid]

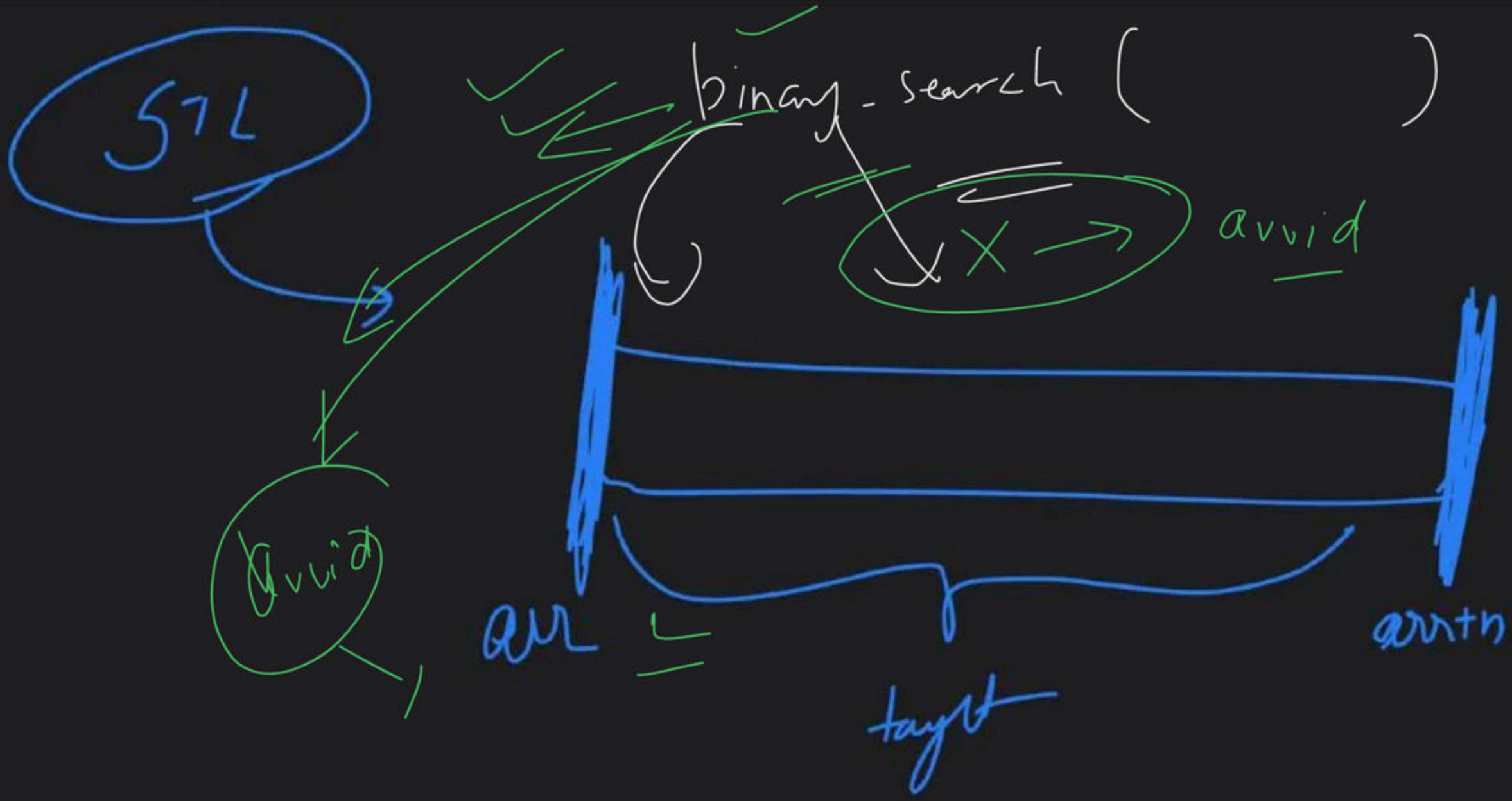
→ left → e = mid - 1

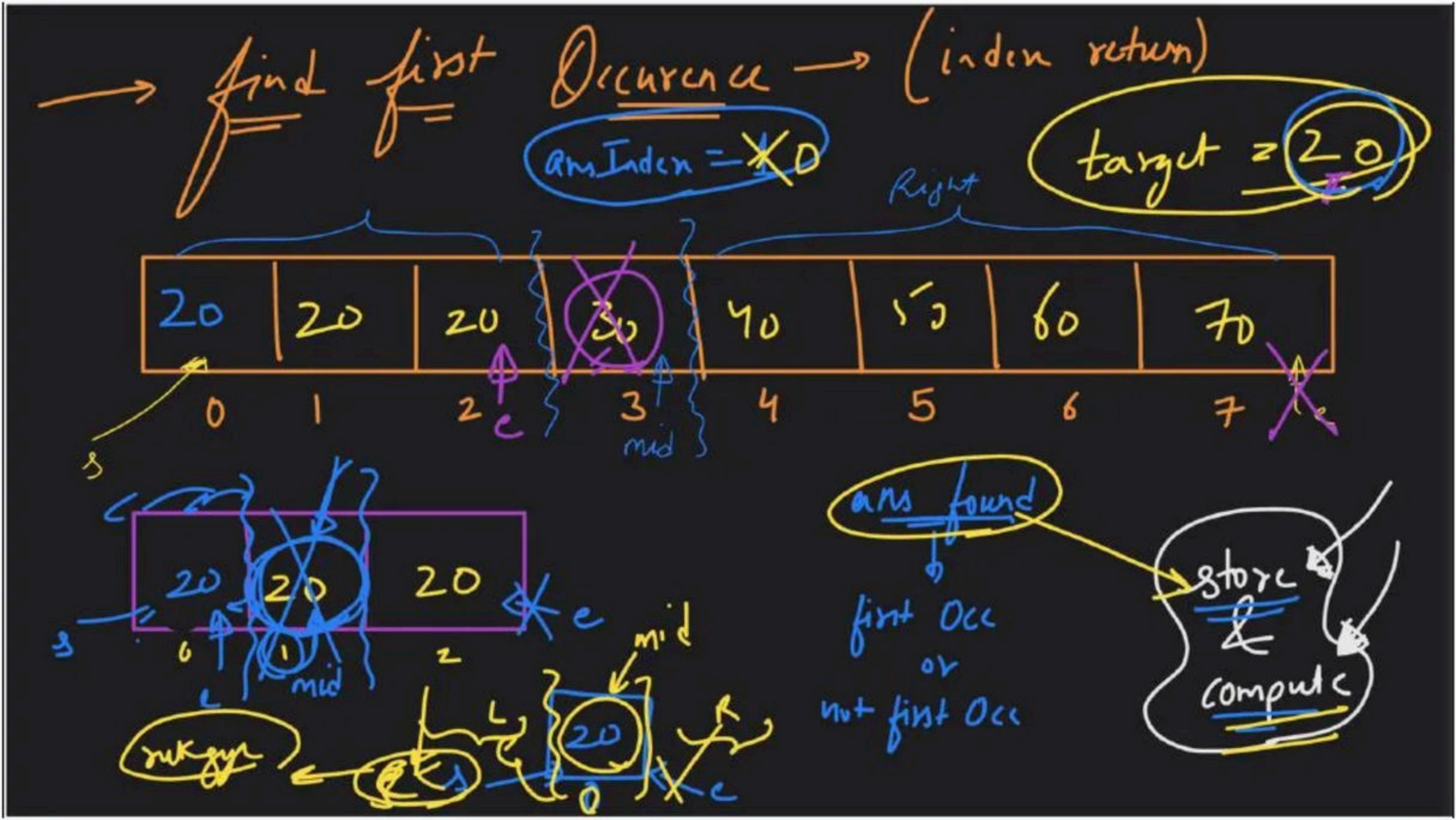
dec order

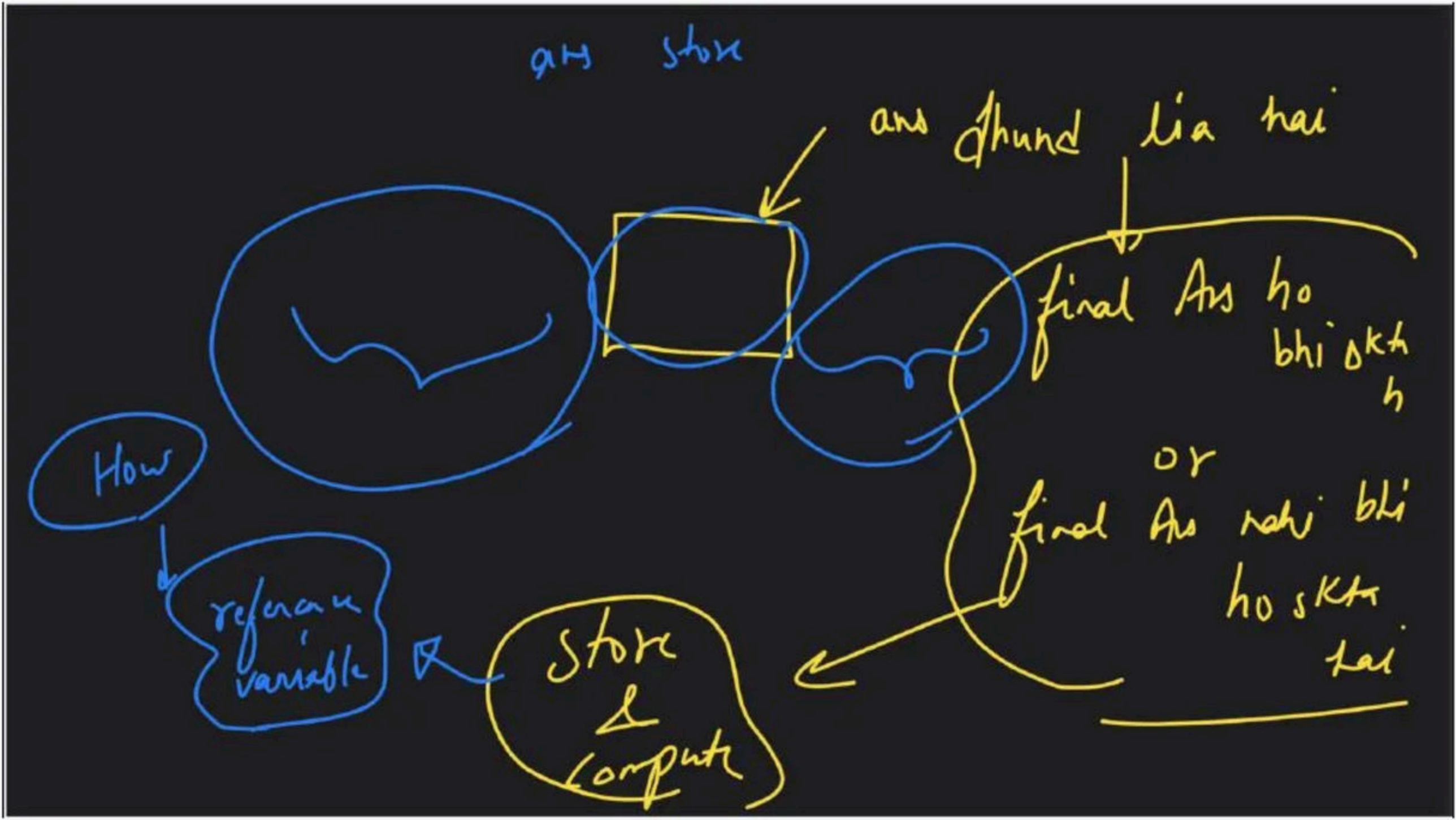
if (target < arr[mid]

→ right → s = mid + 1


2 min
Break


STL
binary_search ( )
vector
auto
arr
arr+n
target

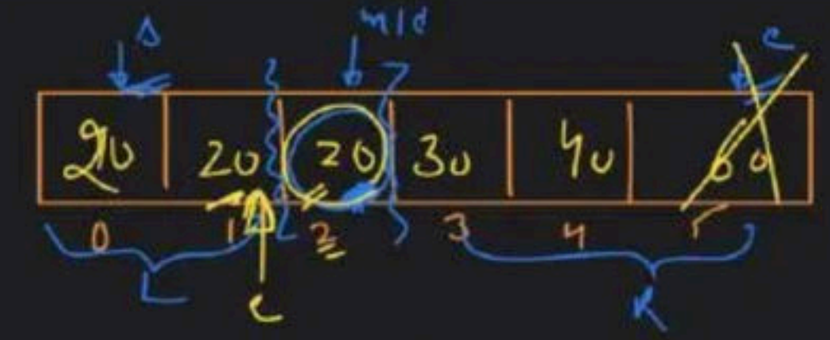

find first Occurrence → (index return)
ansIndex = -1
target = 20
Right
20 20 20 30 40 50 60 70
0 1 2 3 4 5 6 7
s
e
mid
20 20 20
s
e
mid
ans found
first Occ
or
not first Occ
store
compute


ans store
ans dhund lia hai
final Ans ho bhi skta h
or
final Ans nahi bhi ho skta hai
How
reference variable
Store & compute

```
void findFirstOcc(arr, n, target, &ansIndex)
{ int s=0, e=n-1; int mid = s+(e-s)/2;
  while(s<=e)
  {
    if(target == arr[mid])
    { ansIndex = mid;
      e = mid-1;
    }
    if(target > arr[mid])
      s = mid+1
    if(target < arr[mid])
      e = mid-1
    mid = s+(e-s)/2;
  }
```

change →

Always or may not be first occurence → Store & compute

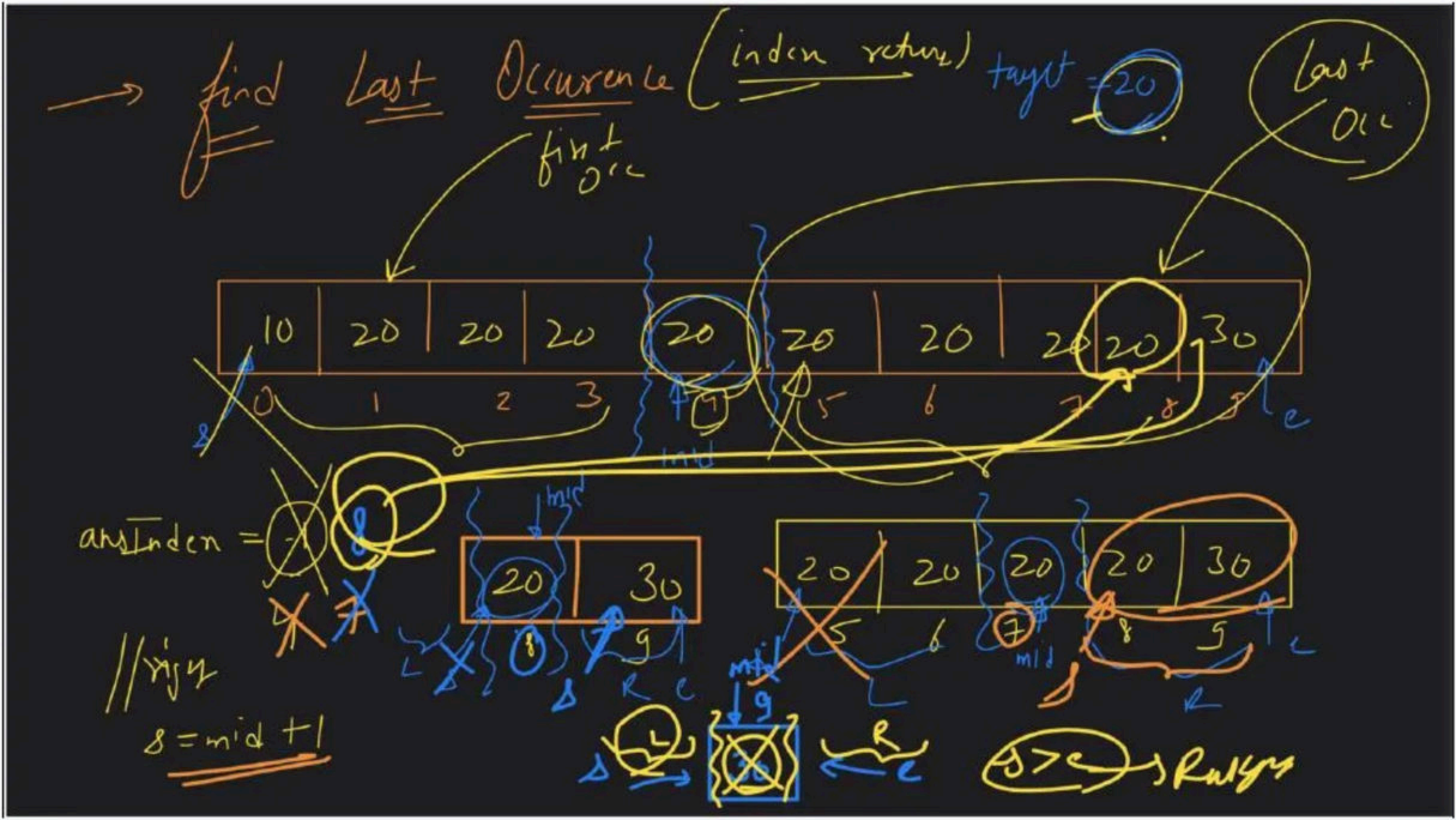


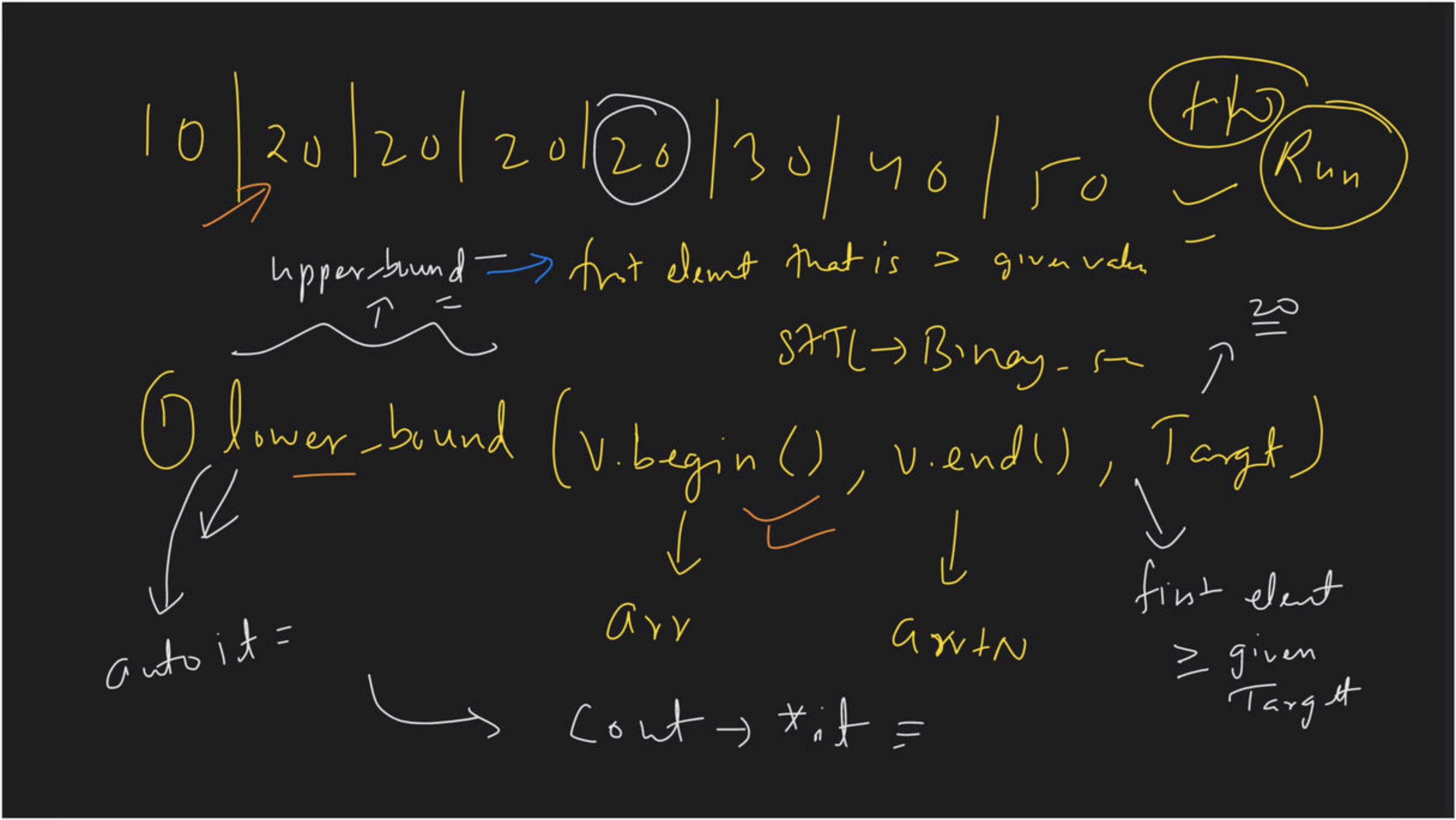


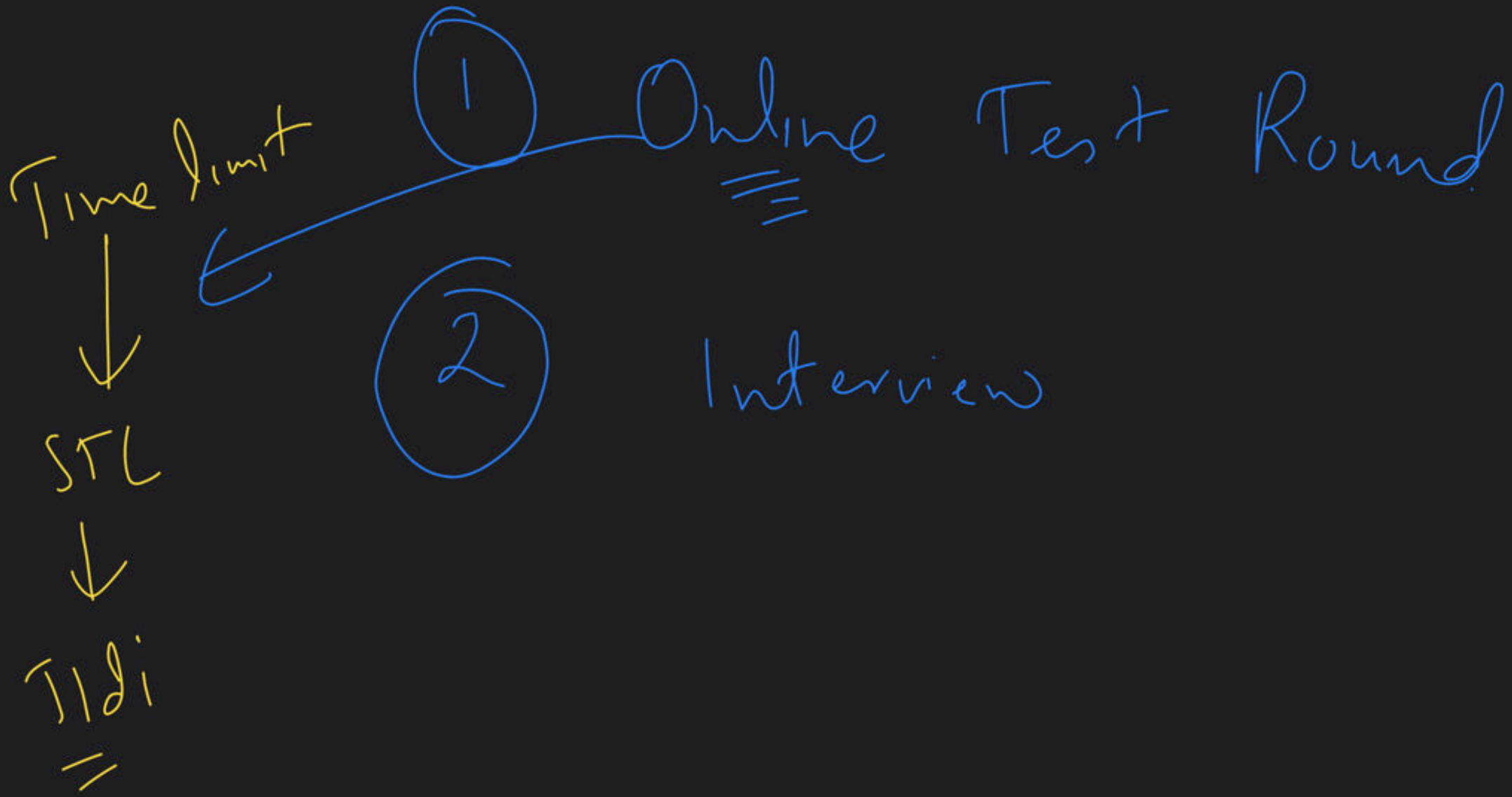


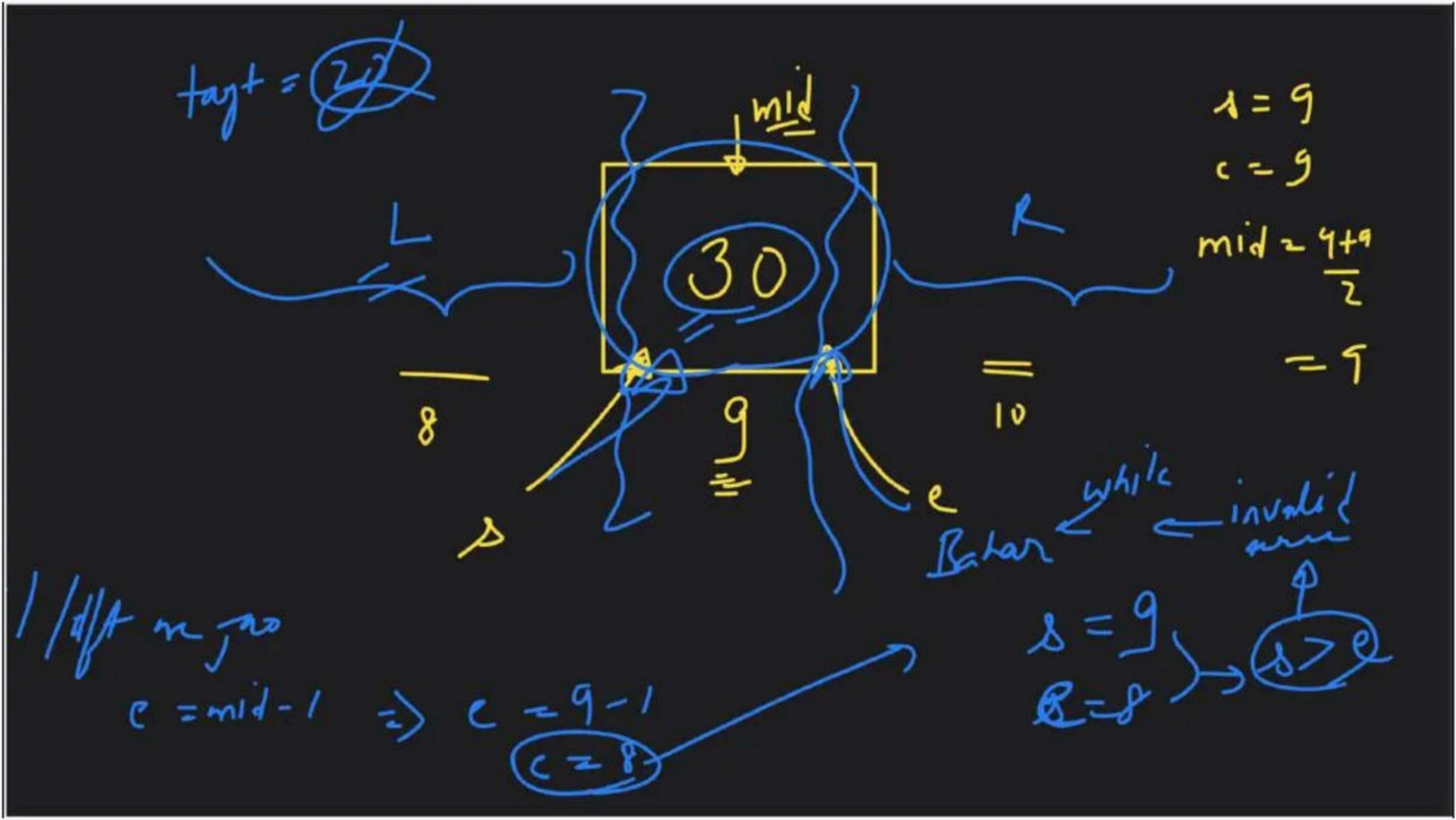


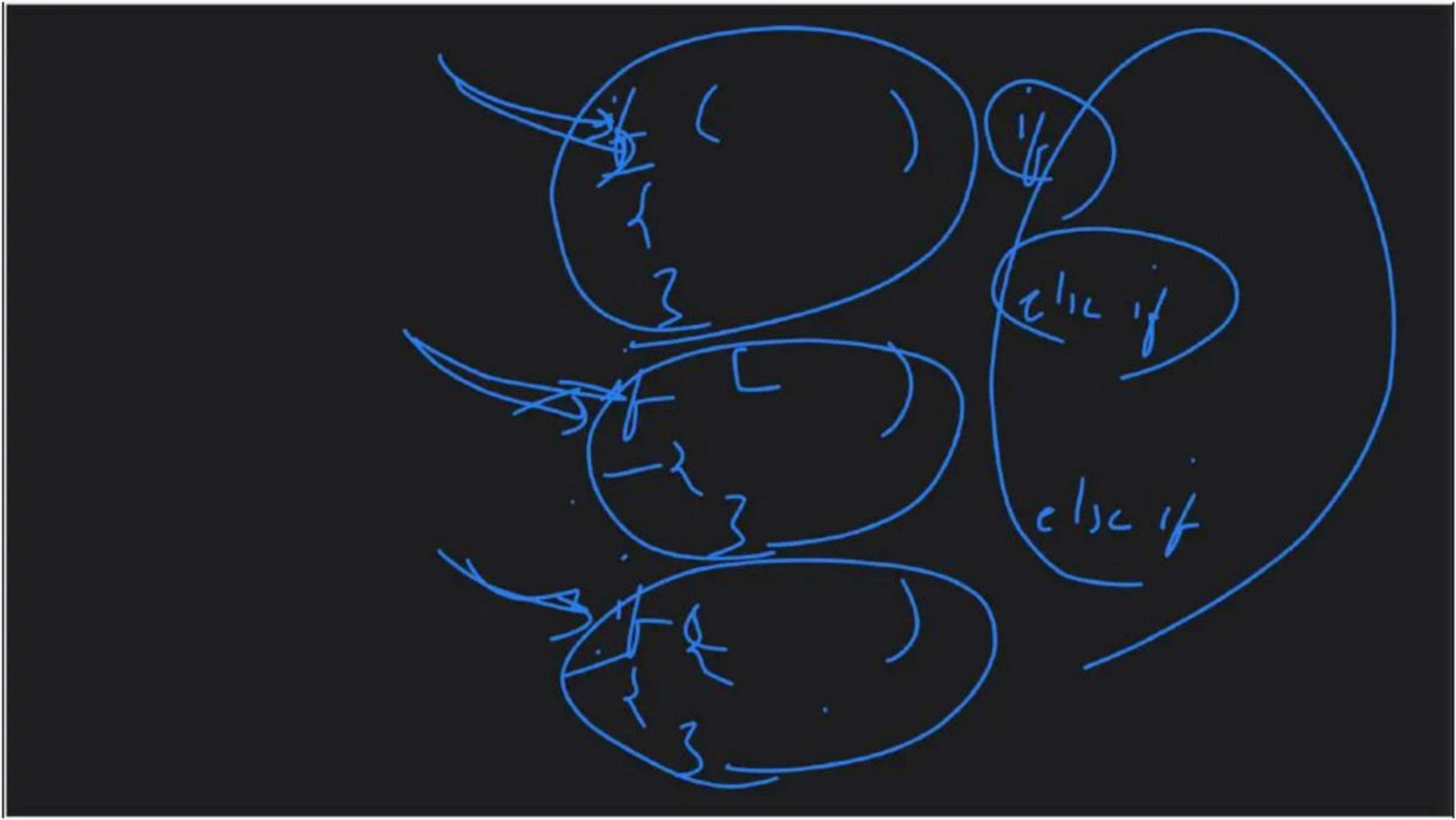
nuk gya

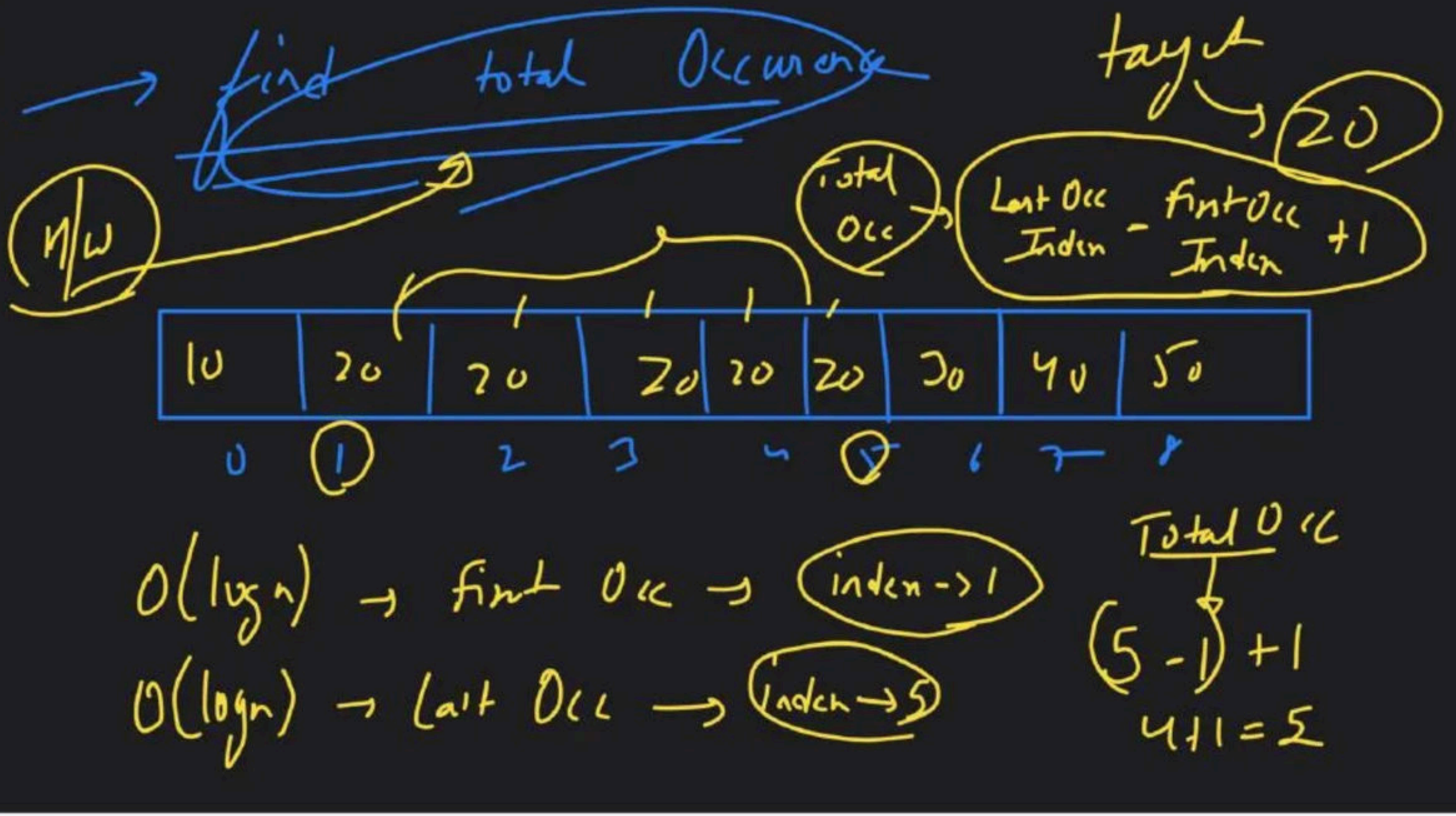

find Last Occurence (index return)
first Occ
target = 20
Last Occ
10 20 20 20 20 20 20 20 20 30
0 1 2 3 4 5 6 7 8 9
ansIndex = -1
mid
20 30
20 20 20 20 30
s = mid + 1
Right

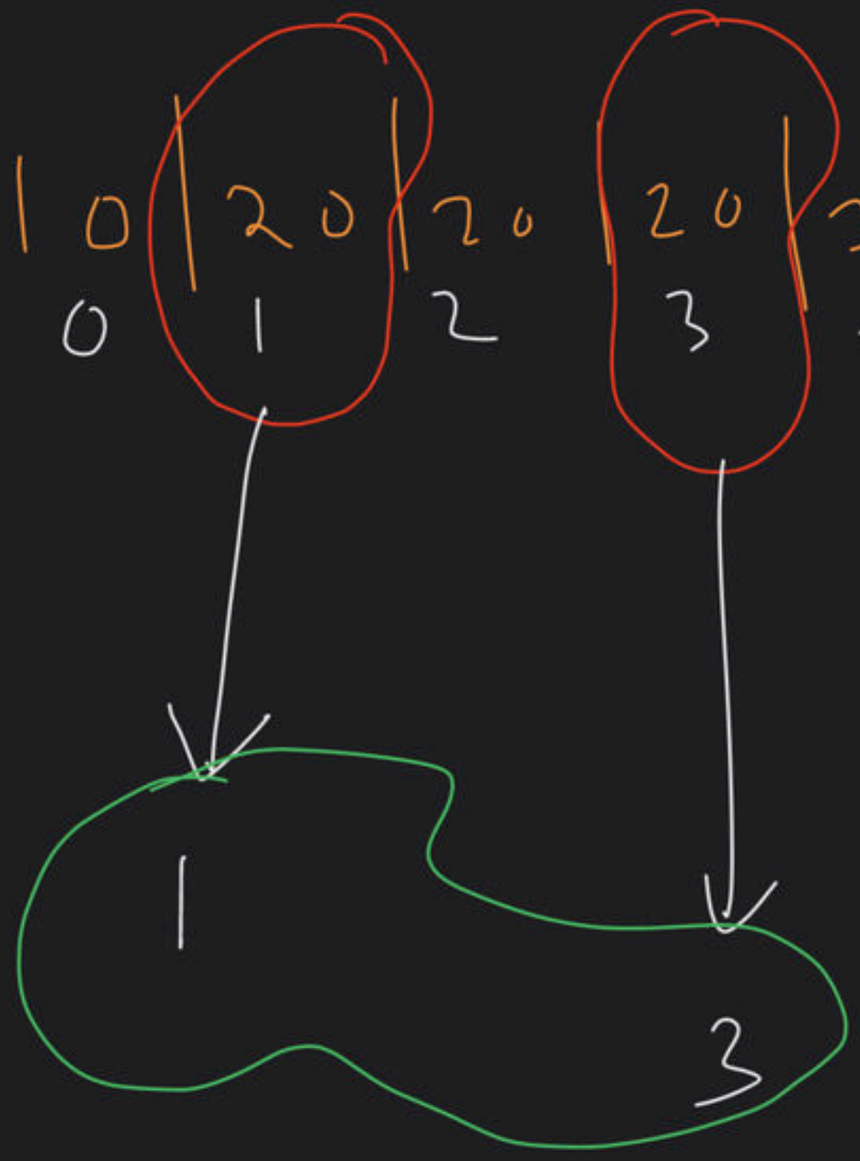

10 | 20 | 20 | 20 | 20 | 30 | 40 | 50
thm
Run
upper_bound → first element that is > given value
STL → Binary_s
≥
lower_bound (v.begin() , v.end() , Target)
arr
arr+N
first element ≥ given Target
auto it =
cout → *it =


Time limit
1
Online Test Round
2
Interview
STL
JIdi


tayt = 27
mid
30
L
R
8
9
10
s
e
s = 9
c = 9
mid = 9+9/2
= 9
Batas
while
invalid
s > e
// dft mnjd
c = mid - 1
=> c = 9 - 1
c = 8


if
else if
else if


find total Occurrence
target
20
N/W
total Occ
Last Occ Index - first Occ Index +1
10 20 20 20 20 20 30 40 50
0 1 2 3 4 5 6 7 8
O(log n) → first Occ → index -> 1
O(log n) → Last Occ → index → 5
Total Occ
(5 - 1) +1
4+1 = 5



Given 2 index

↓

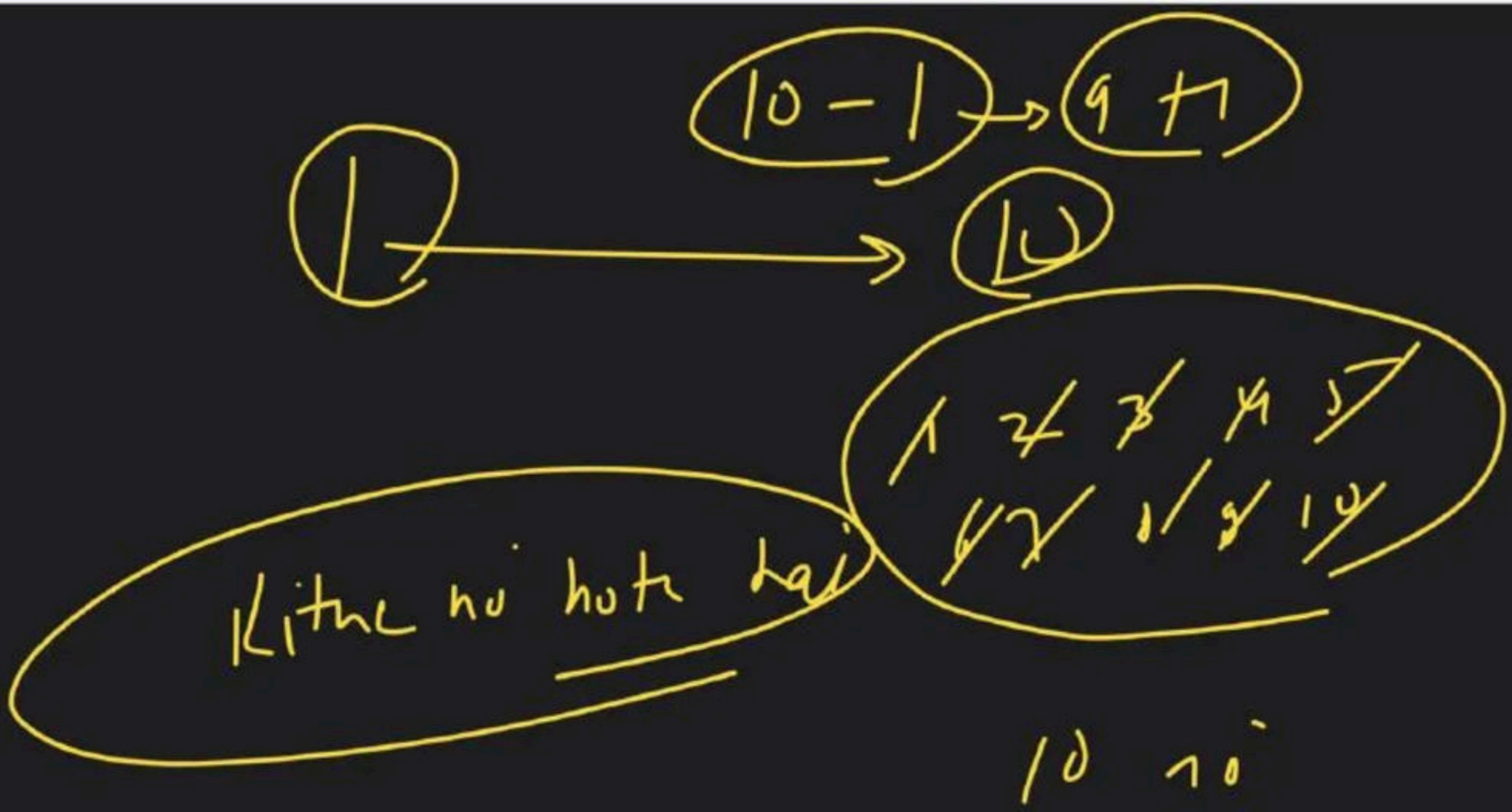
Total no. of integer

3

Last index − first index + 1

3 − 1 + 1 → No. of elements

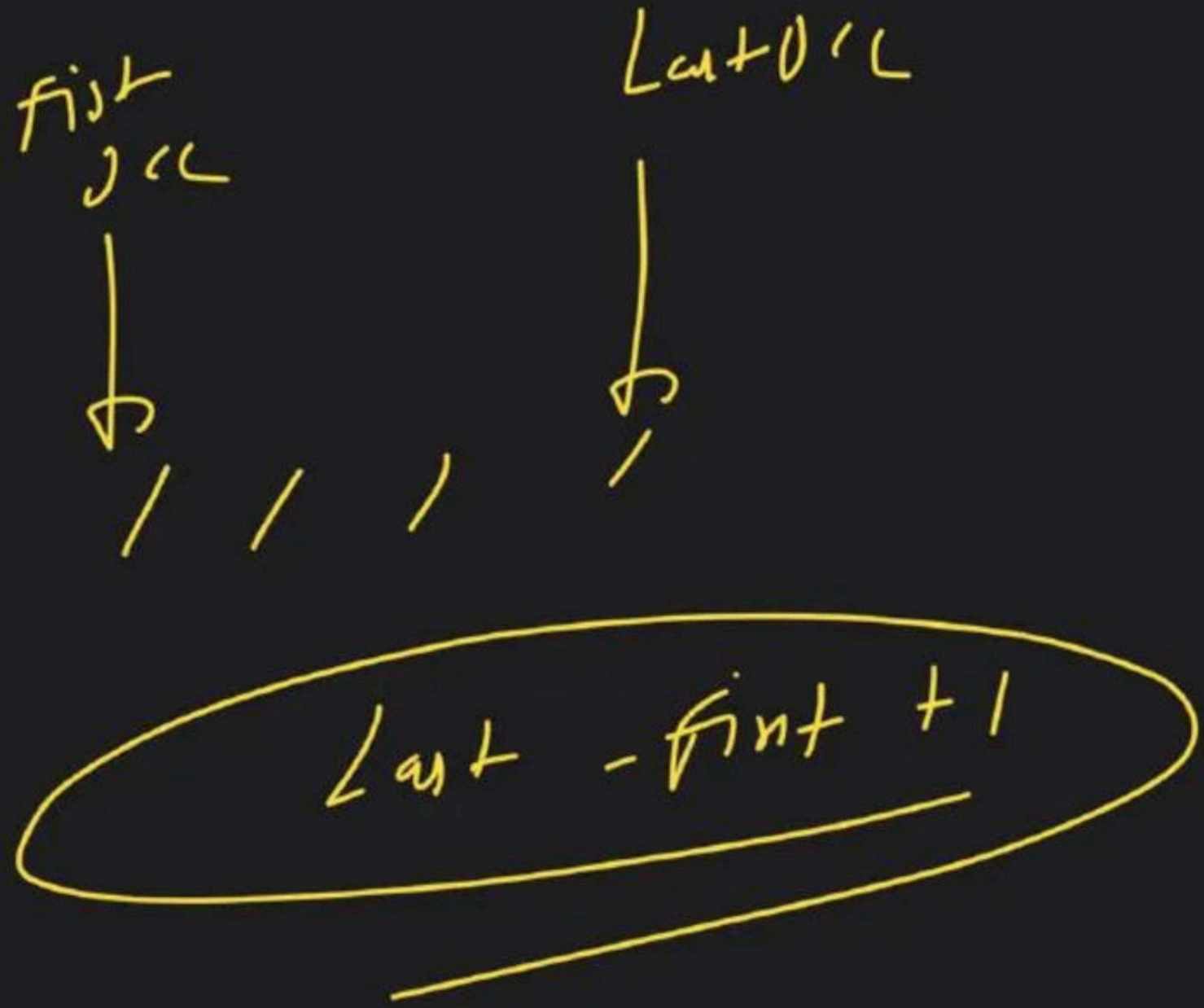

2 min
Break

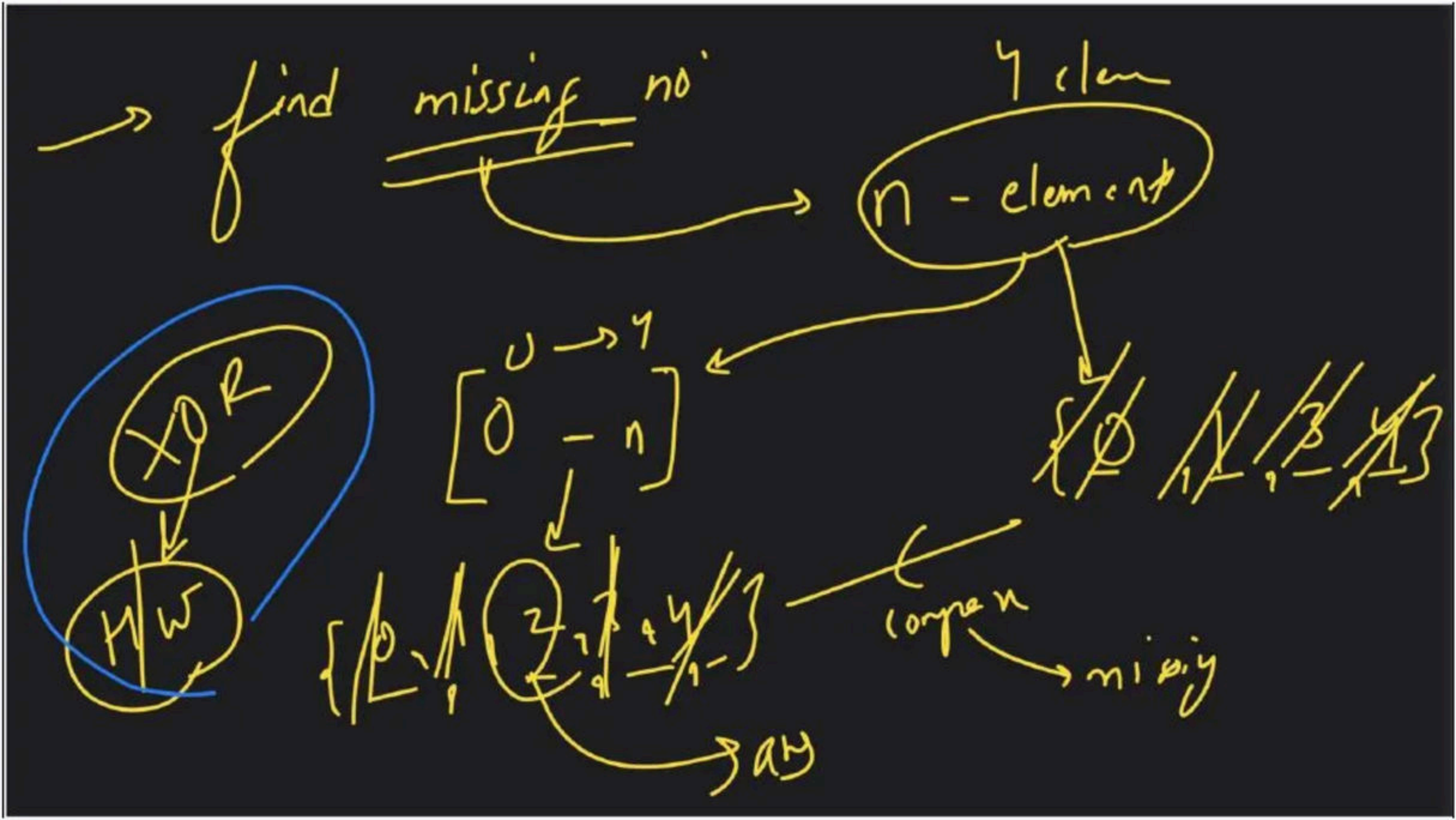

10 - 1 → 9 ती
1
Kitne nu hote hai
10 तो

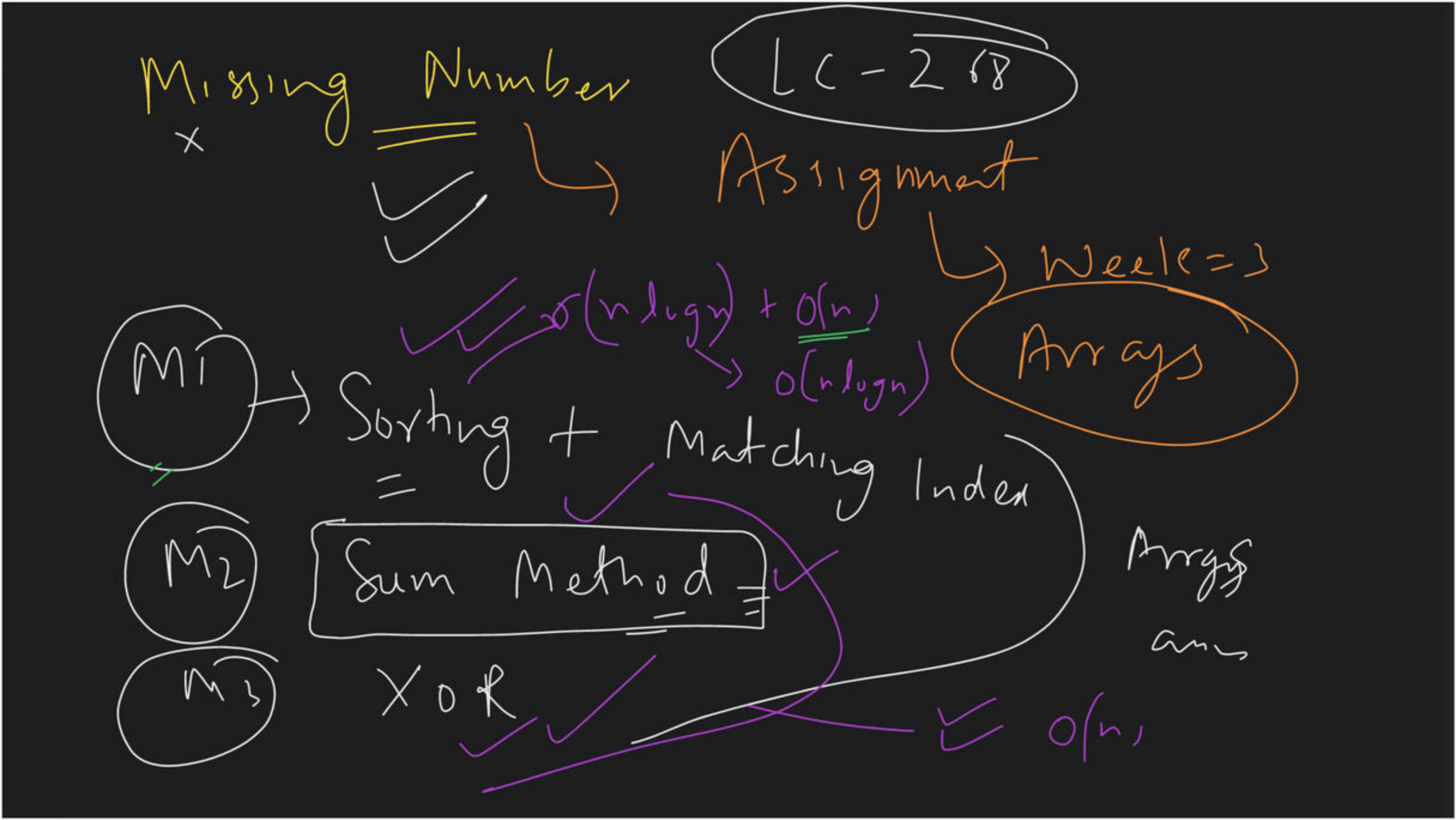
15 - 11 = 4 + 1

11 → 15

11 12 13 14 15


first occ
Last occ
Last - first + 1


find missing no
n - elements
XOR
HW
[0 - n]
compare
missing


Missing Number
LC-268
Assignment
Week-3
Arrays
M1
O(nlogn) + O(n)
O(nlogn)
Sorting + Matching Index
M2
Sum Method
M3
XOR
Arrays
O(n)

My

Sorting + Binary search → O(n log n) + O log(n) ⟹ O(n log n)



diff = 1

↳ fmt

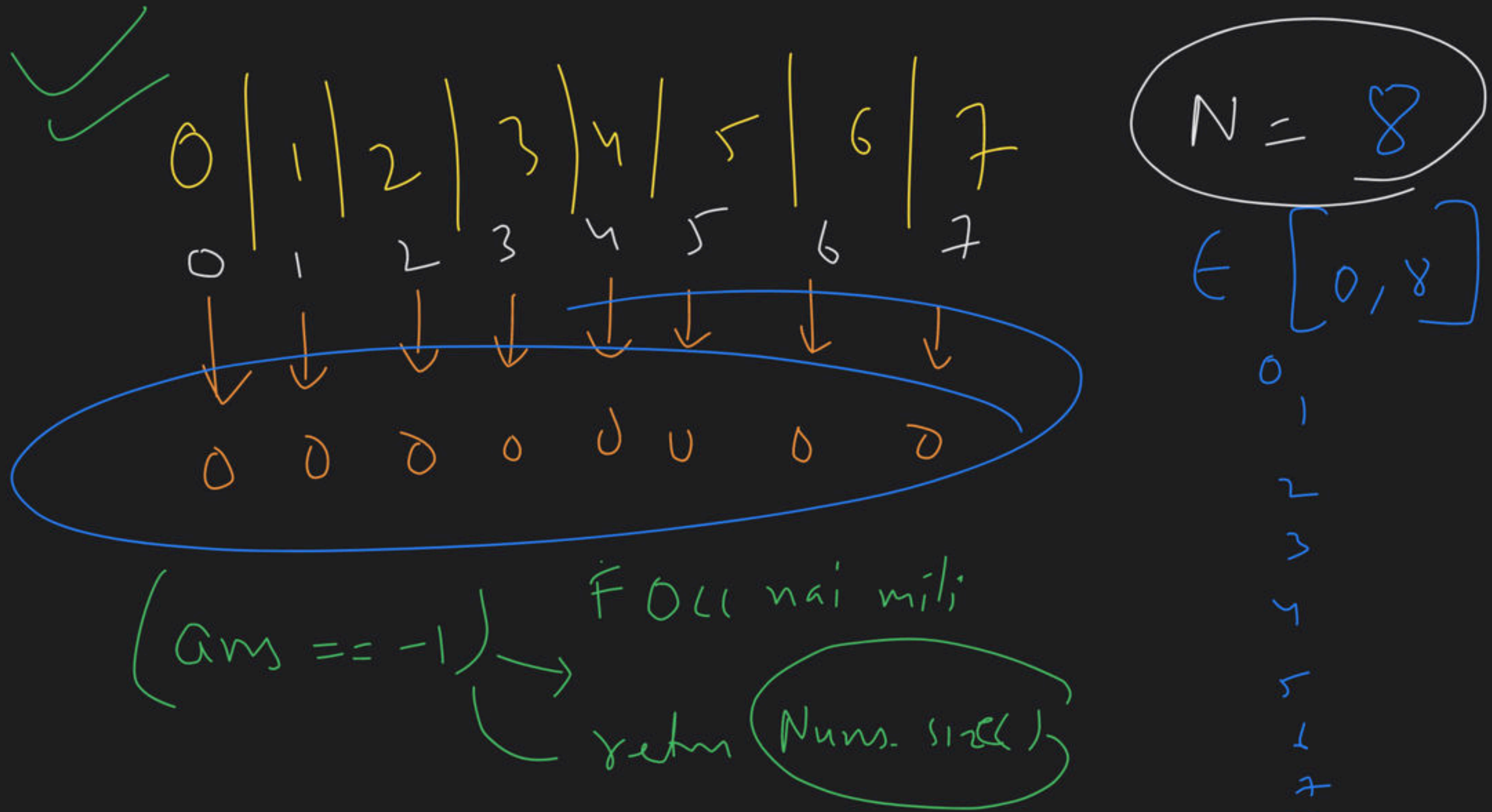
occ. nikal lo

N = 8

arr[i] ∈ [0, N]

[0, 8]

0 1 2 3 4 5 6 7 8

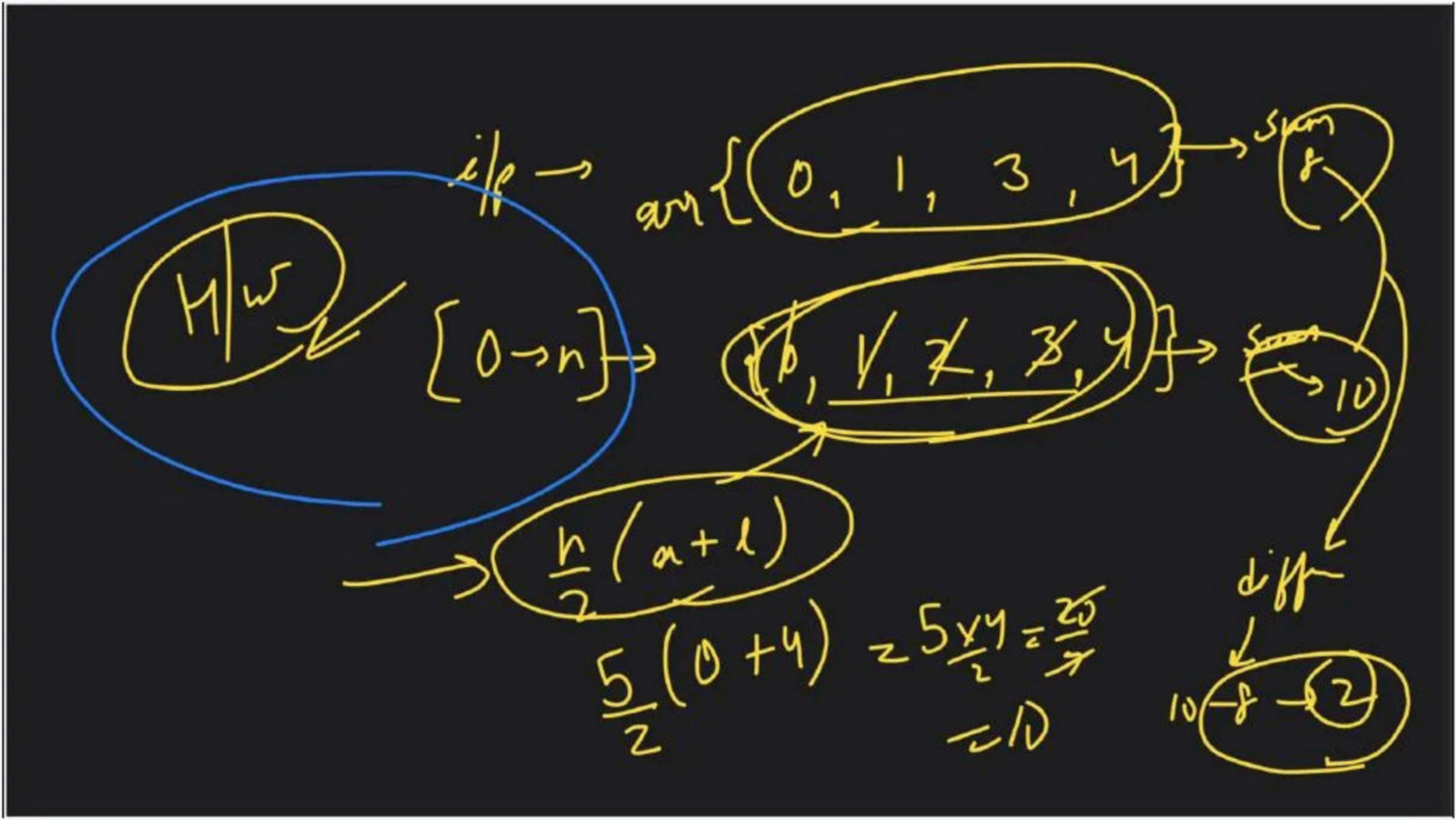

0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
0 1 2 3 4 5 6 7
0 0 0 0 0 0 0 0
N = 8
∈ [0, 8]
0 1 2 3 4 5 6 7 8
(ans == -1)
Fcll nai mili
return (Nums. size())

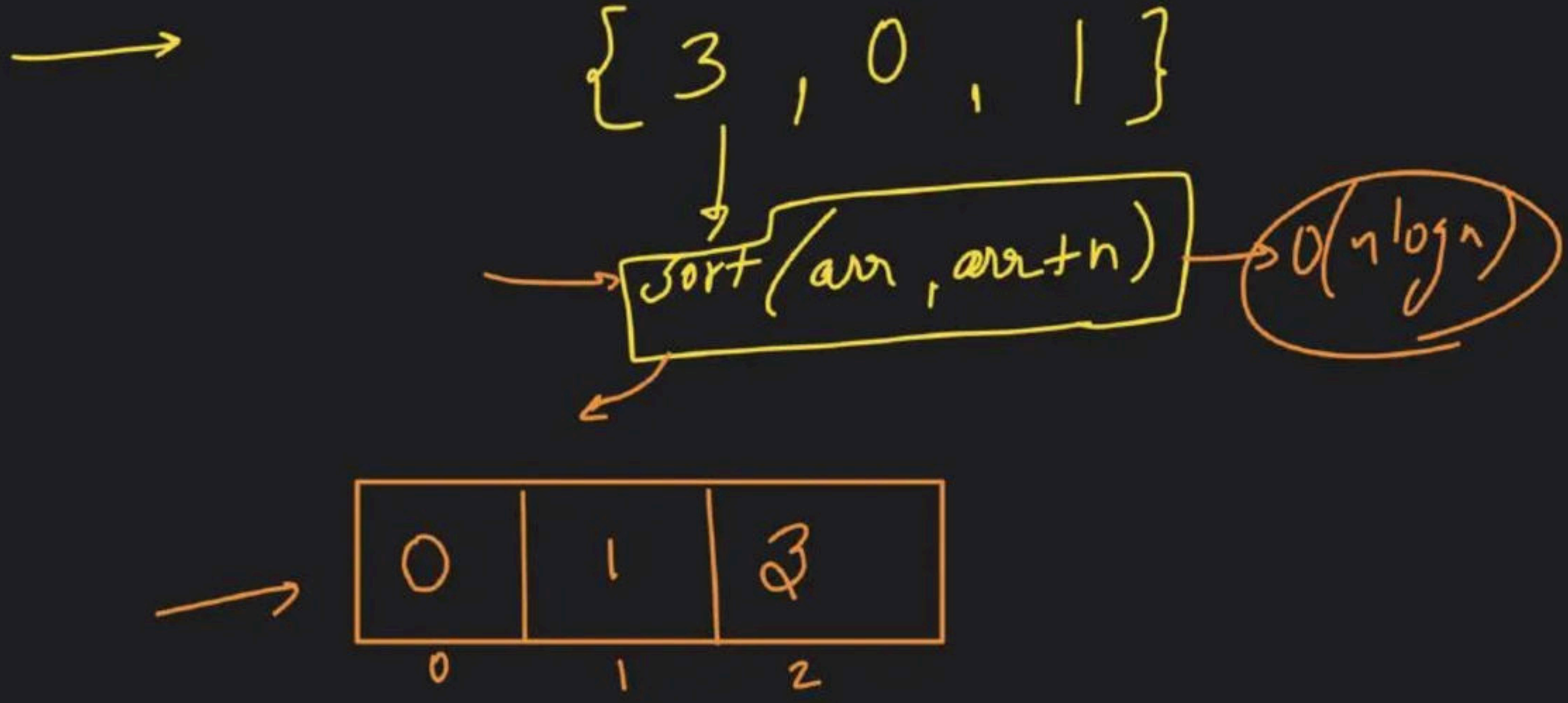
s = 0, e = n-1; ans = -1

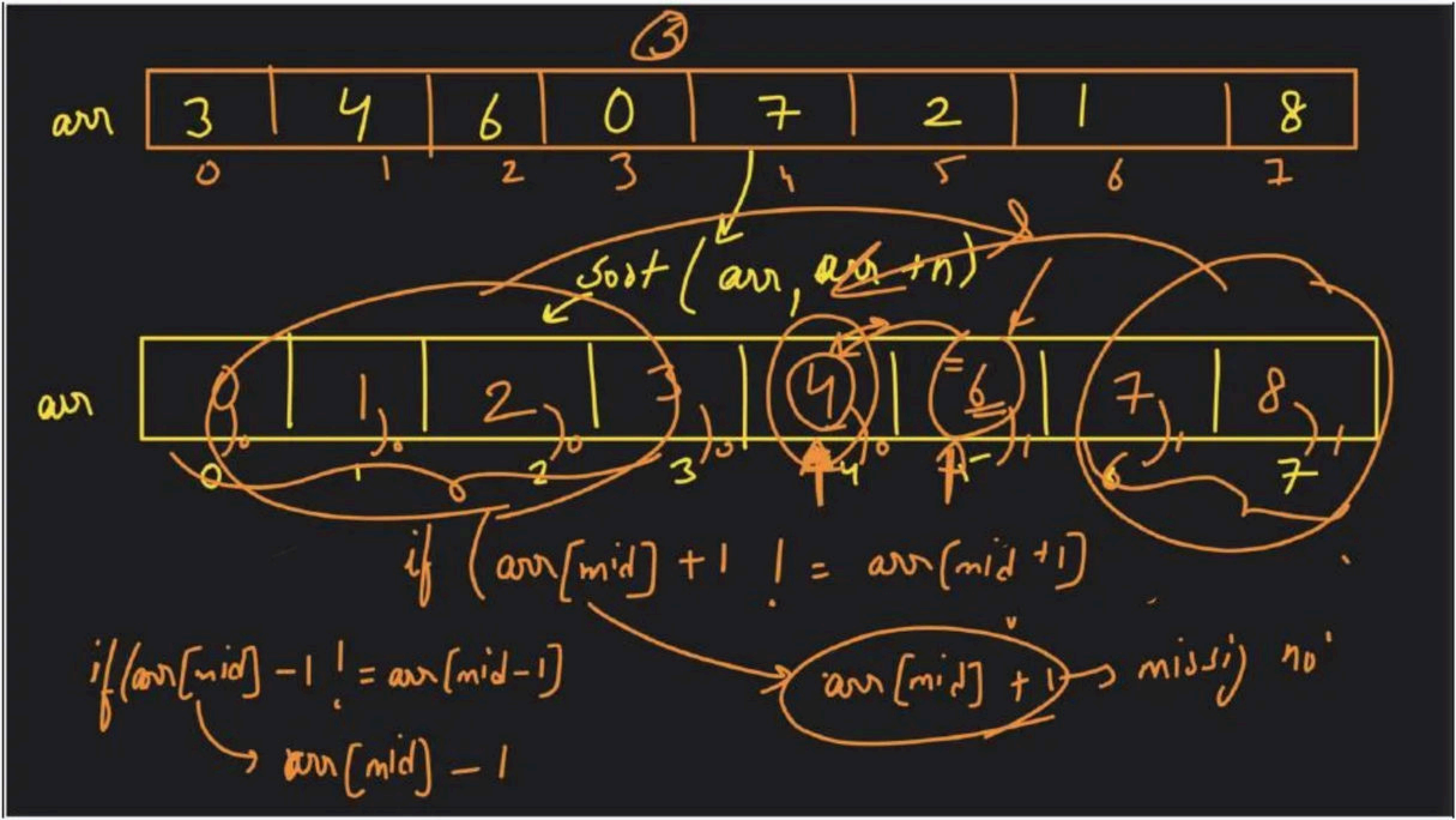
```
while(s<=e)
{   int mid = (s+e)/2;
    int diff = arr[mid] - mid;
    if(diff == 1)
    { ans = mid; e = mid-1
    }
    else if(diff == 0)
    {
        s = mid+1
    }
}
```

after this

```
if(ans == -1)
    ans = N
```


H/W
[0→n]
if →
arr{0, 1, 3, 4}
sum
0, 1, 2, 3, 4
10
n/2 (a+l)
5/2 (0+4) = 5
xy/2 = 20/2
= 10
diff
10 - 8 - 2


{ 3 , 0 , 1 }
sort(arr , arr+n)
O(nlogn)
0
1
3
0
1
2


arr
3 4 6 0 7 2 1 8
0 1 2 3 4 5 6 7
sort(arr, arr+n)
arr
1 2 4 6 7 8
0 1 2 3 4 5 6 7
if (arr[mid]+1 != arr[mid+1]
if(arr[mid]-1 != arr[mid-1]
arr[mid]-1
arr[mid]+1 → missing no



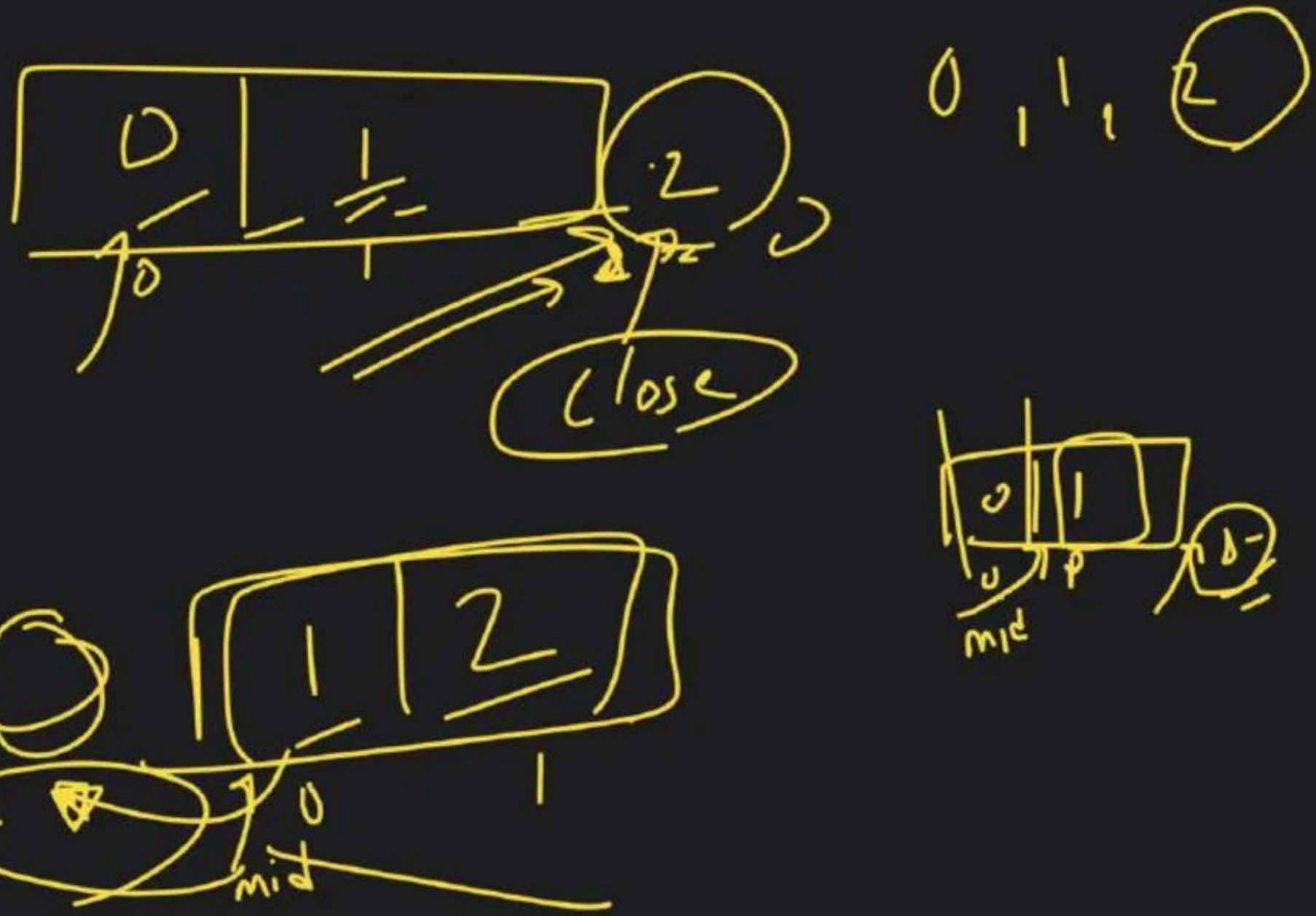
mid-1 mid

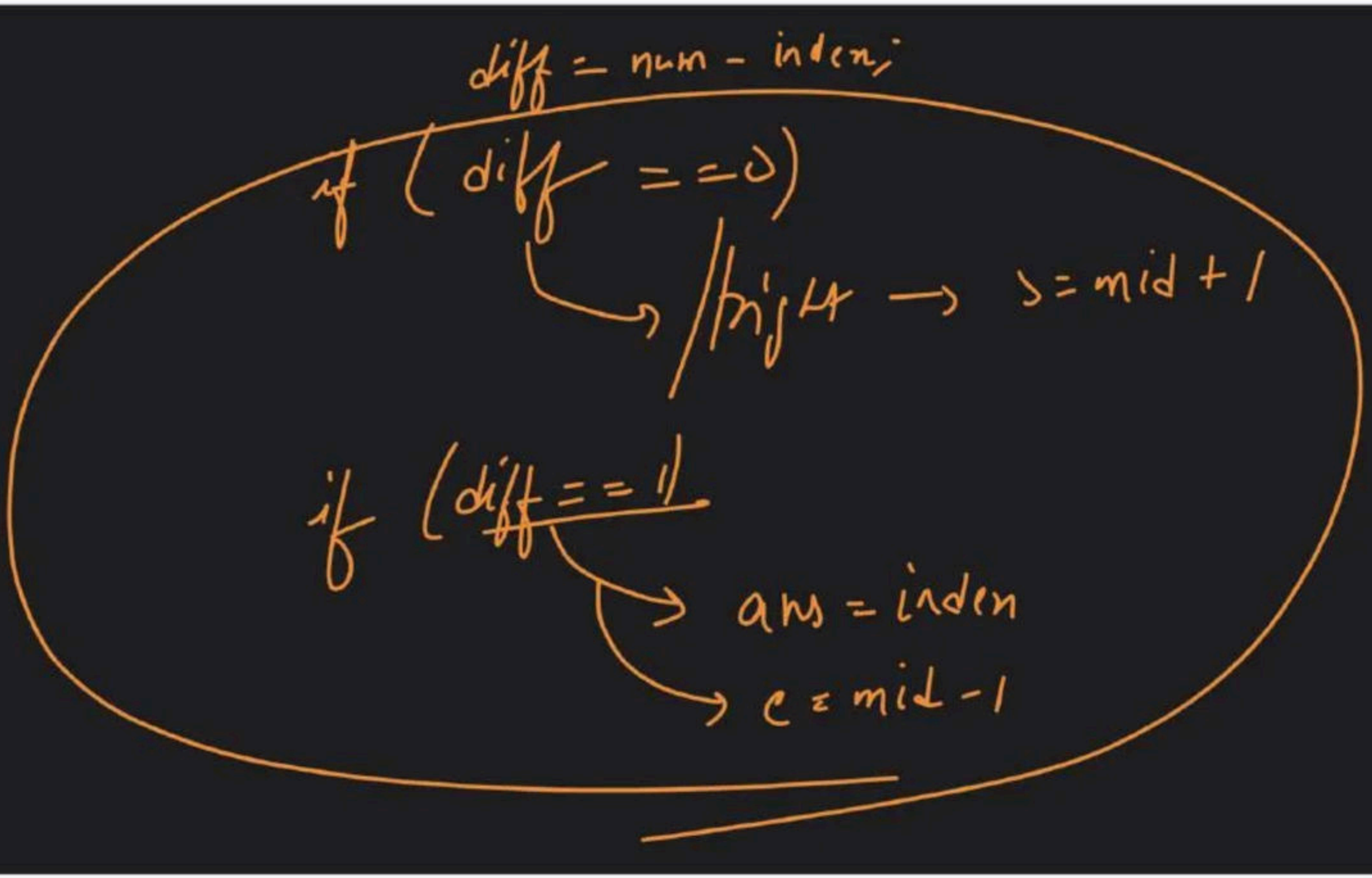
if (arr[mid+1] - arr[mid] != 1)

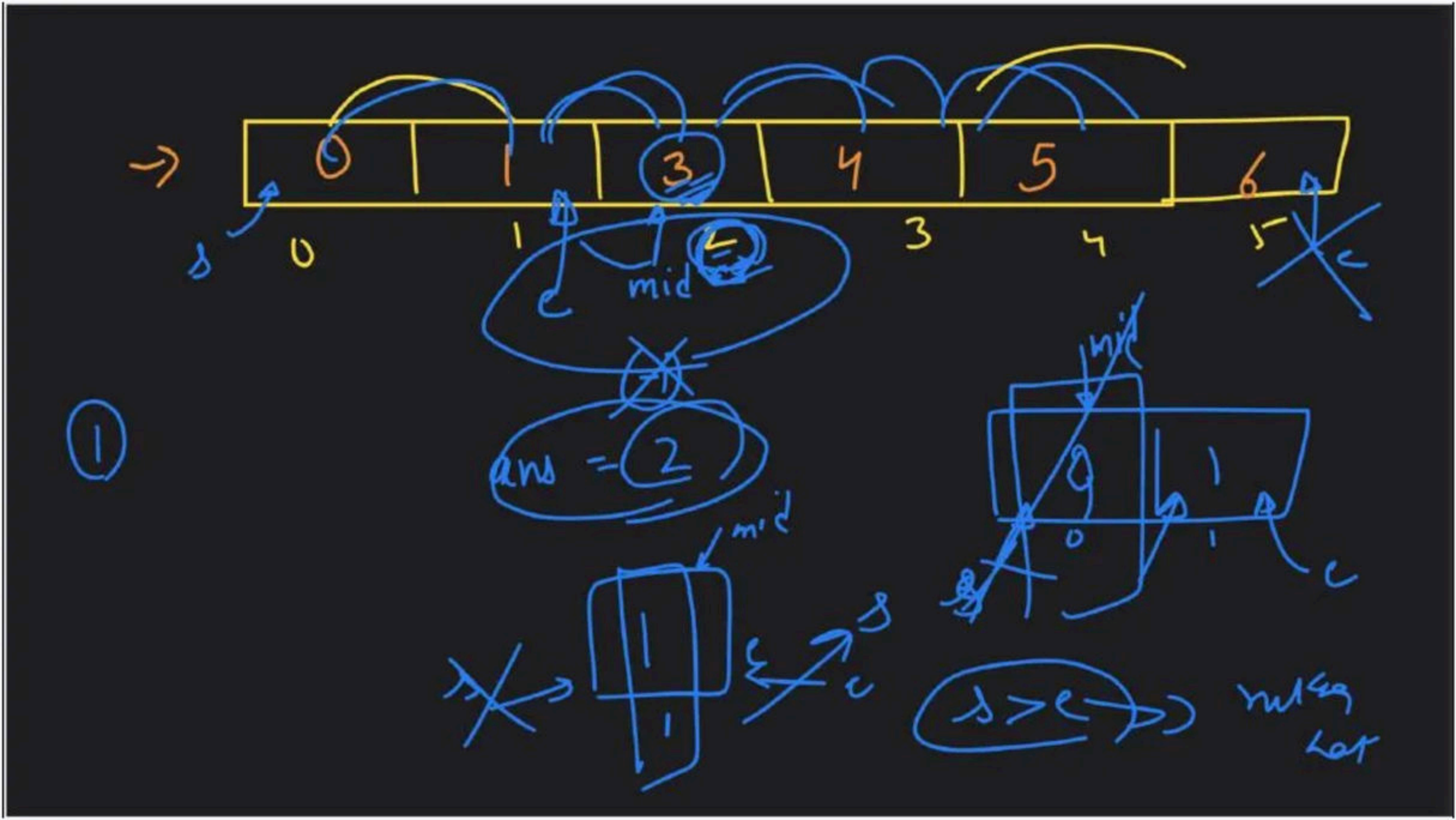
arr[mid] + 1

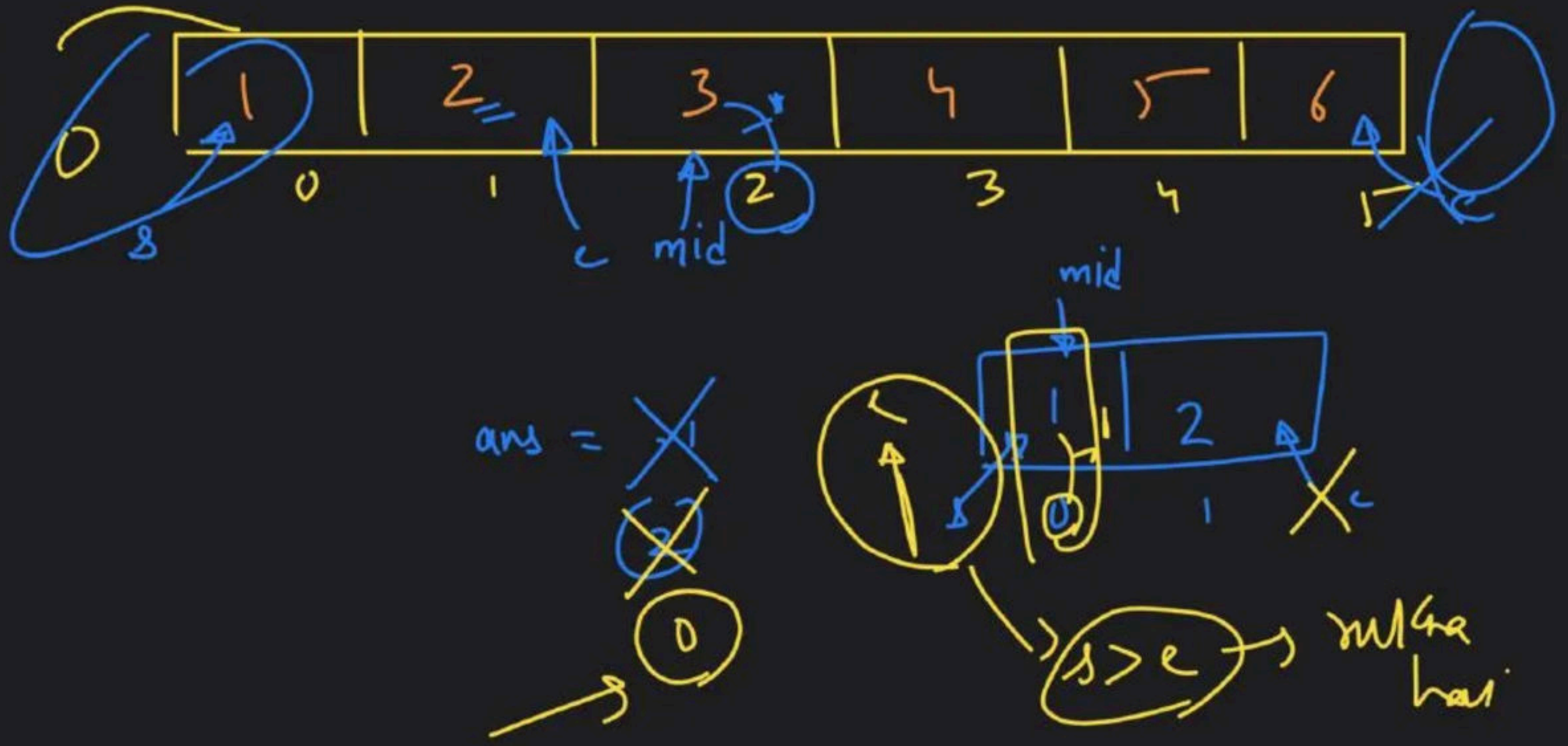
if (arr[mid] - arr(mid-1] != 1

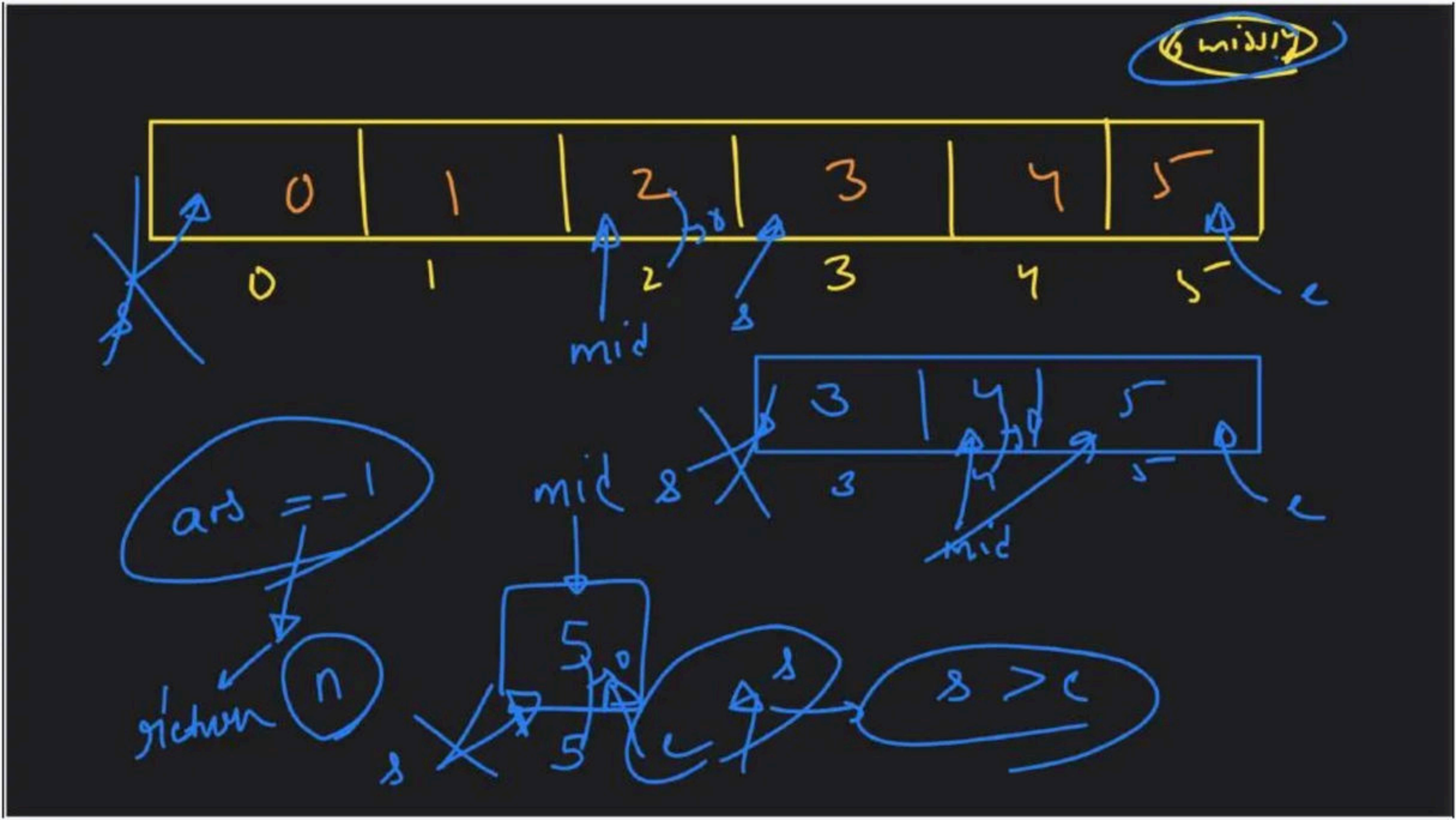
return arr[mid]-1


0
1
2
close
mid


diff = num - indexi
if (diff == 0)
//nijA → s = mid + 1
if (diff == 1)
ans = index
e = mid - 1


0
1
3
4
5
6
mid
ans = 2
mid
mid
Lot

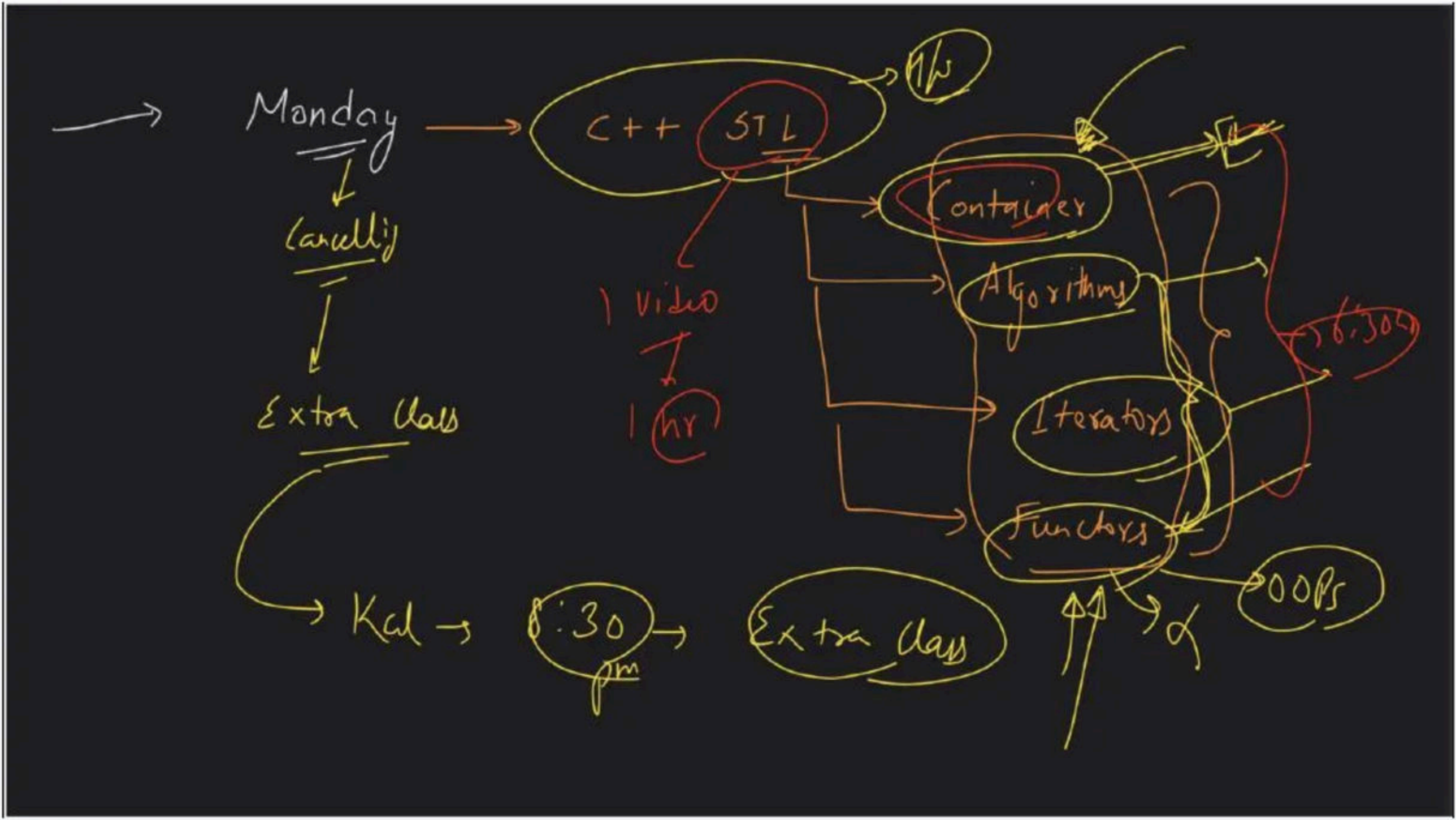

1
2
3
4
5
6
0
0
1
2
3
4
1
s
e
mid
mid
ans =
1
2
0
1
2
0
1
e
s>e


0
1
2
3
4
5
mid
s
e
ans = -1
return
n
mid
s > e

Searching and Sorting Class - 2

Special class

Love Babbar • May 29, 2024


Monday
Carelli
Extra Class
C++
STL
1 Video
1 hr
Container
Algorithms
Iterators
Functors
OOPs
Extra Class
6:30 pm

→ Binary Search

L.S
↳ O(n)

O(logn)

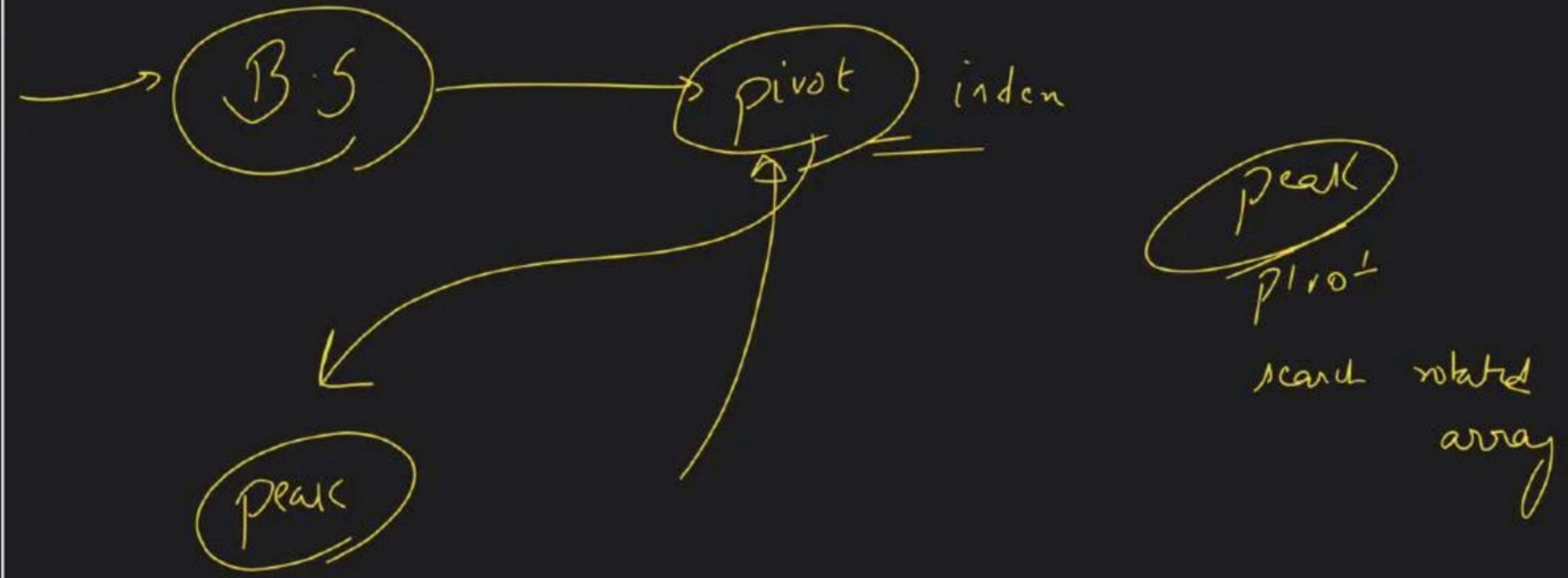
Pattern → Classical → left right

e = mid - 1

s = mid + 1

store & compute /may or may not be

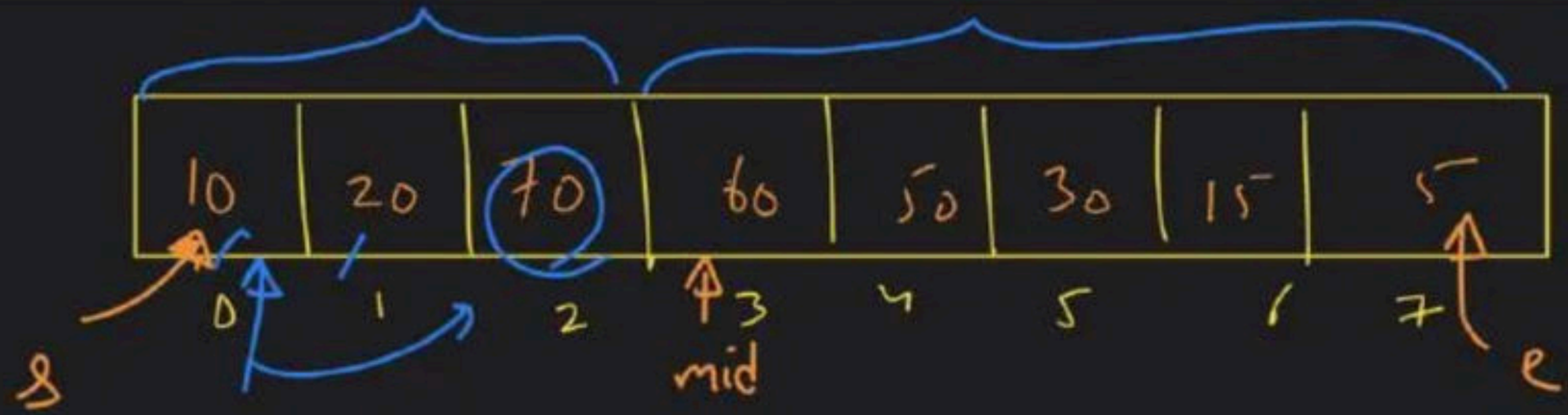
e = mid - 1 → e = mid → s <= e or s < e


B S
pivot
inden
peak
pivot
peak
search rotated
array

0	10	5	2
0	1	2	3



mountain array

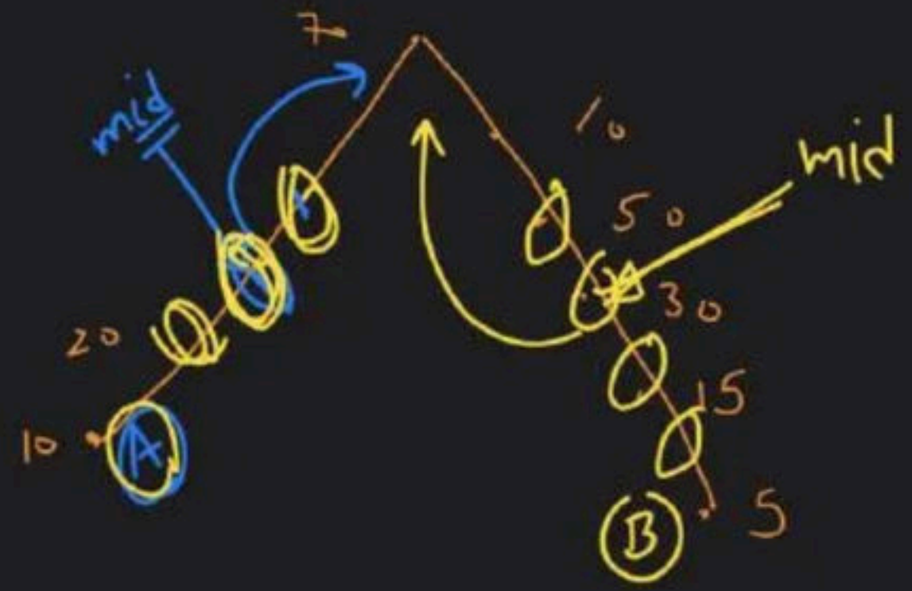


B.S

s = 0

e = 7

mid = 3



mid → line (A)

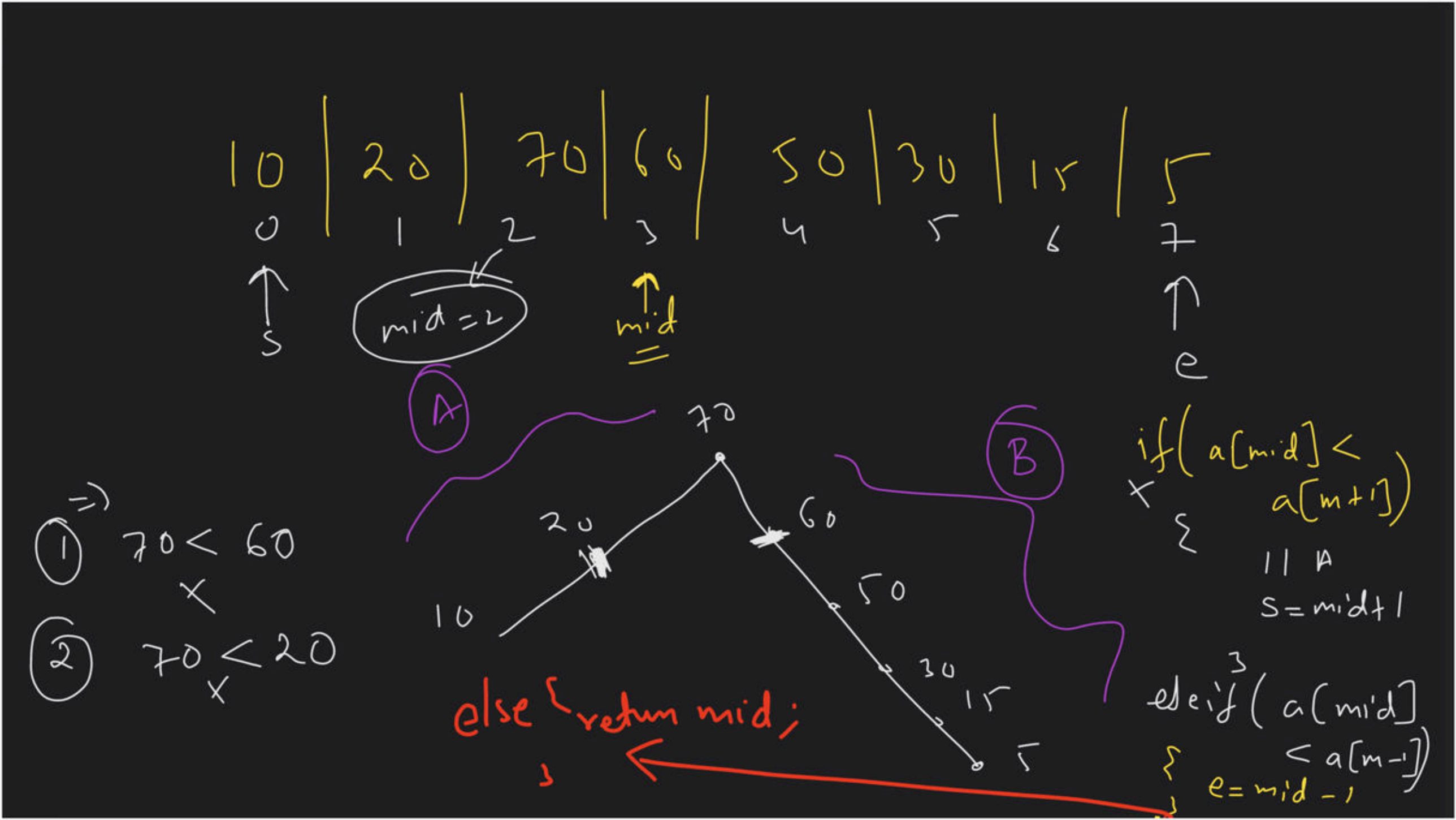
↳ inc

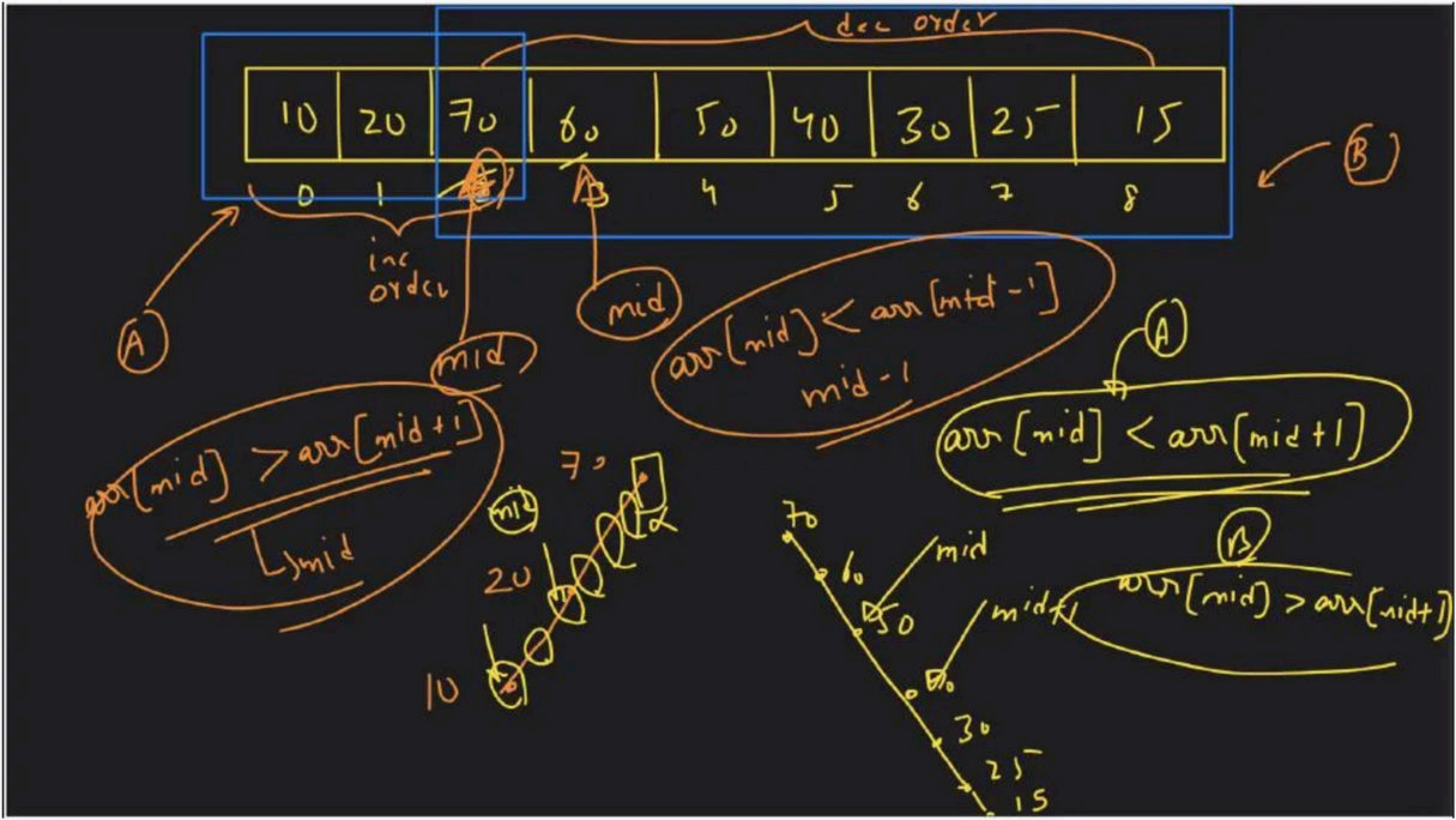
arr[mid] < arr[mid+1]

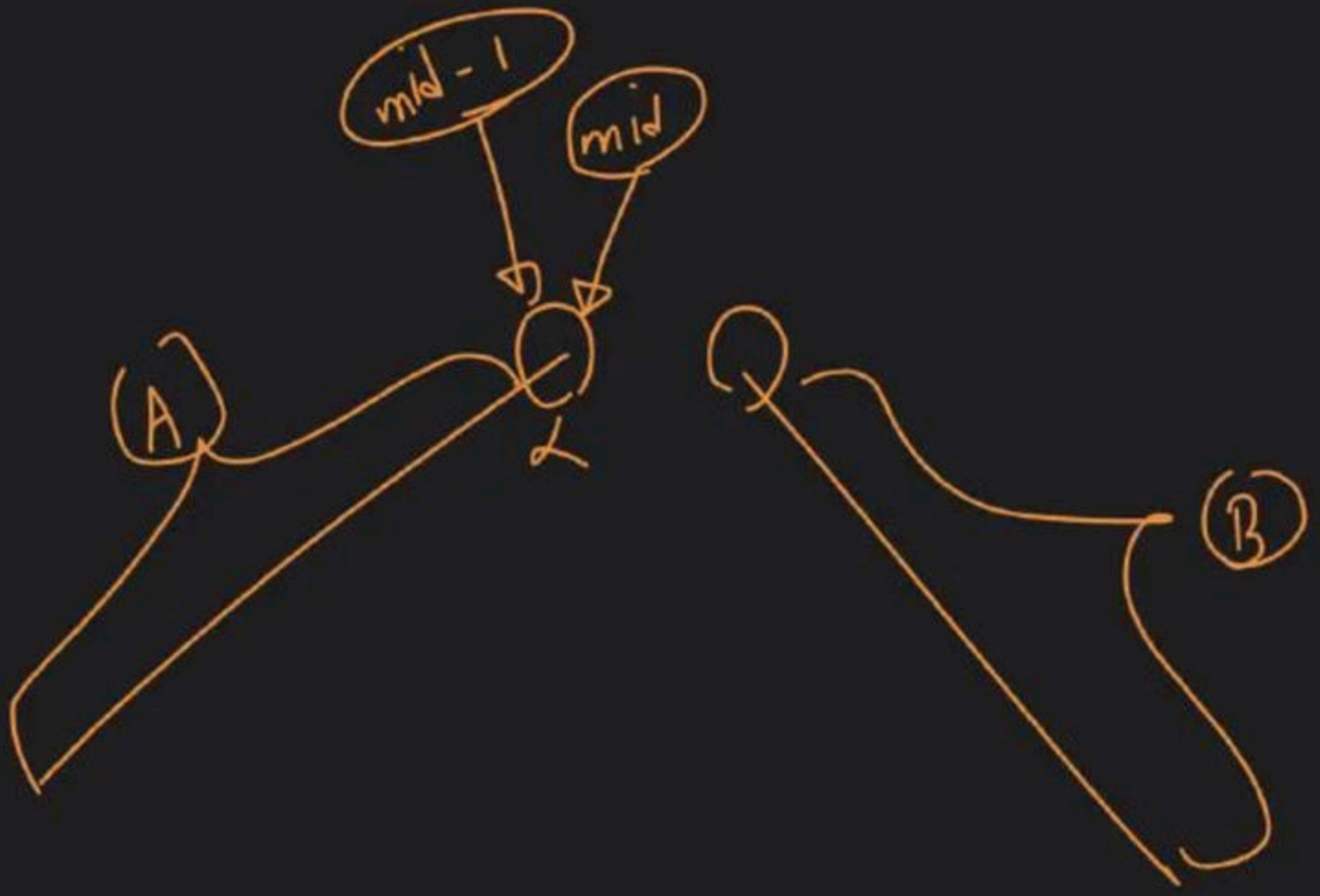
↳ Right

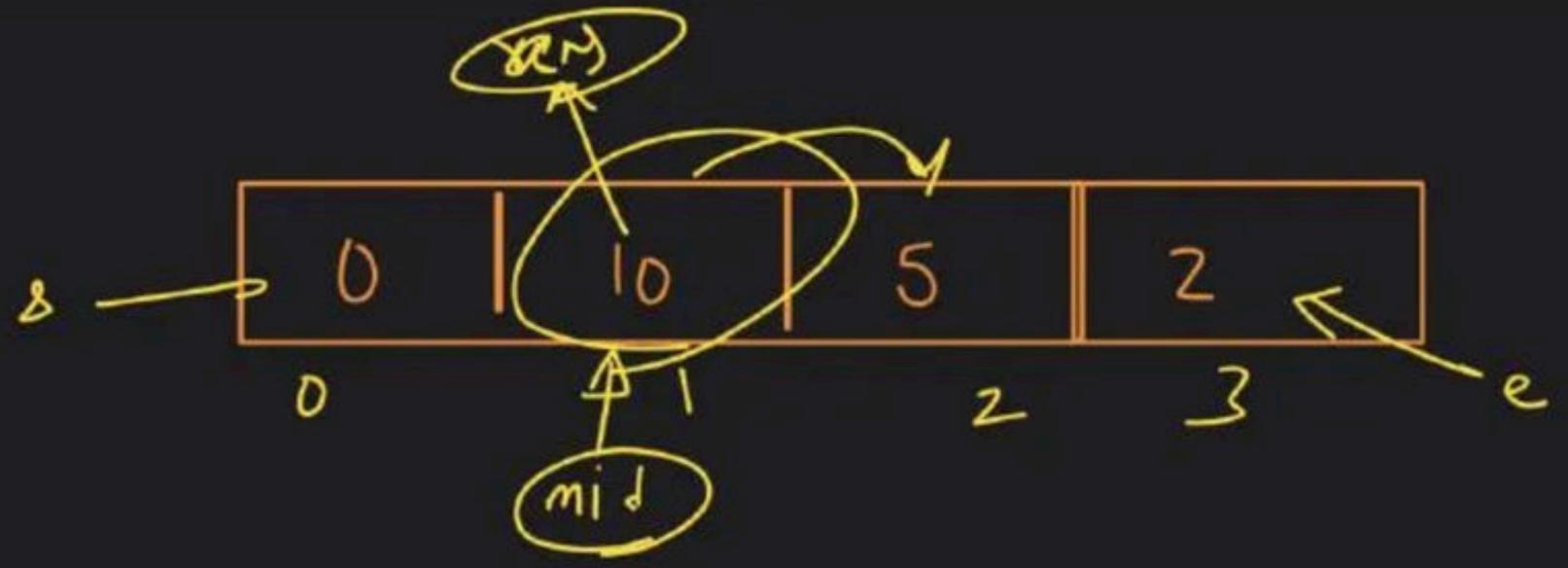
arr[mid] > arr[mid+1]

↳ left

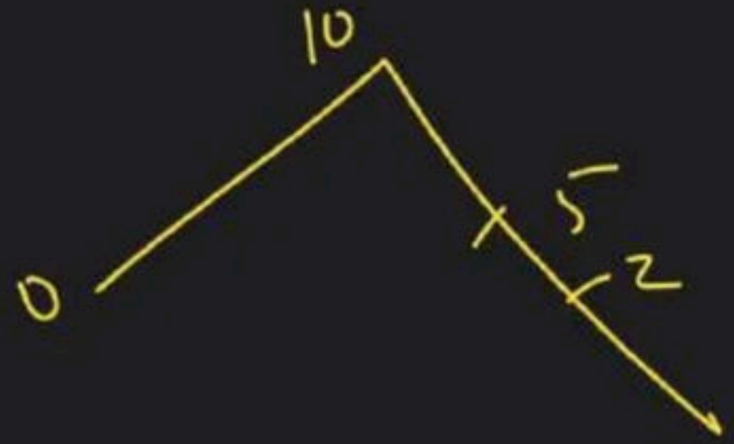

10 | 20 | 70 | 60 | 50 | 30 | 15 | 5
0 1 2 3 4 5 6 7
s
mid = 2
mid
e
A
B
70
20
60
10
50
30
15
5
① ⇒ 70 < 60
×
② 70 < 20
×
if(a[mid] < a[m+1])
|| A
s = mid + 1
else { return mid;
}
else if(a[mid] < a[m-1])
e = mid - 1

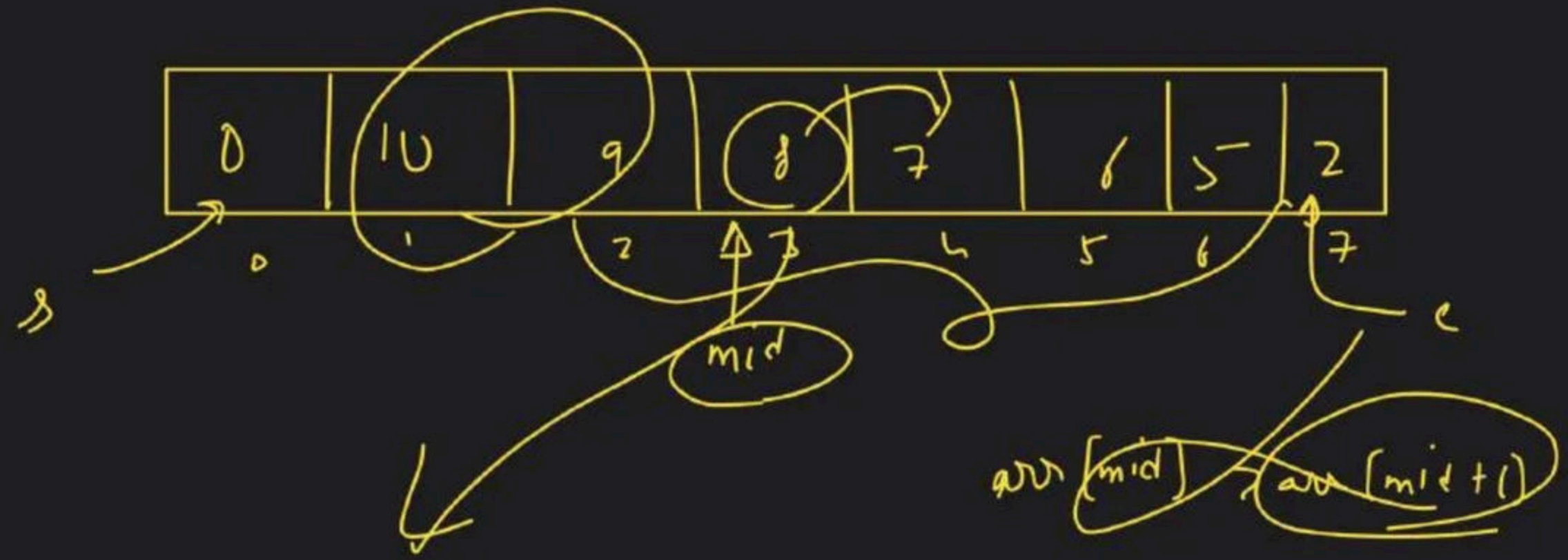

dec order
10
20
70
60
50
40
30
25
15
0
1
2
3
4
5
6
7
8
B
inc order
A
mid
mid
arr[mid] < arr[mid-1]
mid-1
A
arr[mid] > arr[mid+1]
mid
arr[mid] < arr[mid+1]
70
mid
20
10
70
60
mid
50
mid+1
B
arr[mid] > arr[mid+1]
30
25
15

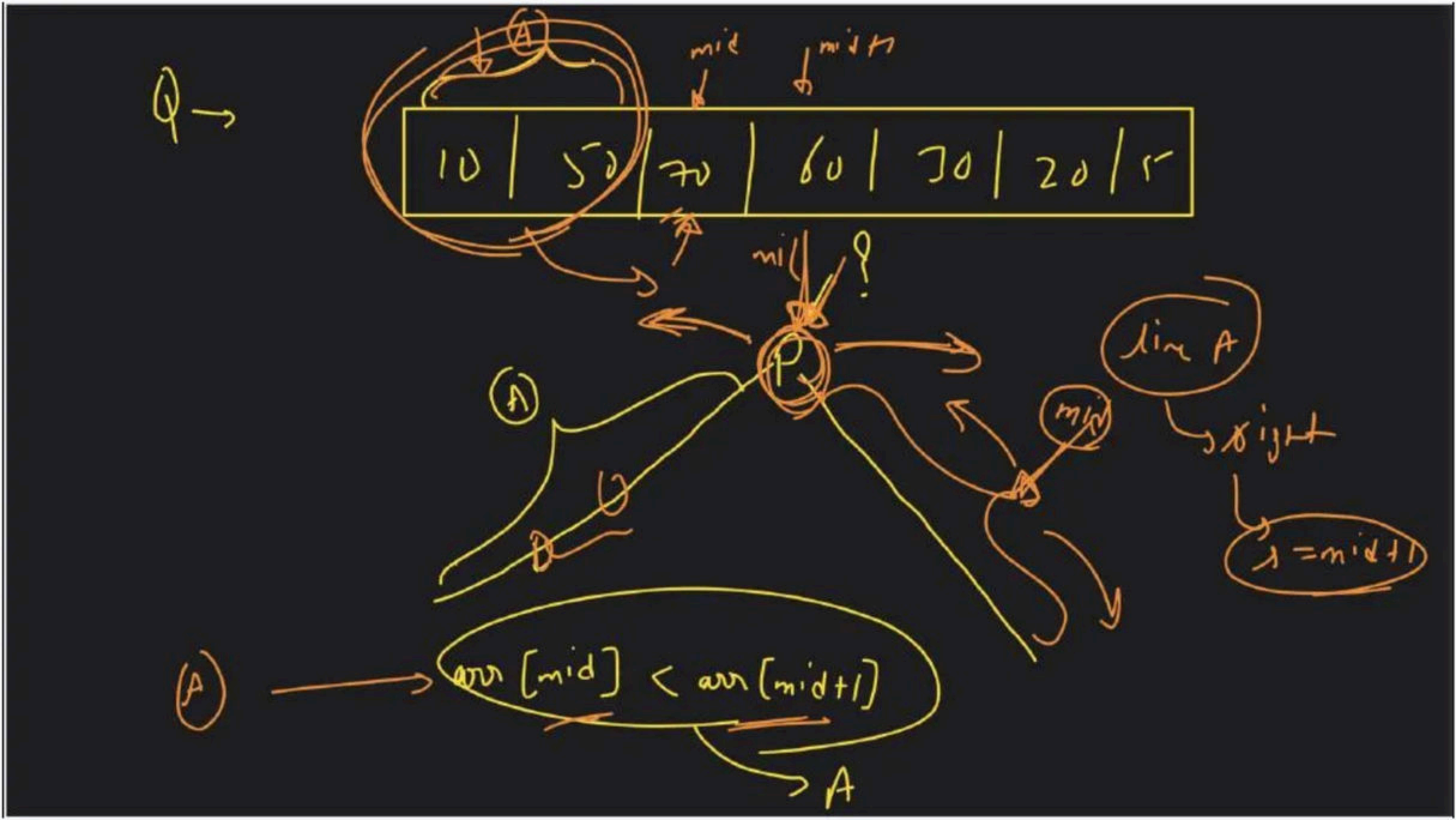

mid - 1
mid
A
B

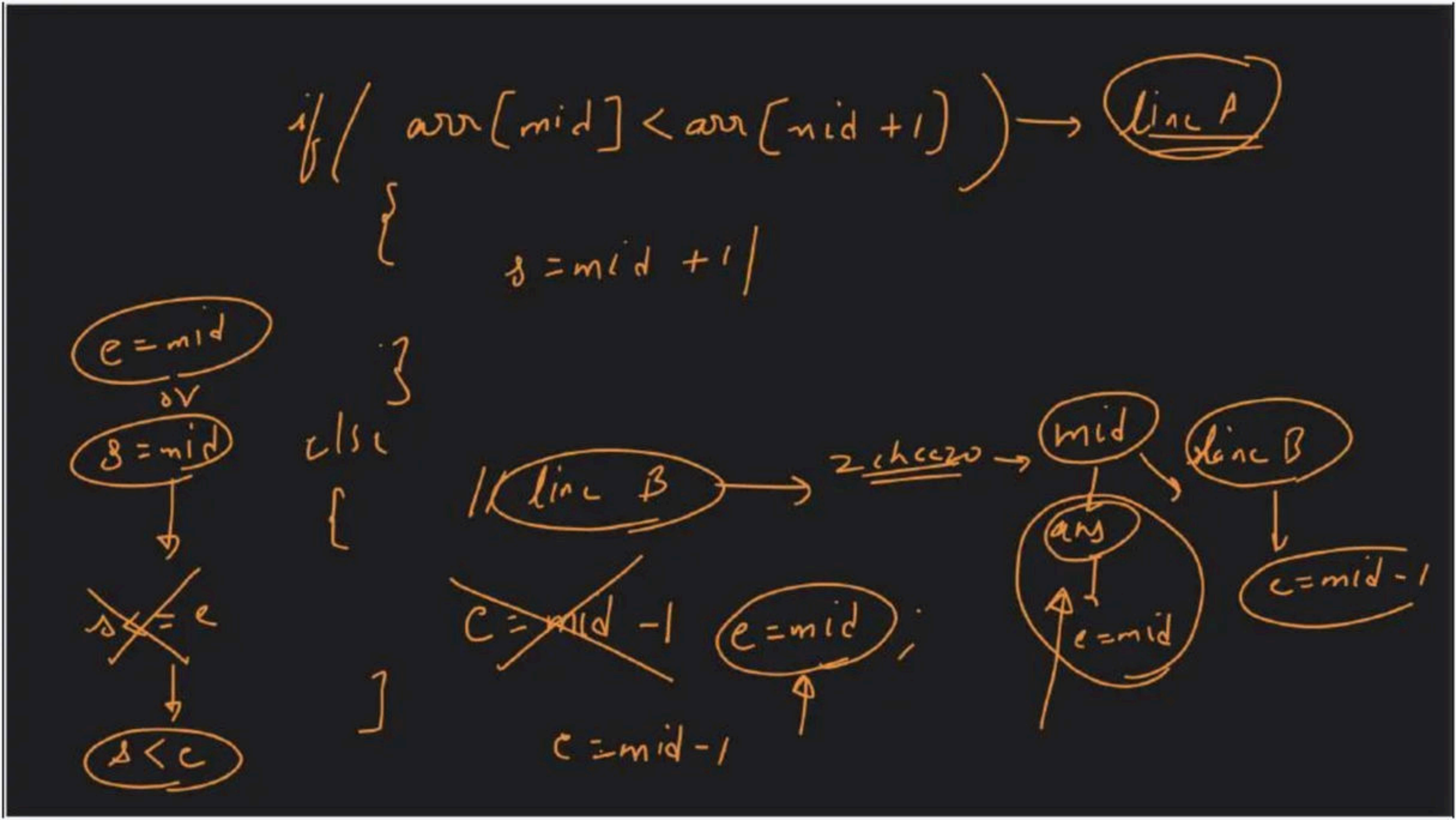


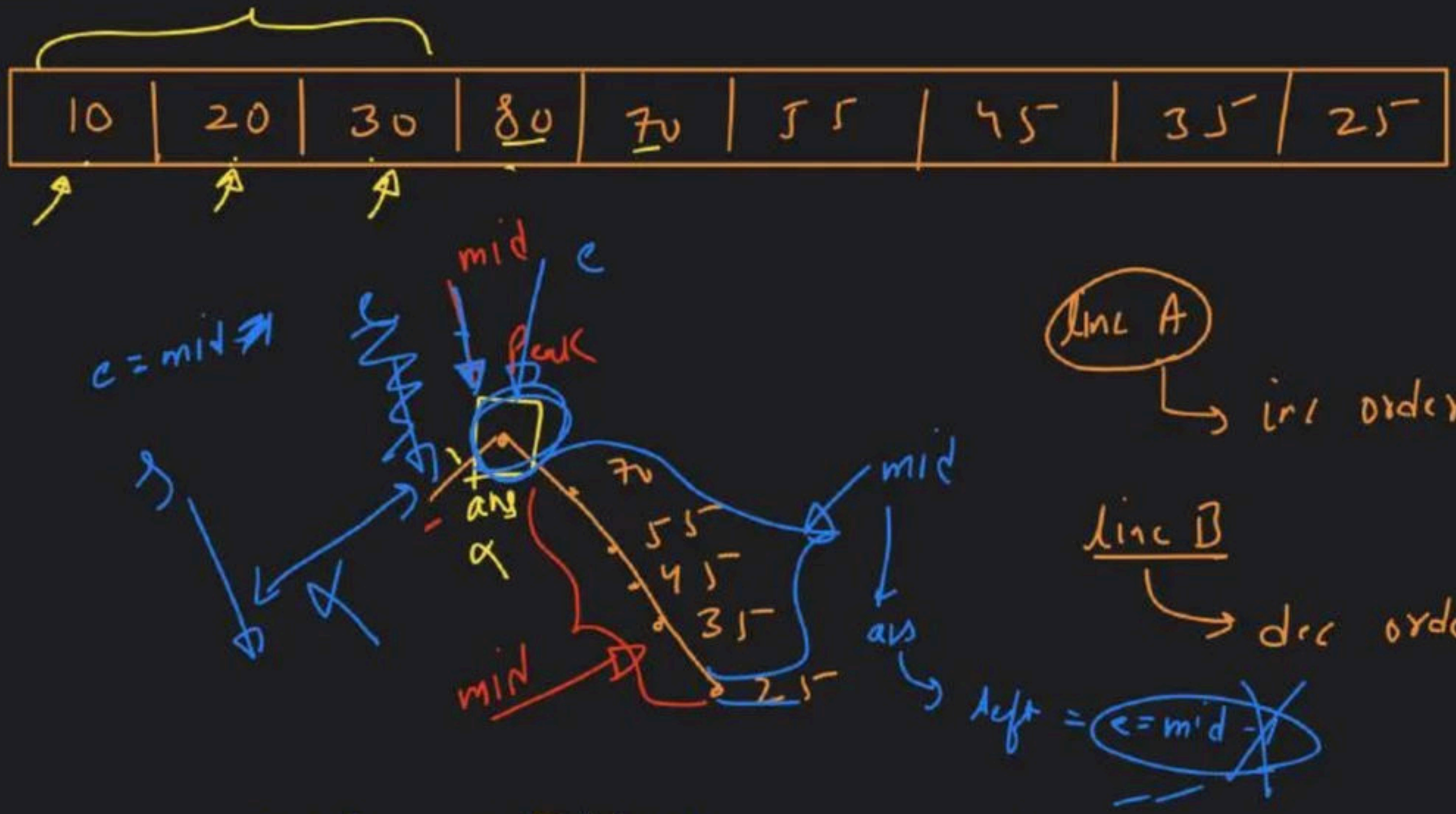
ans[mid] > ans[mid + 1]



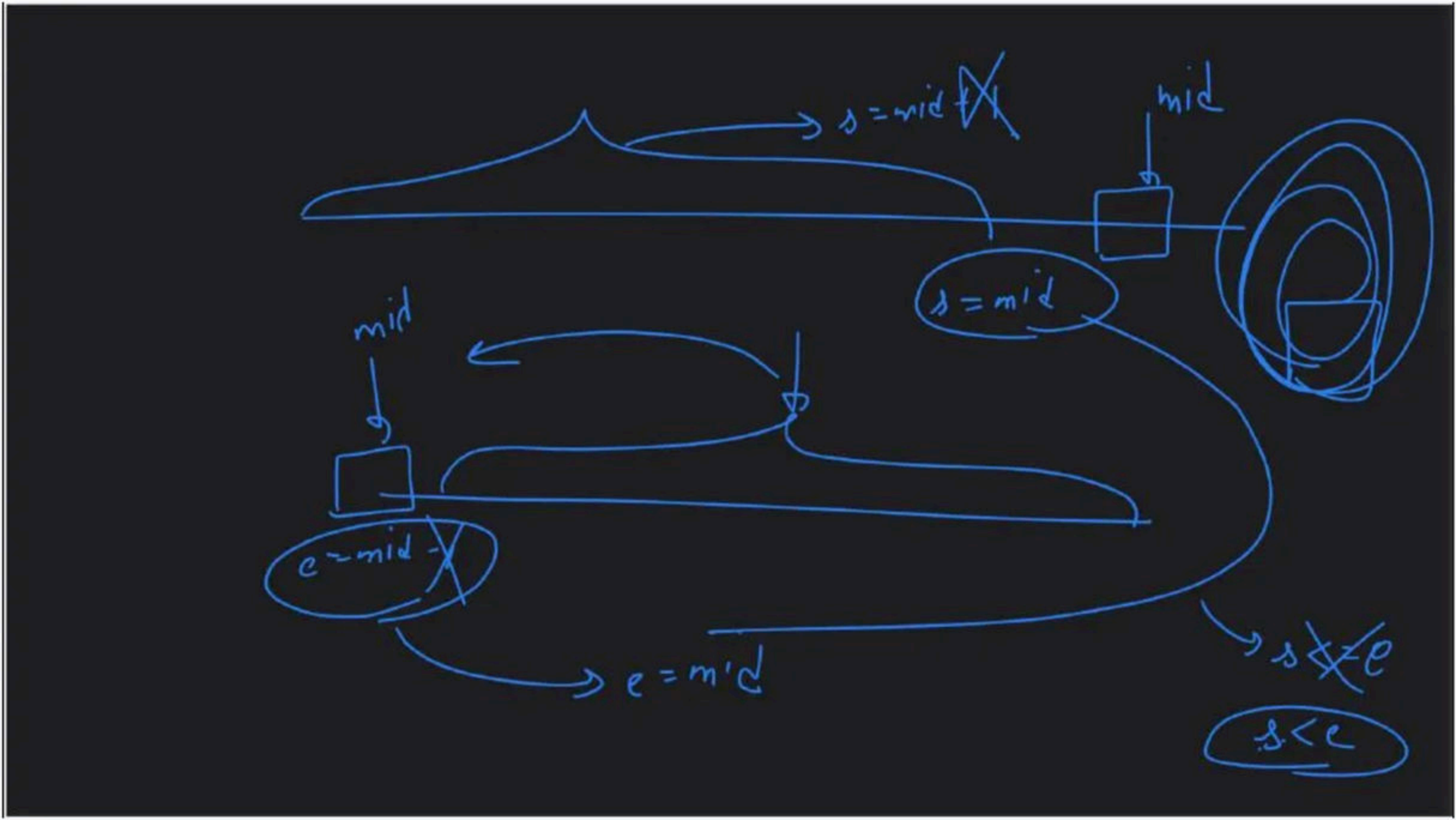

0
10
9
8
7
6
5
2
0
1
2
3
4
5
6
7
s
e
mid
arr[mid]
arr[mid+1]


Q →
A
mid
mid+1
10
50
70
60
30
20
mid ?
P
A
mid
low A
right
l = mid+1
A
arr [mid] < arr (mid+1)
A


if(arr[mid] < arr[mid+1]) → linea A
{
s = mid + 1;
}
else
{
linea B → secondo → mid
linea B
mid
ang
e = mid
s = mid
s <= e
s < e
c = mid - 1
e = mid;
c = mid - 1
e = mid - 1
]

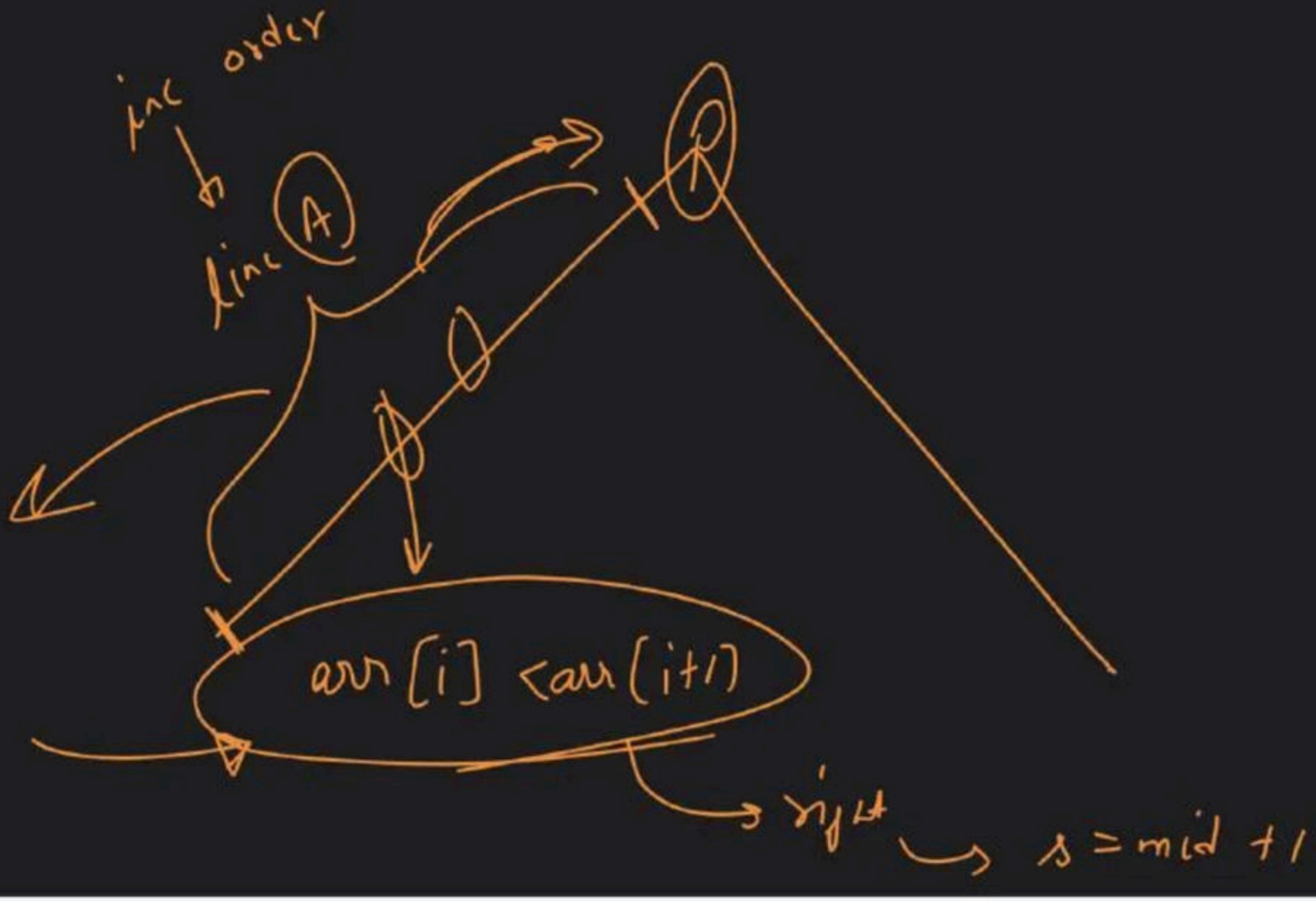


Inc A → arr[mid] < arr[mid+1] ⟶ move right → s = mid+1

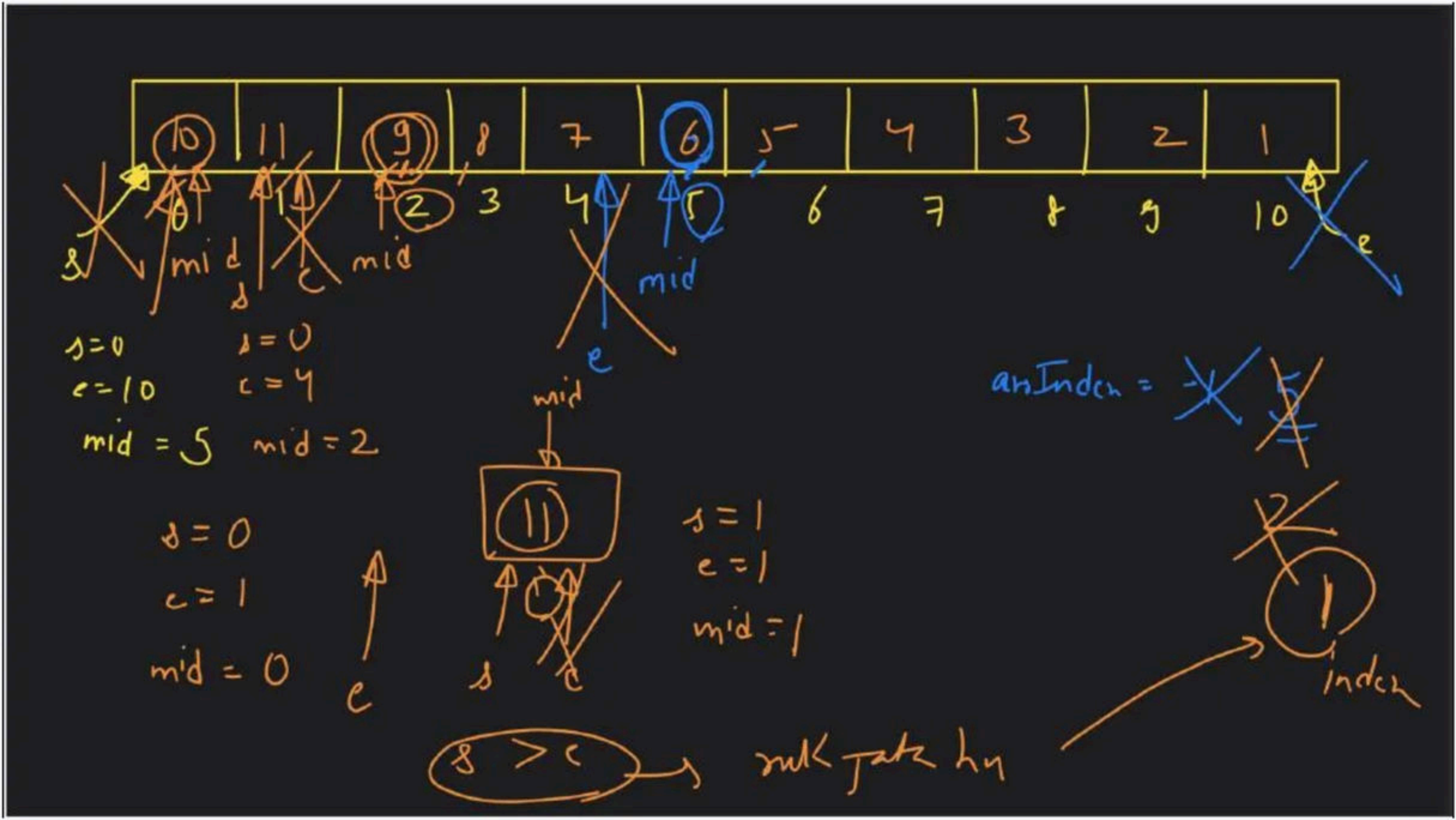

s = mid
mid
s = mid
mid
e = mid
e = mid
s < e

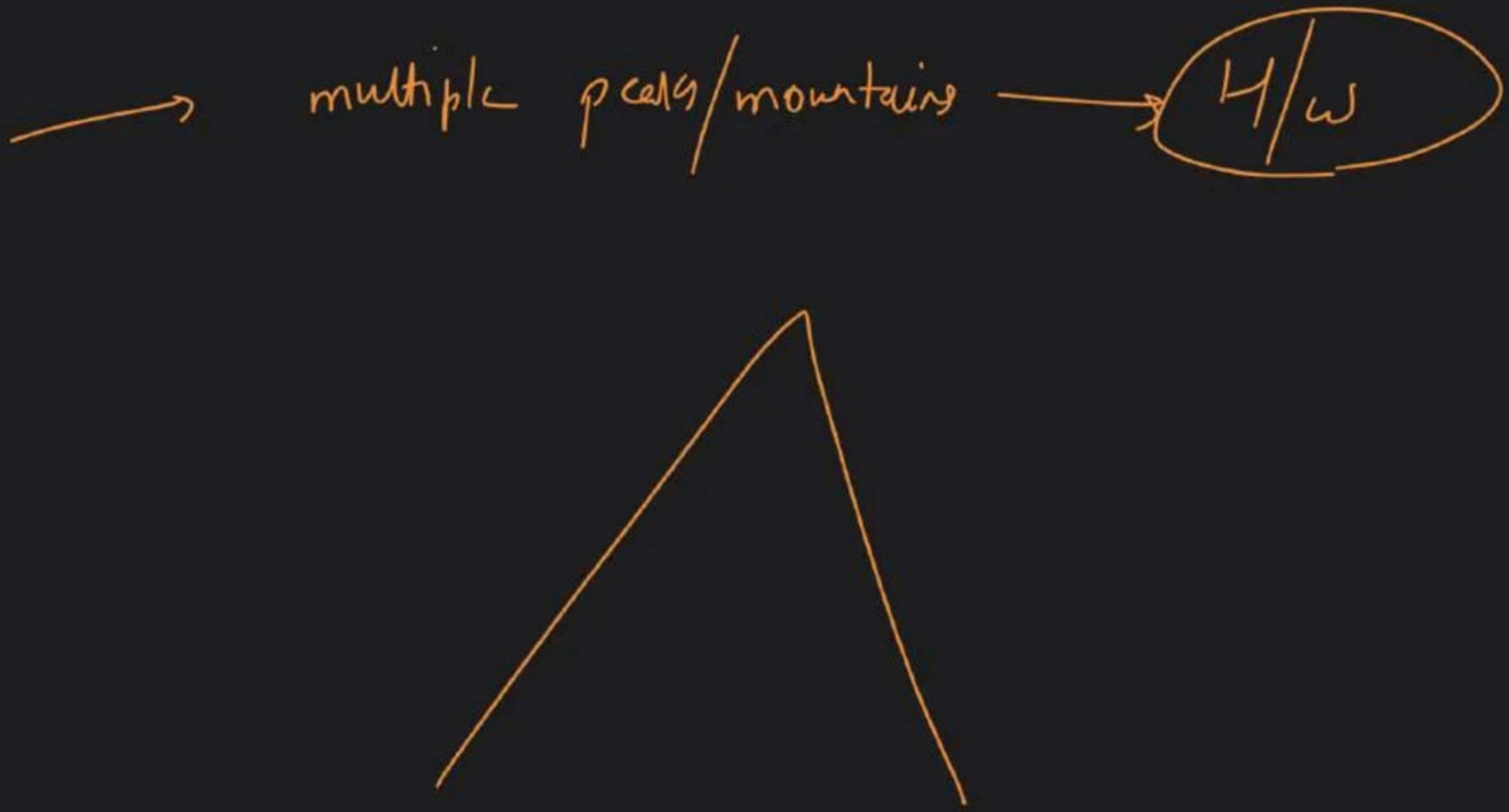
```
if ( arr[mid] < arr[mid + 1])
  {
     s = mid + 1
  }
else
{


}
```


inc order
A
B
arr[i] < arr(i+1)
right
s = mid + 1


10
40
90
80
70
60
50
30
20
0
1
2
3
4
5
6
7
8
mid
mid
mid
s
e
l
l
s > e
null ayır

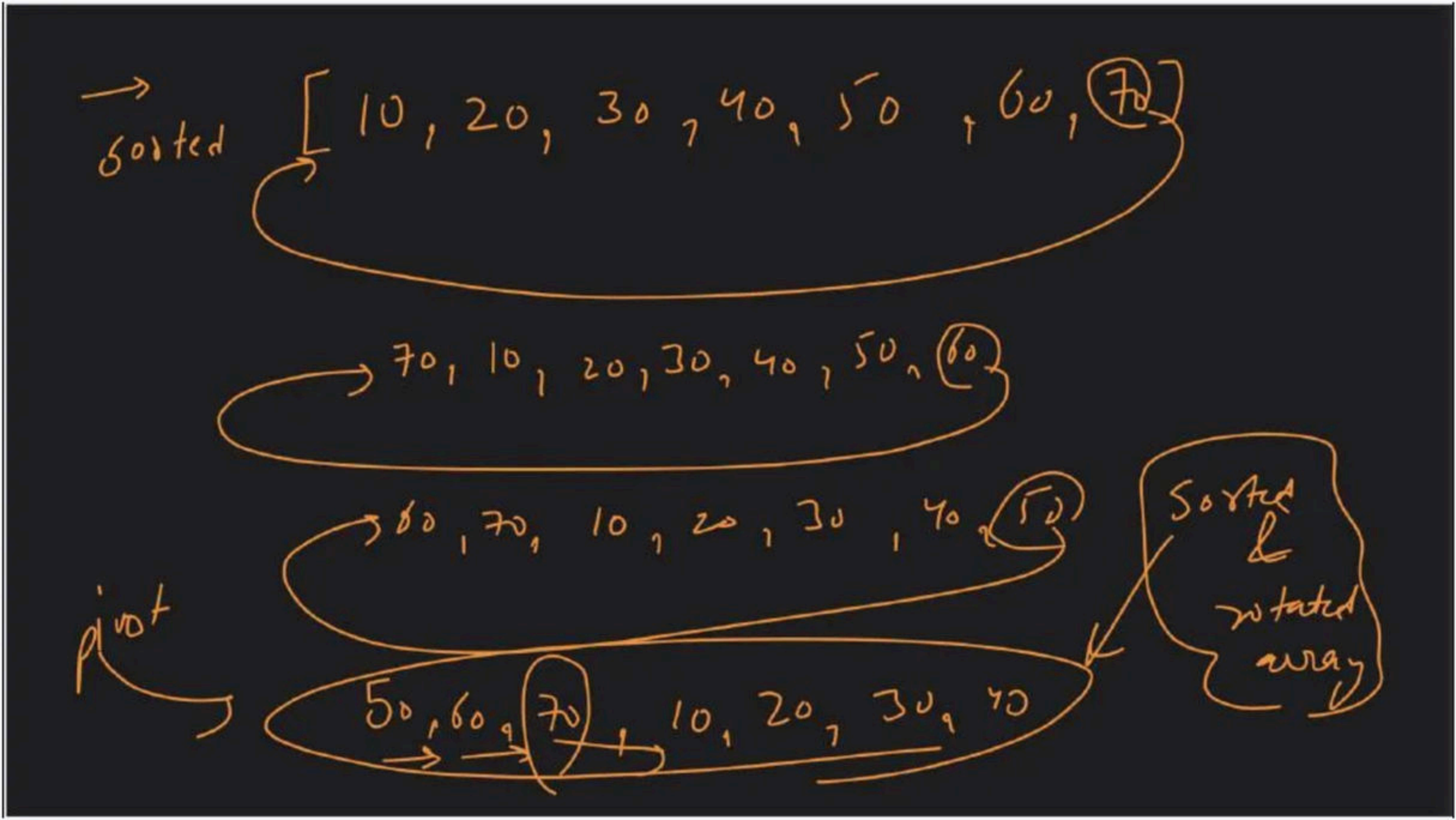

10
11
9
8
7
6
5
4
3
2
1
0
1
2
3
4
5
6
7
8
9
10
s
mid
e
s = 0
e = 10
mid = 5
s = 0
e = 4
mid = 2
mid
e
mid
s
e
s = 0
e = 1
mid = 0
s = 1
e = 1
mid = 1
ansIndex =
s > e
ans jata hai
Index

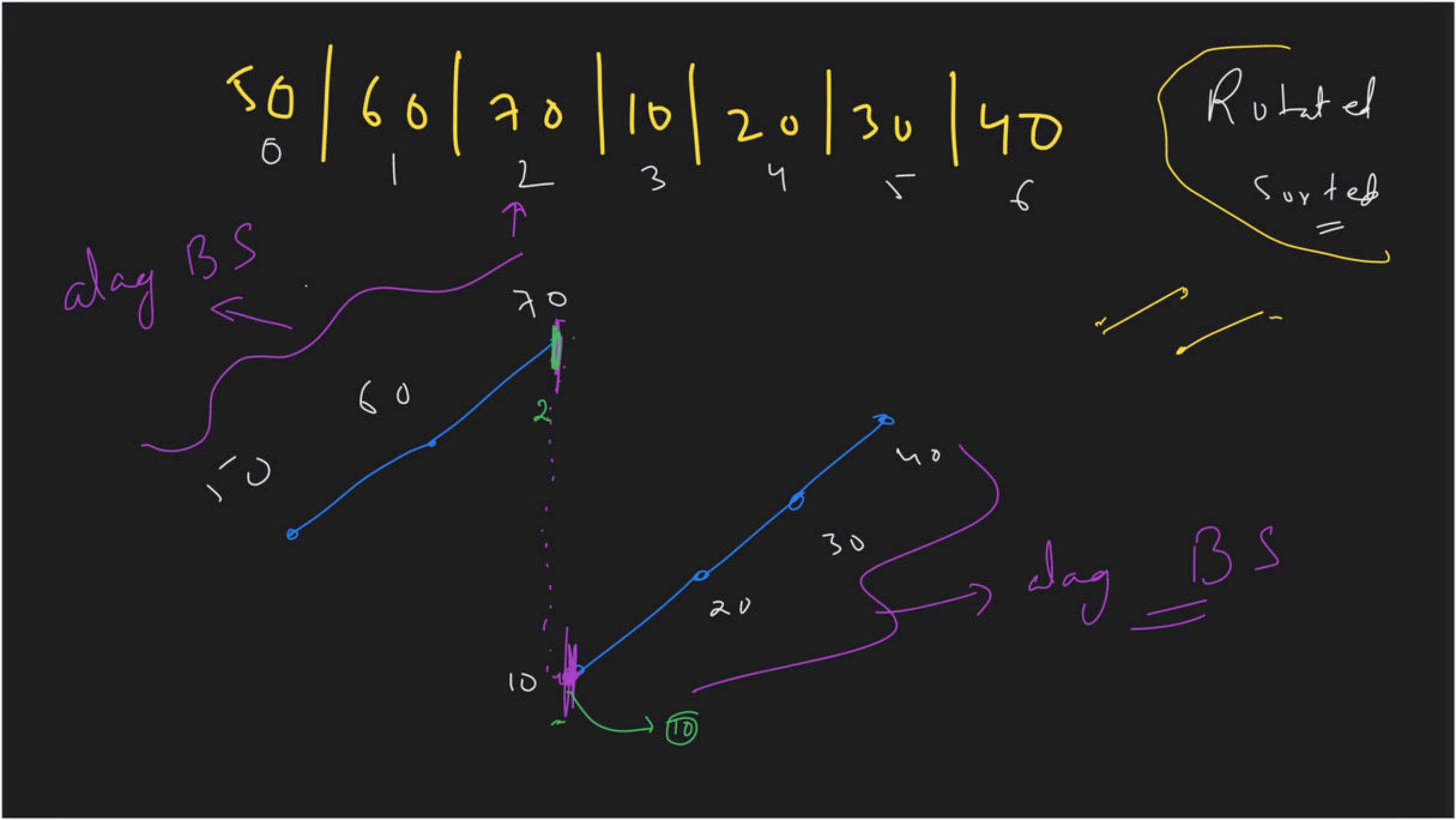

multiple peaks/mountains
H/W

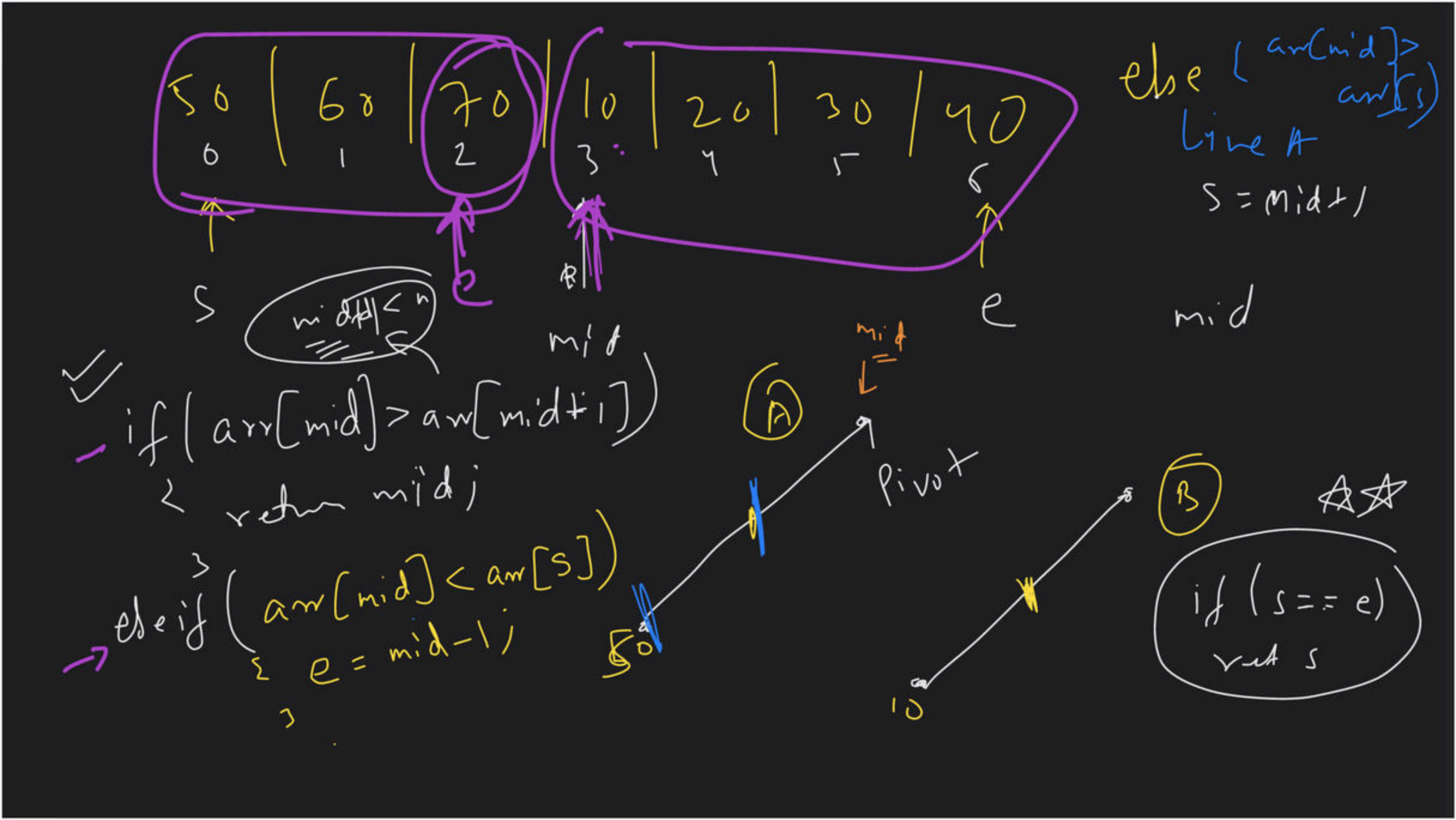
→ find pivot index

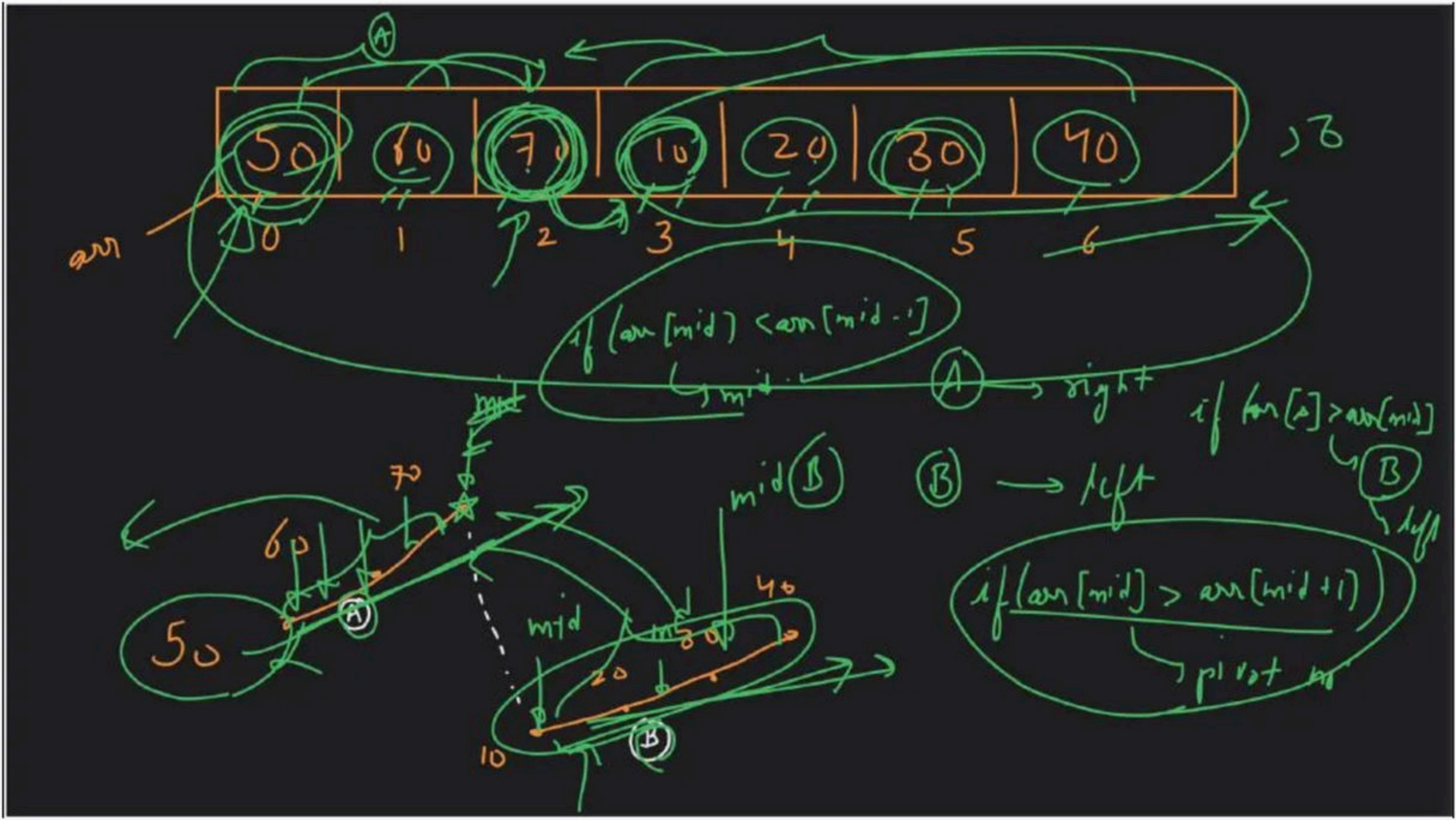


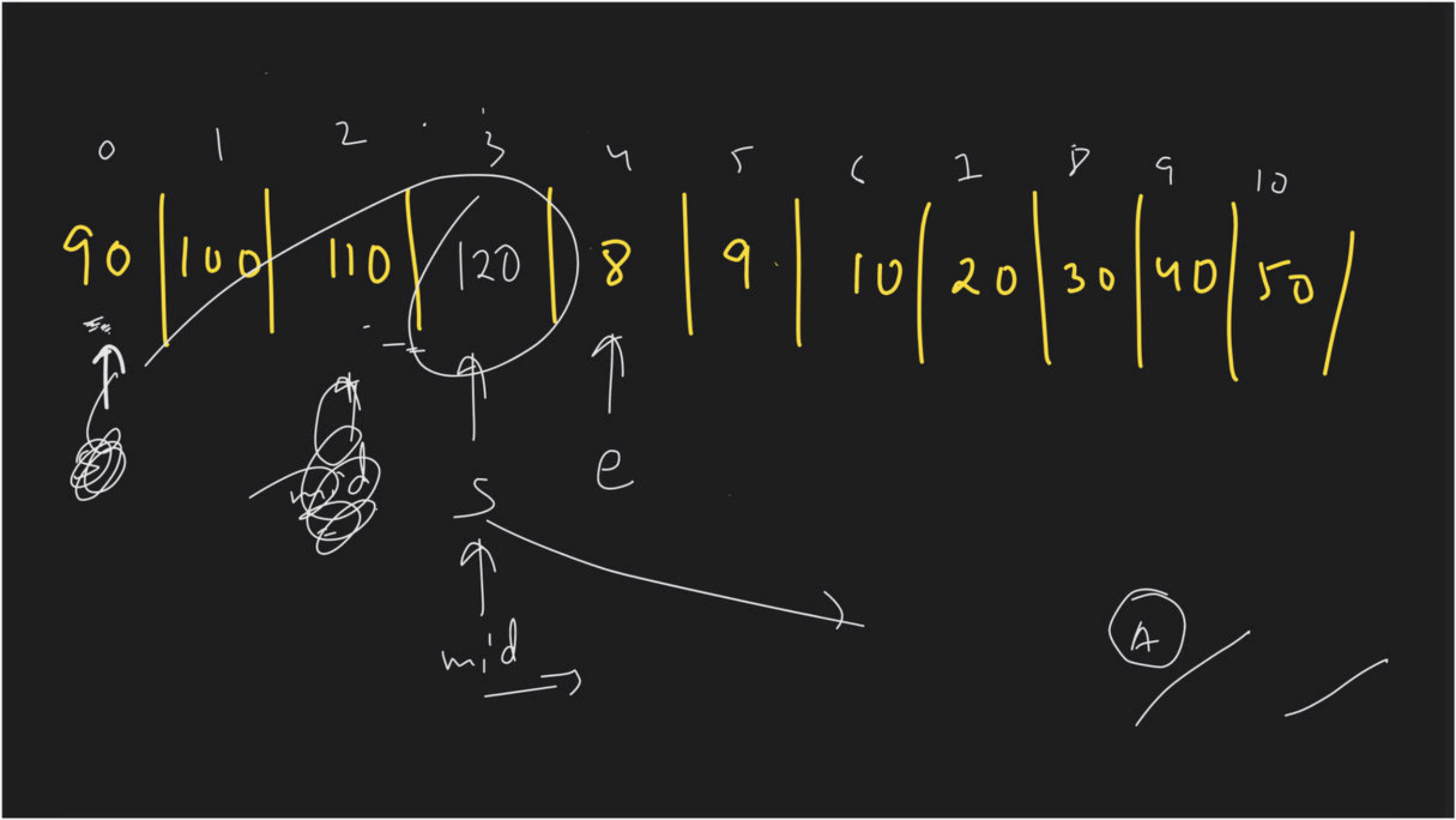


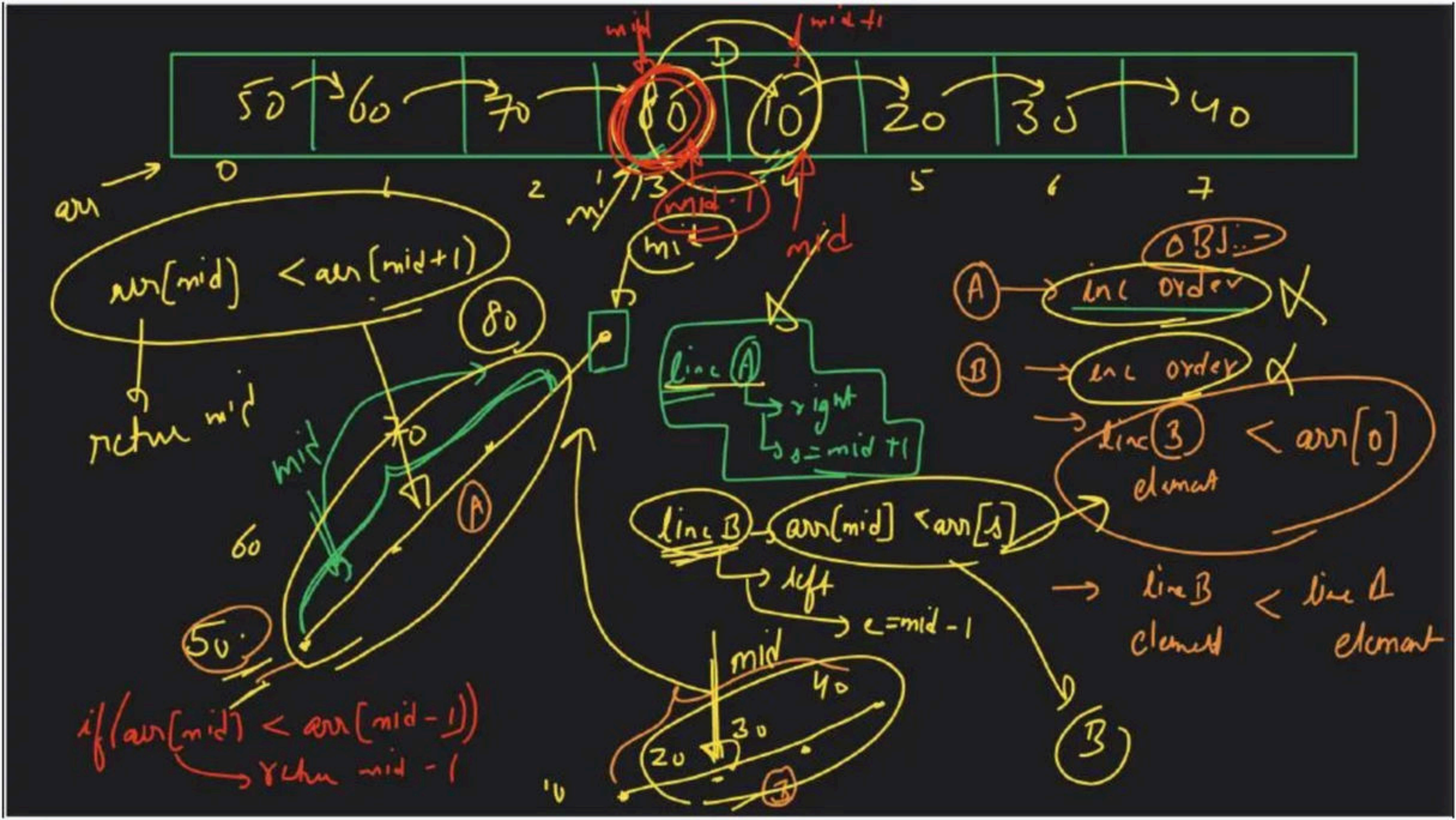

sorted
10, 20, 30, 40, 50, 60, 70
70, 10, 20, 30, 40, 50, 60
60, 70, 10, 20, 30, 40, 50
pivot
50, 60, 70, 10, 20, 30, 40
Sorted & rotated array

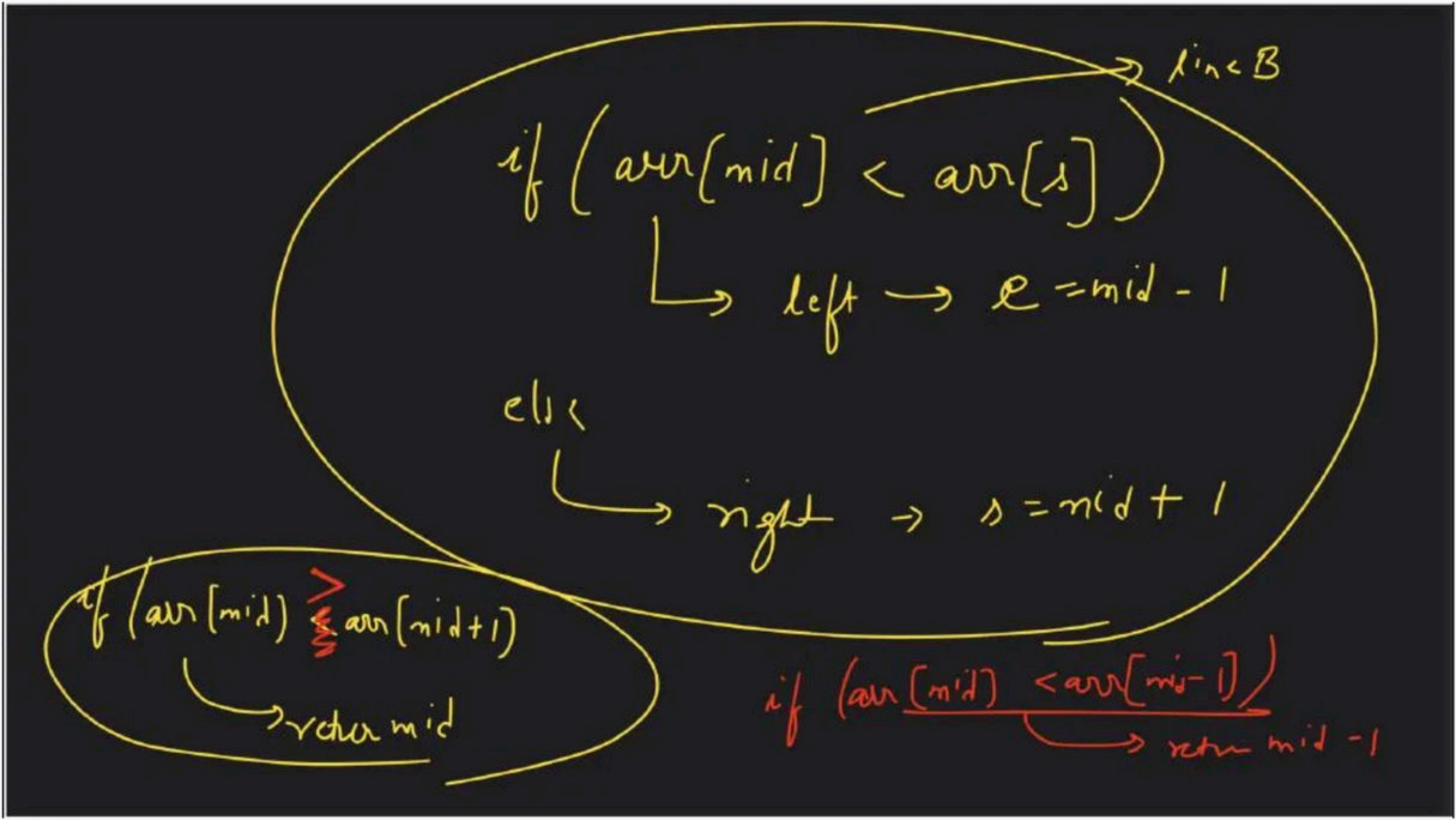

50 | 60 | 70 | 10 | 20 | 30 | 40
0 1 2 3 4 5 6
Rotated Sorted
diag BS
70
60
10
2
40
30
20
10
diag == BS

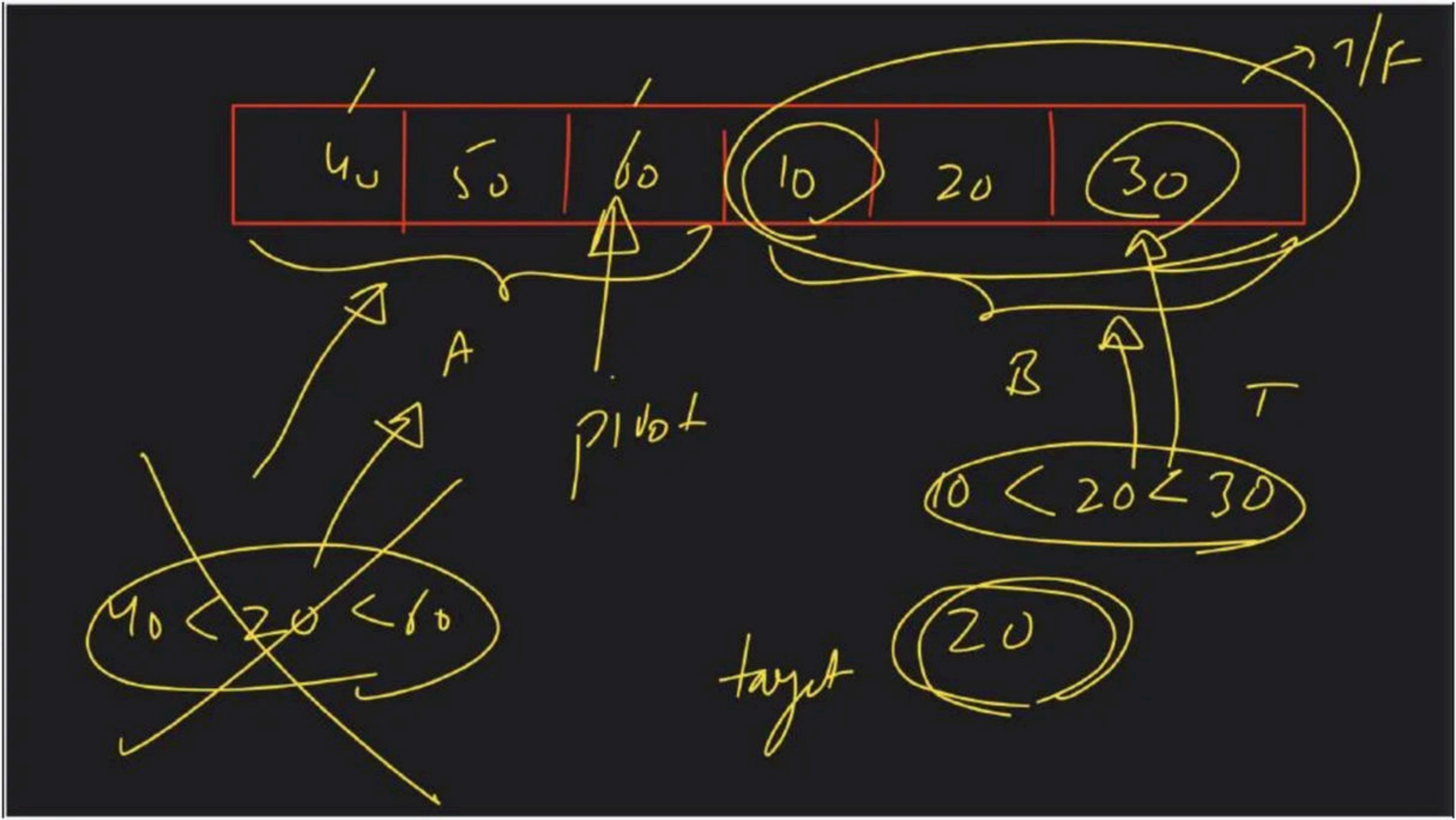

50 | 60 | 70 | 10 | 20 | 30 | 40
0 1 2 3 4 5 6
s
e
mid
mid<n
if(arr[mid]>arr[mid+1])
return mid;
else if(arr[mid]<arr[s])
e = mid-1;
else (arr[mid]>arr[s])
Line A
s = mid+1
A
mid
Pivot
50
10
B
if(s==e)
return s

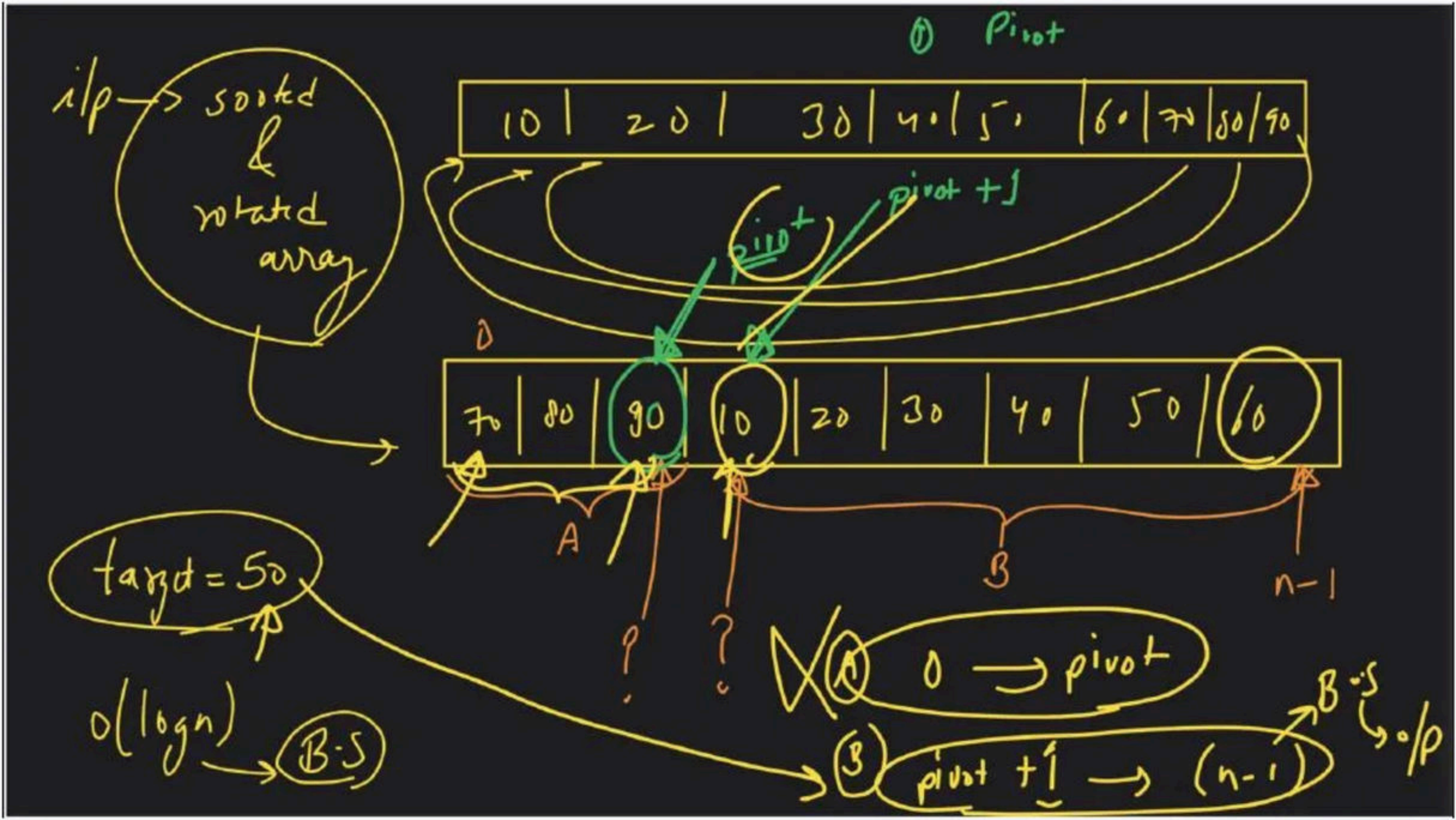

A
50
60
70
10
20
30
40
arr
0
1
2
3
4
5
6
if (arr[mid] < arr[mid-1]
mid
A
right
if arr[A] > arr[mid]
B
left
mid
B
B → left
70
60
50
A
mid
40
30
20
10
B
if (arr[mid] > arr[mid+1])
pivot

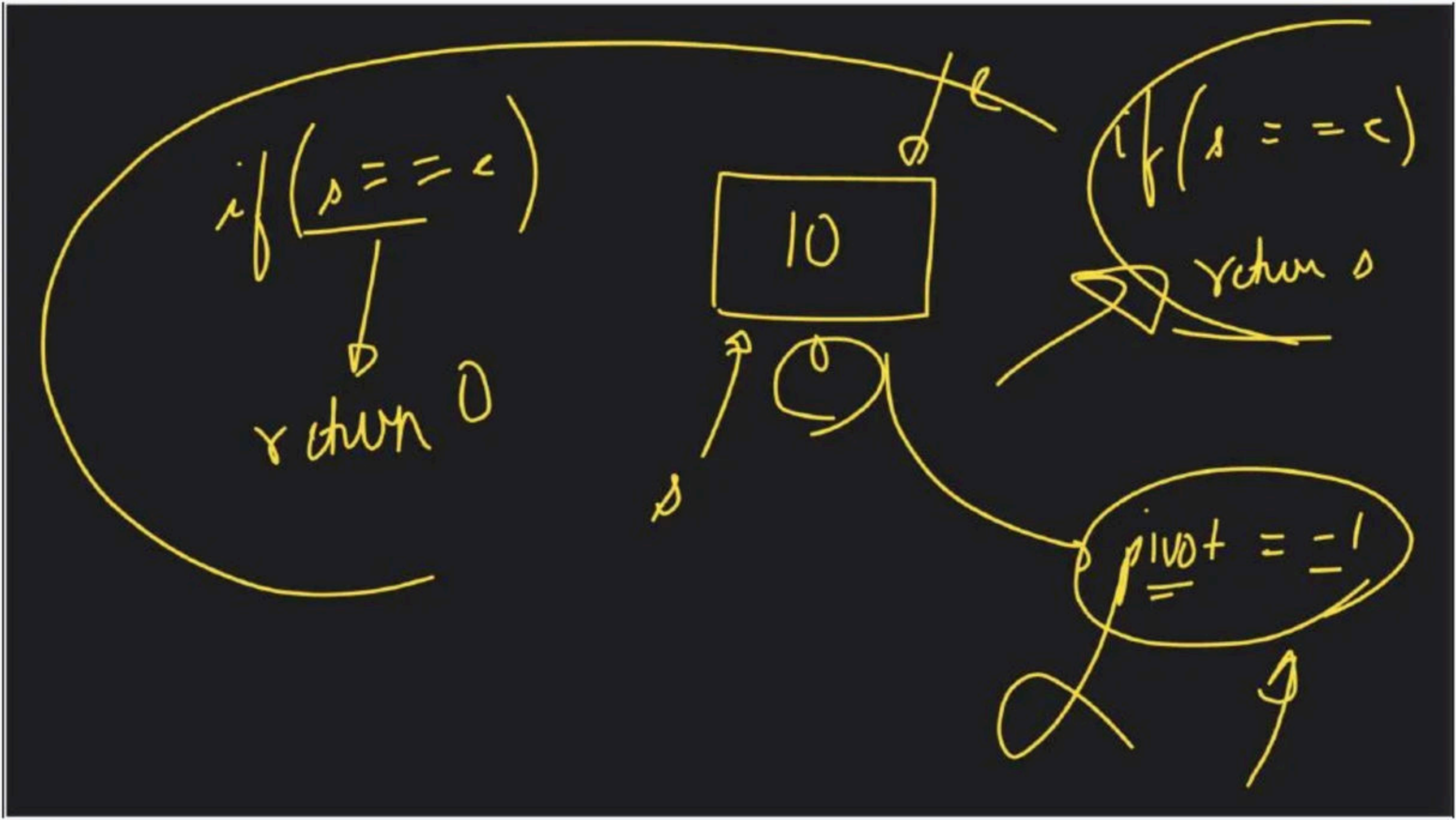

0 1 2 3 4 5 6 7 8 9 10
90 100 110 120 8 9 10 20 30 40 50
s
e
mid
A

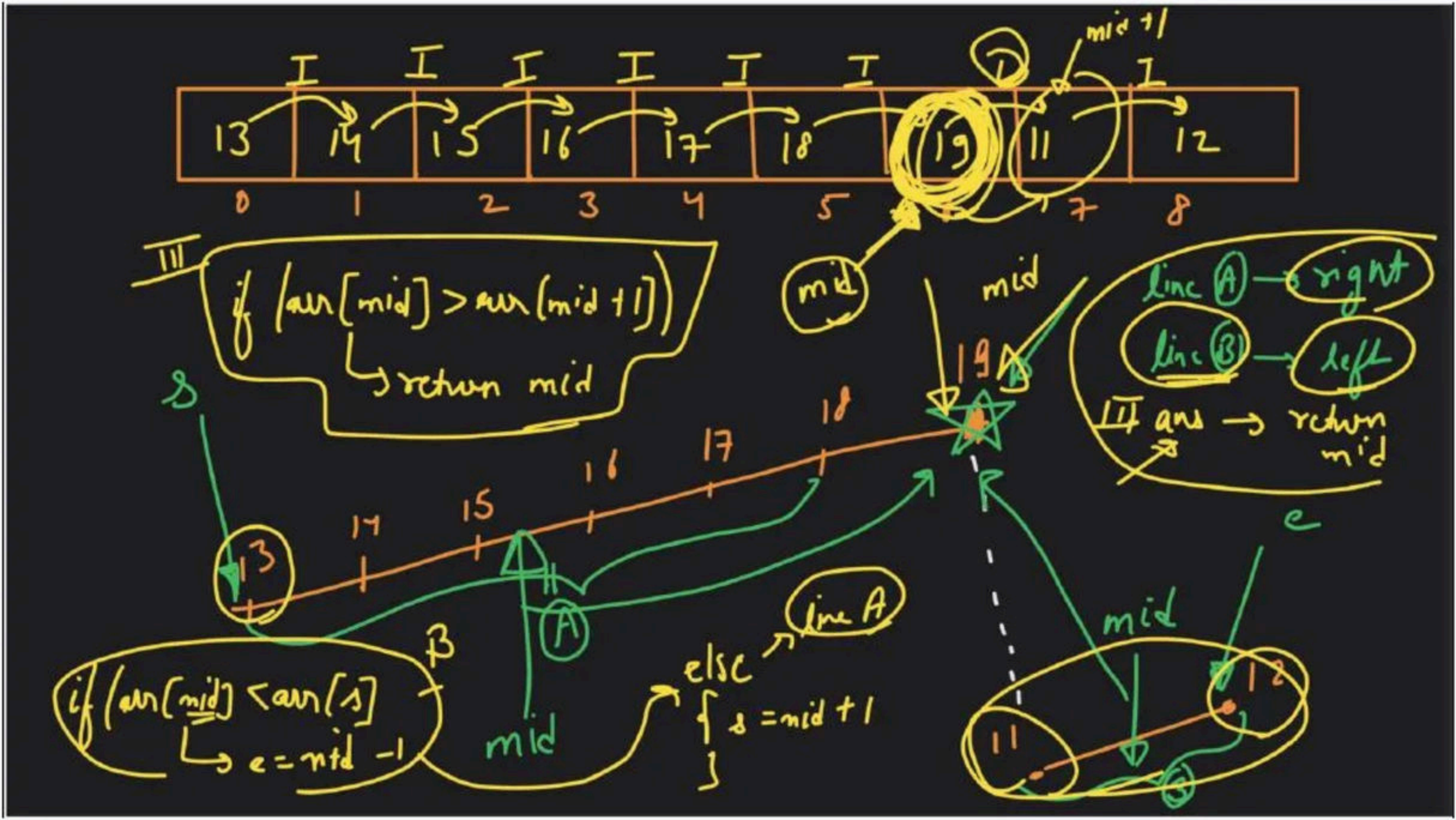

50
60
70
80
10
20
30
40
0
1
2
3
4
5
6
7
arr
mid
mid+1
arr[mid] < arr[mid+1]
return mid
if(arr[mid] < arr[mid-1])
return mid-1
left
right
line A
line B
arr[mid] < arr[0]
arr[mid] < arr[5]
OBJ:-
inc order
inc 3 element
line B element < line A element
B


line < B
if (arr[mid] < arr[s])
left → e = mid - 1
else
right → s = mid + 1
if (arr[mid] ≥ arr[mid+1])
return mid
if (arr[mid] < arr[mid-1])
return mid - 1


T/F
40
50
60
10
20
30
A
pivot
B
T
10 < 20 < 30
40 < 20 < 60
target
20


i/p
sorted & rotated array
① Pivot
10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90
Pivot
Pivot +1
0
70 | 80 | 90 | 10 | 20 | 30 | 40 | 50 | 60
A
B
n-1
target = 50
O(log n)
B.S
?
?
0 → pivot
pivot +1 → (n-1)
B.S o/p

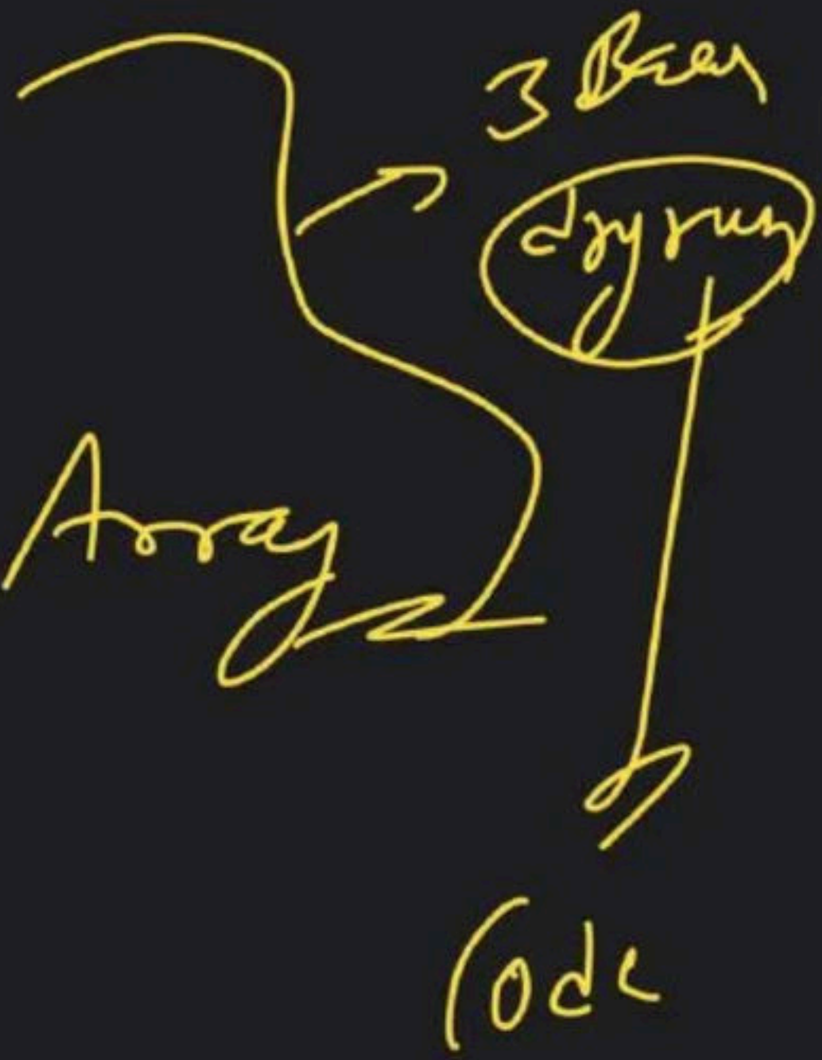

if (s == c)
return 0
10
s
if (s == c)
return s
pivot = -1


13
14
15
16
17
18
19
11
12
0
1
2
3
4
5
7
8
mid+1
mid
mid
if (arr[mid] > arr[mid+1])
return mid
line A → right
line B → left
ans → return mid
s
e
13
14
15
16
17
18
19
A
B
mid
if (arr[mid] < arr[s]
e = mid - 1
else
line A
{ s = mid + 1 }
mid
11
12

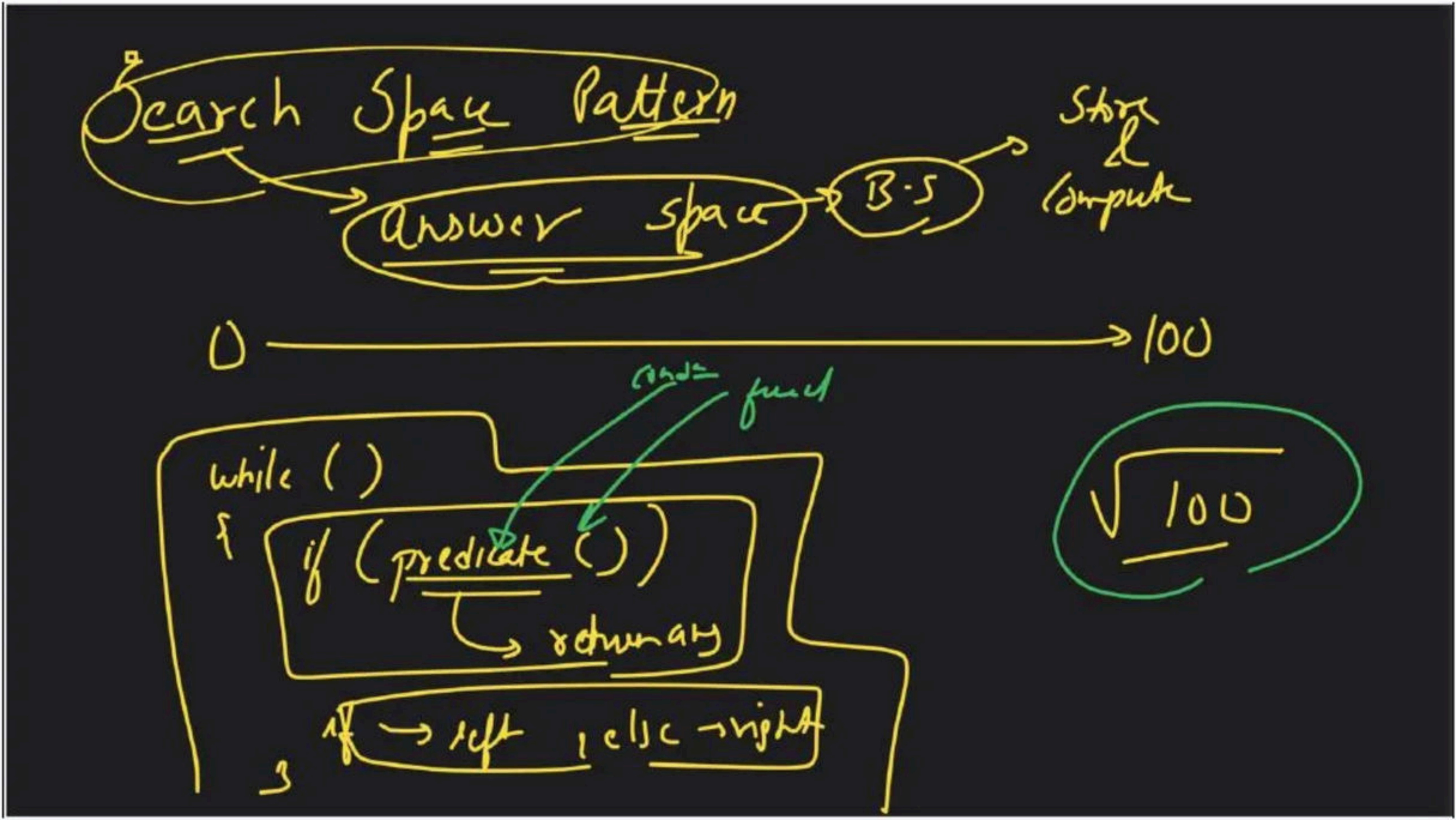
→ peak Index

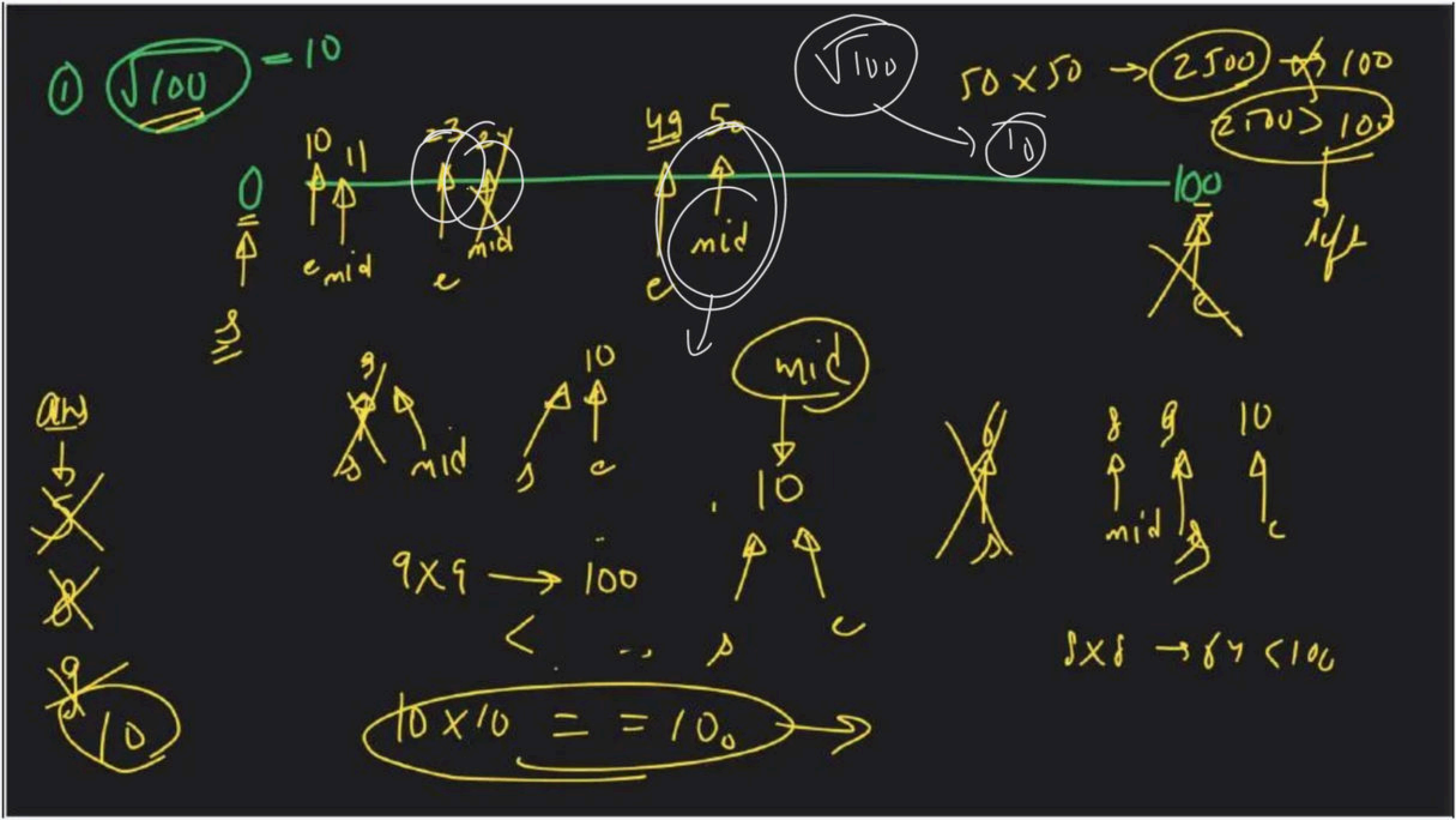
→ pivot Index

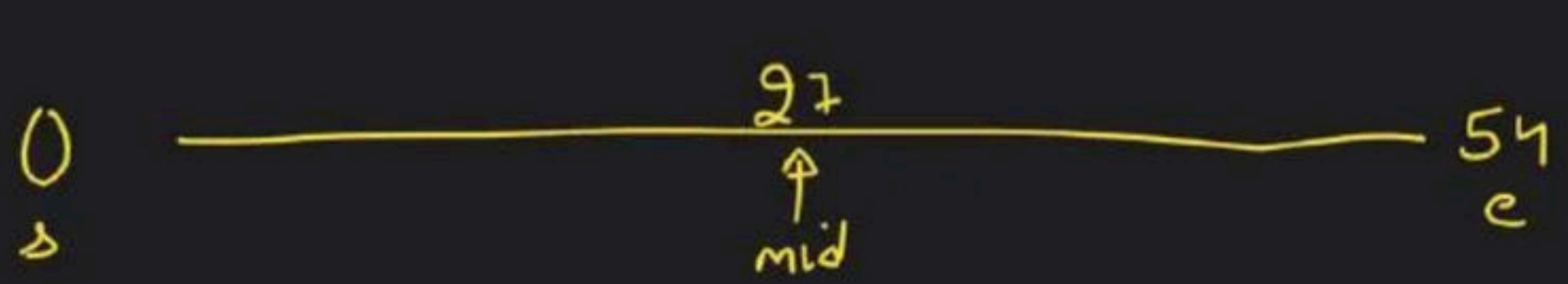
→ search → rotated Array

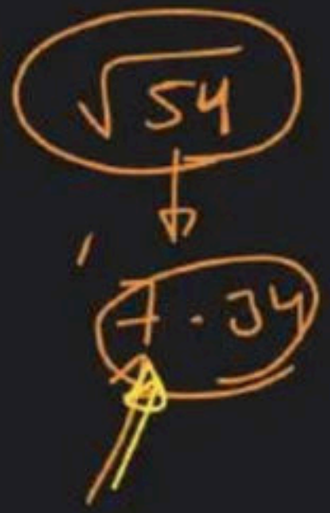



2 min
Break
B.S
CP


Search Space Pattern
Answer space
B.S
0
100
while ( )
{
if (predicate ())
return ans
if → left else → right
}
√100

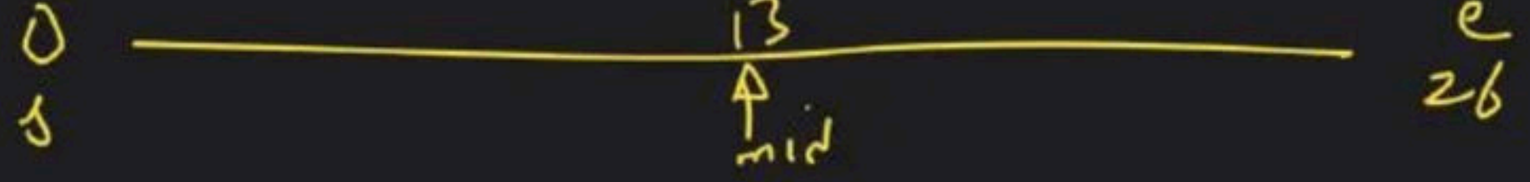

① √100 = 10
√100
50 × 50 → 2500
2500 > 100
2500 > 100
10
11
0
100
49
50
mid
mid
mid
mid
10
9
10
mid
10
9 × 9 → 100
10 × 10 = = 10
8 × 8 → 64 < 100
8
9
10
mid
10





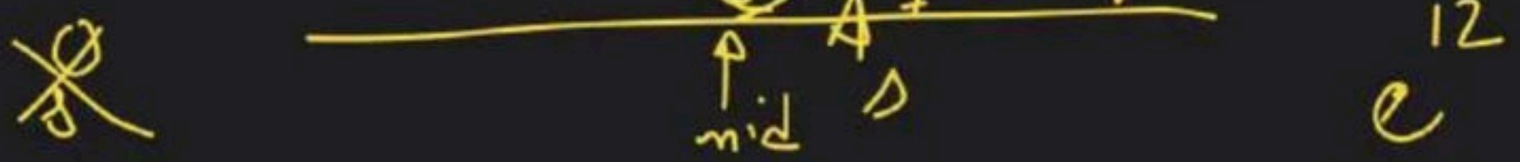
$27 \times 27 = 729 > 54$

↳ $e = mid - 1$ left

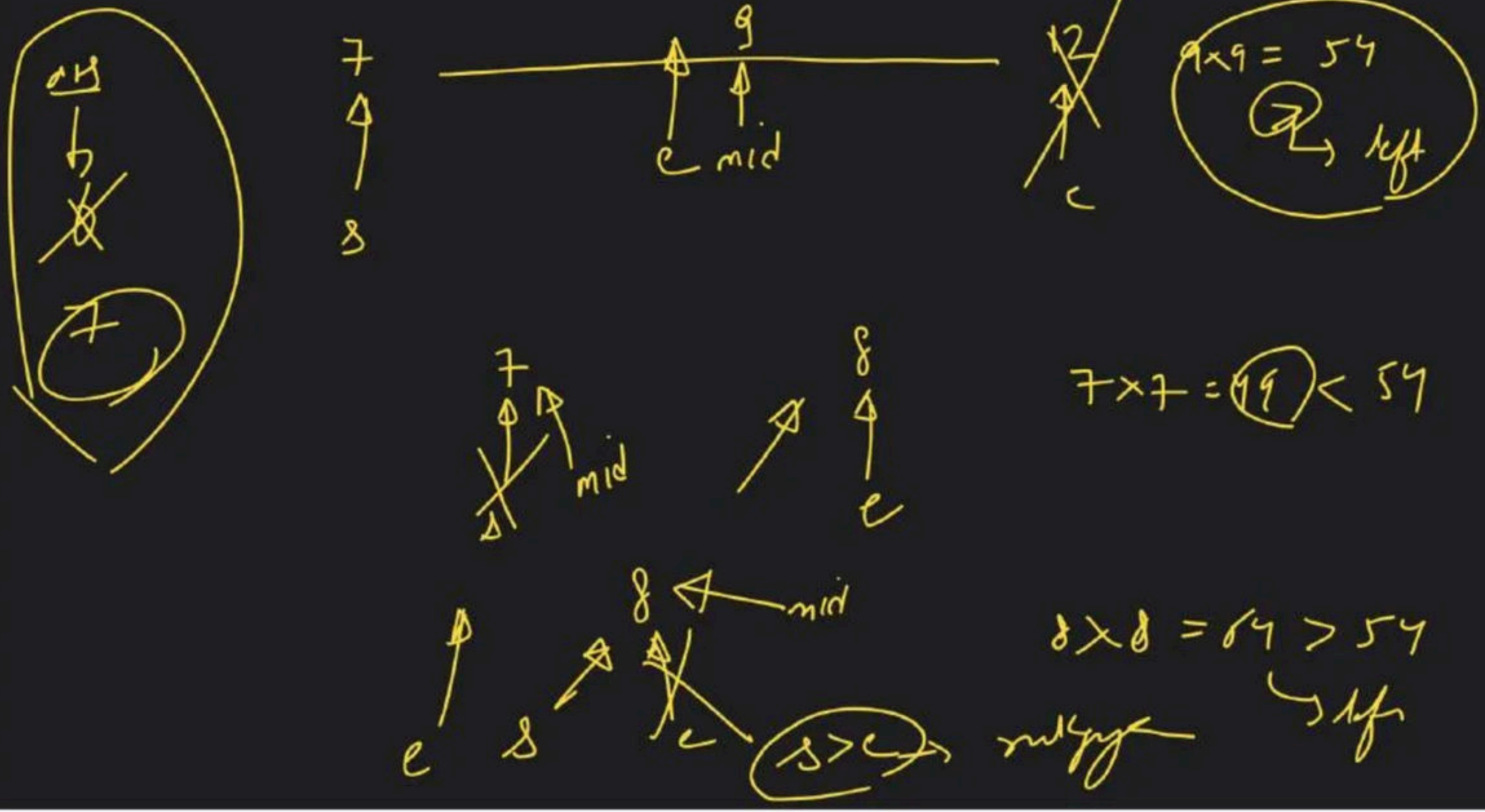


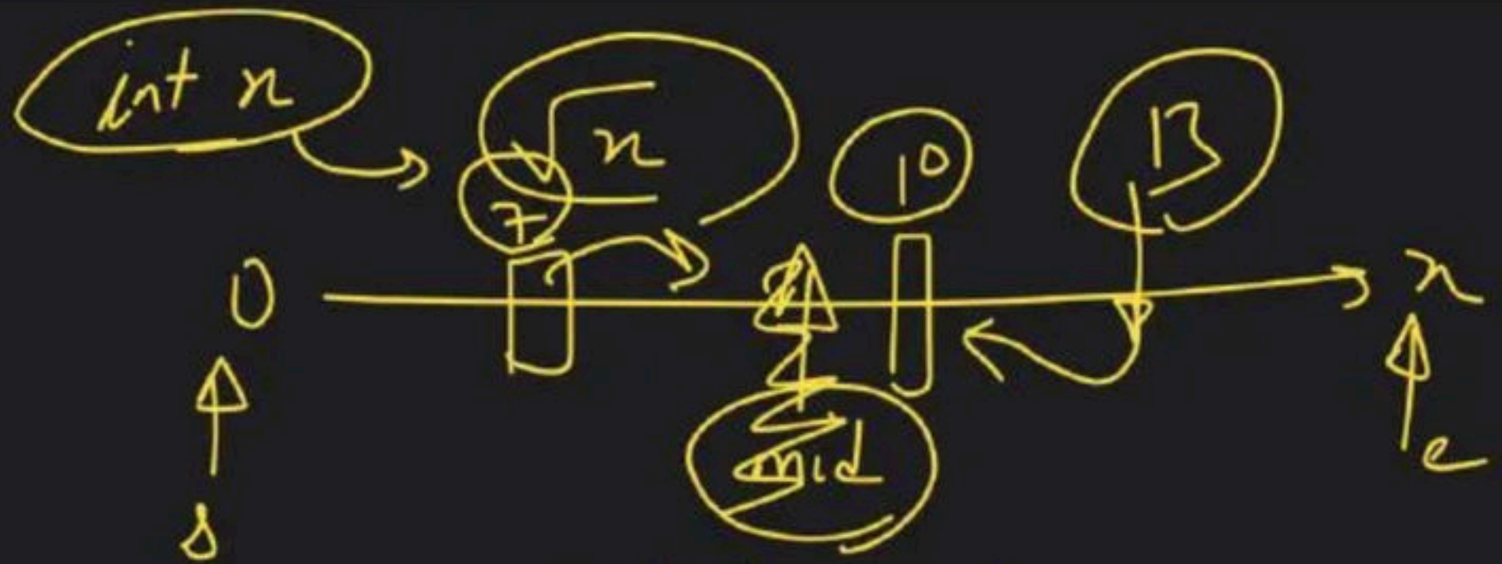
$13 \times 13 = 169 > 54$ ↳ left

6 store & compute



$6 \times 6 = 36 < 54$


7
s
9
e
mid
12
e
9×9 = 54
right
mid
7
s
8
e
7×7 = 49 < 54
8
mid
e
s
e
s > e
8×8 = 64 > 54
right

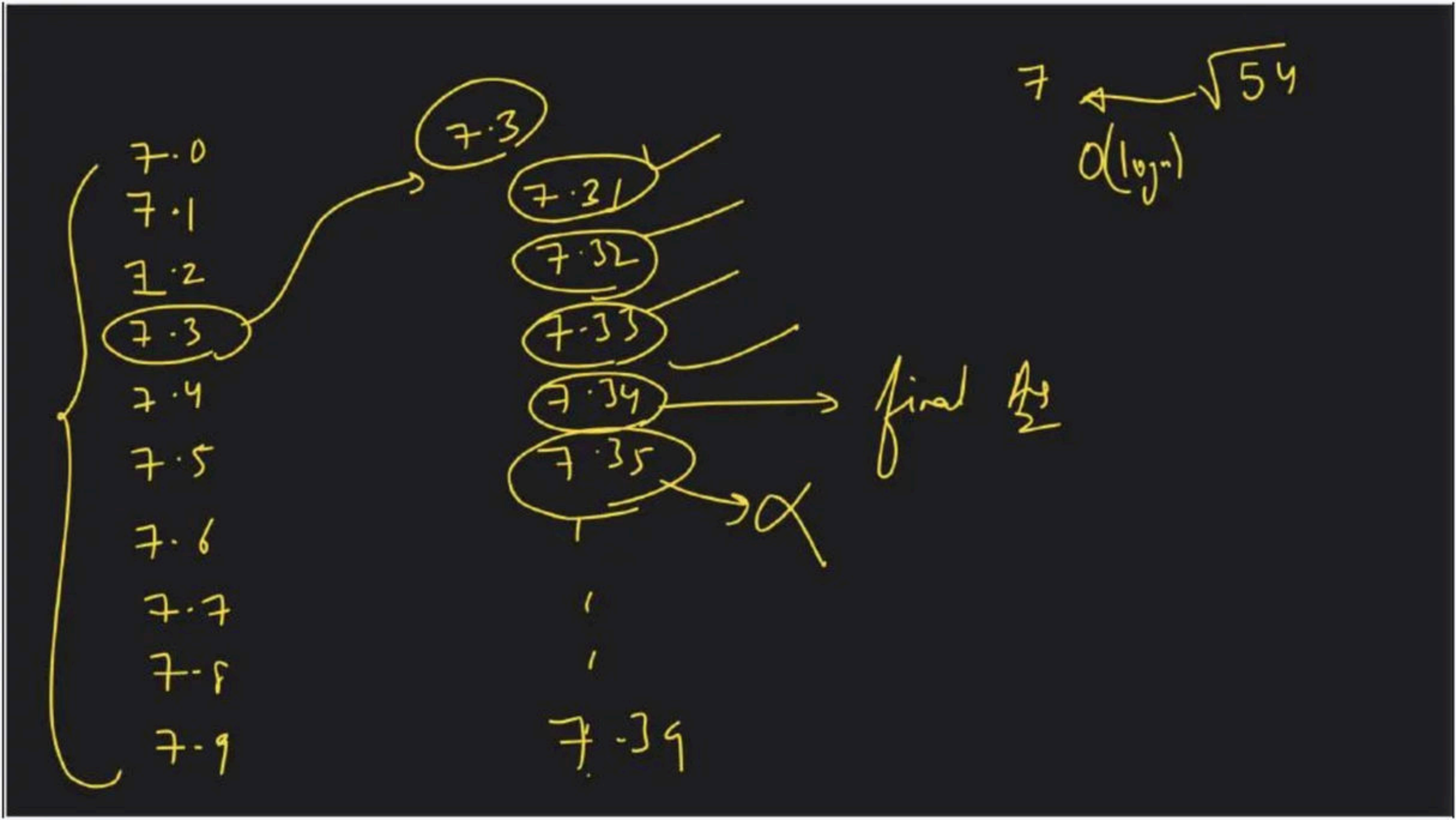


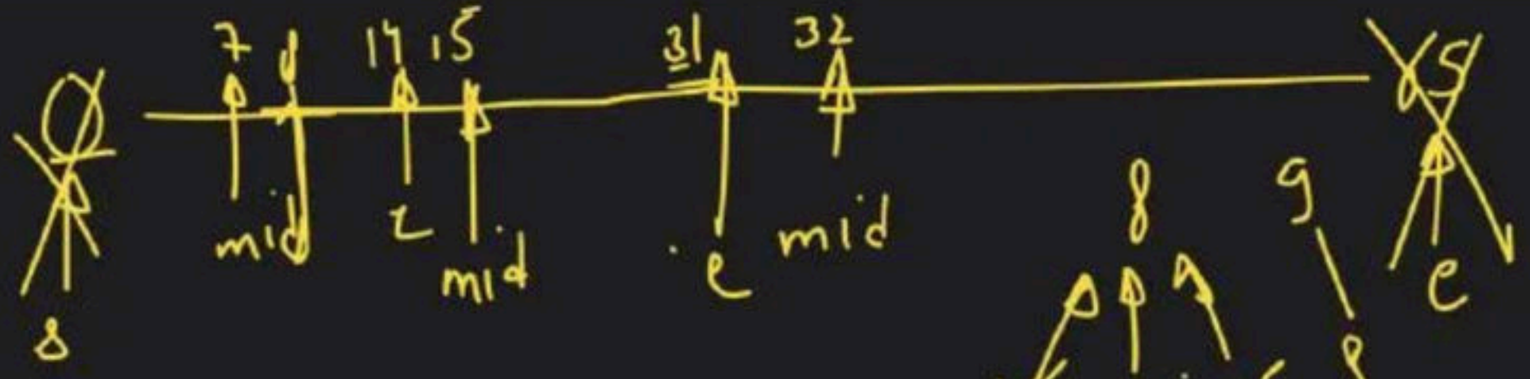

```
while( s<=e)
{
    if (mid * mid == n )
        return mid
    if ( mid * mid > n)
        right = e = mid - 1
    else
        left = s = mid + 1
}
```

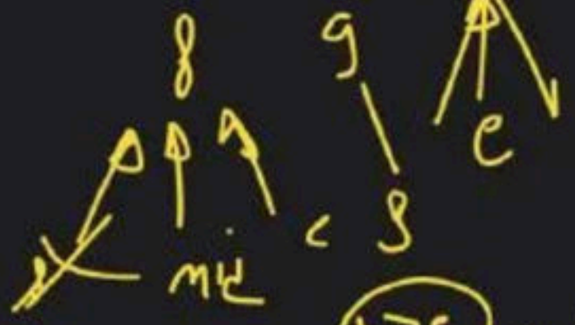
Kal

Xtra Class


√54
7
2
7·34


7 ← √54
O(log n)
7·0
7·1
7·2
7·3
7·4
7·5
7·6
7·7
7·8
7·9
7·3
7·31
7·32
7·33
7·34
7·35
7·39
find it
α





$32 \times 32 \not< 65$

↳ right → s = mid + 1

$15 \times 15 < 65$

↳ left → e = mid - 1

$7 \times 7 = 49 = < = 15$

49 < 65

$\sqrt{65}$

8. — —

$1 \times 8 == 8 \not= 65$

<

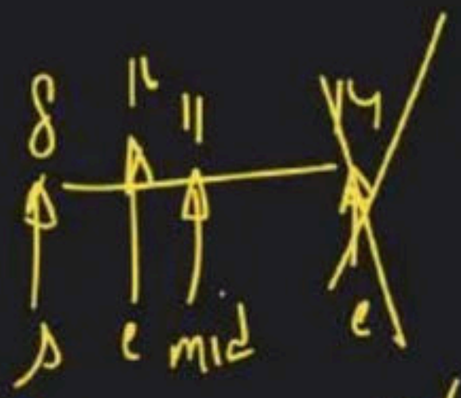
$11 \times 11 = 65$

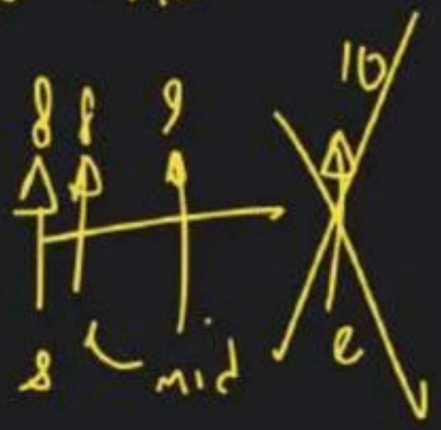
>

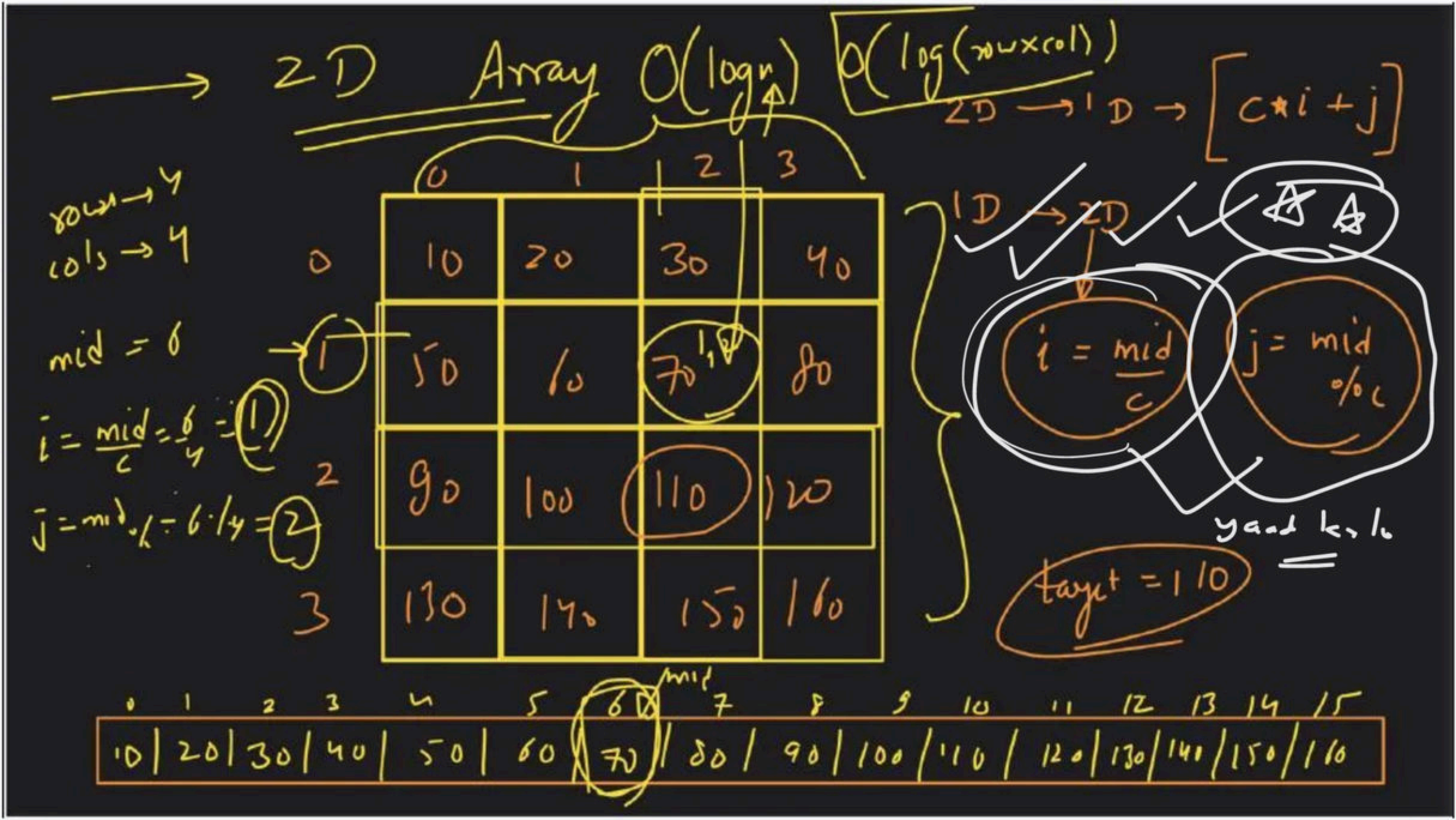
$9 \times 9 \not< 65$

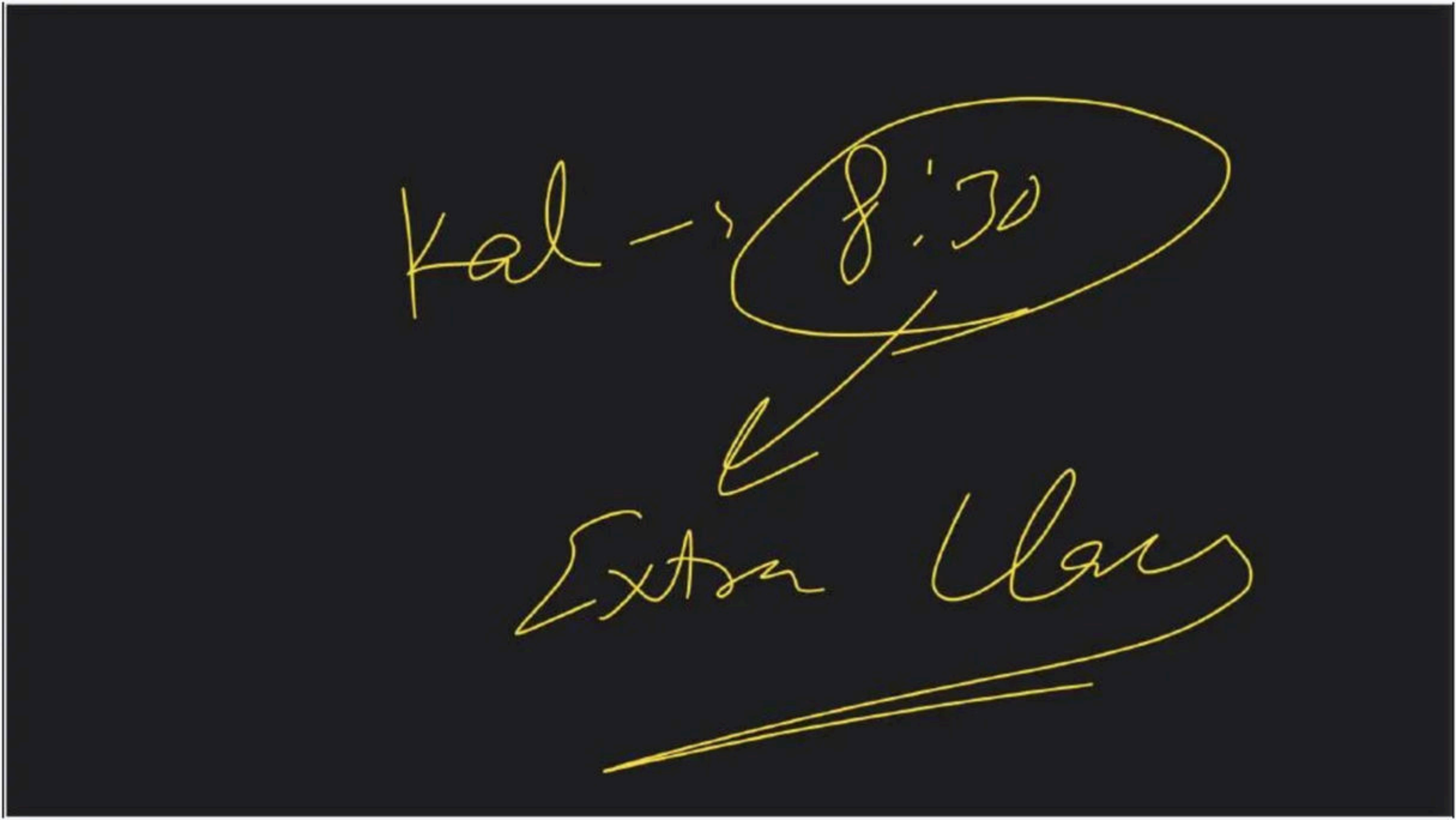
>

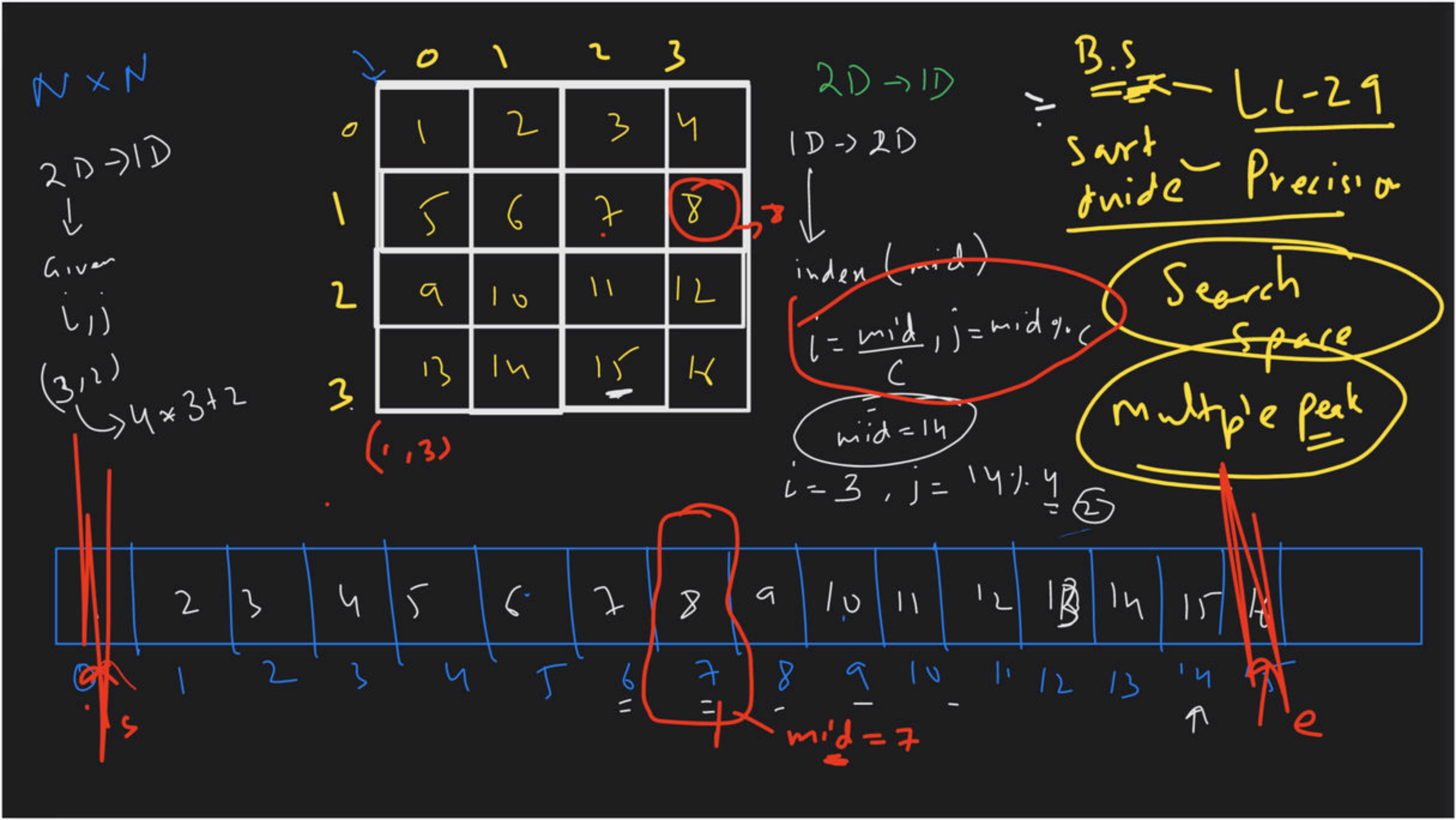







2D Array O(log n)
O(log(row×col))
2D → 1D → c*i + j
row → 4
col → 4
mid = 6
i = mid/c = 6/4 = 1
j = mid%c = 6%4 = 2
0 1 2 3
0 10 20 30 40
1 50 60 70 80
2 90 100 110 120
3 130 140 150 160
1D → 2D
i = mid/c
j = mid%c
yaad karlo
target = 110
mid
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
10 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160


Kal - 8:30
Extra Hour


N × N
2D → 1D
Given
(i,j)
(3,2)
4 * 3 + 2
0 1 2 3
0 1 2 3 4
1 5 6 7 8
2 9 10 11 12
3 13 14 15 16
(1,3)
2D → 1D
1D → 2D
index (mid)
i = mid / C , j = mid % C
mid = 14
i = 3 , j = 14 % 4 = 2
B.S
LL-29
Sort
Guide
Precision
Search Space
Multiple Peak
2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
s
e
mid = 7

2D ⟶ 1D ⟶ (c * i + j)

1D ⟶ 2D ⟶ mid index pr

standard Korrection

rowIndex = mid / columns

colIndex = mid % columns

Searching and Sorting Class - 3

Special class

Love Babbar • May 30, 2024

→ i/p →

Divisor = 2

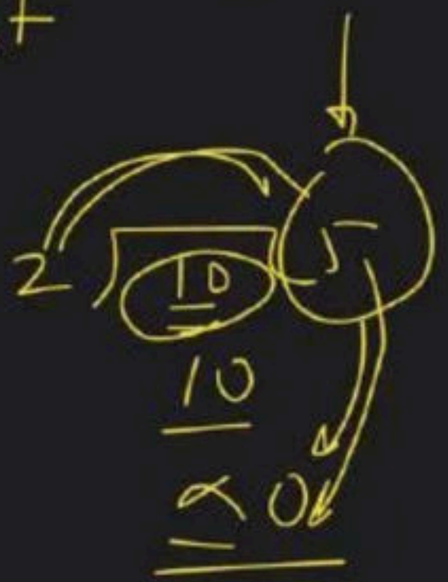
dividend = 10

Divisor

Dividend

Quotient

Quotient



Remainder

2 * 5 + 0 = 10

divisor * Quotient + Remainder = Dividend

dividend = 10

divisor = 2

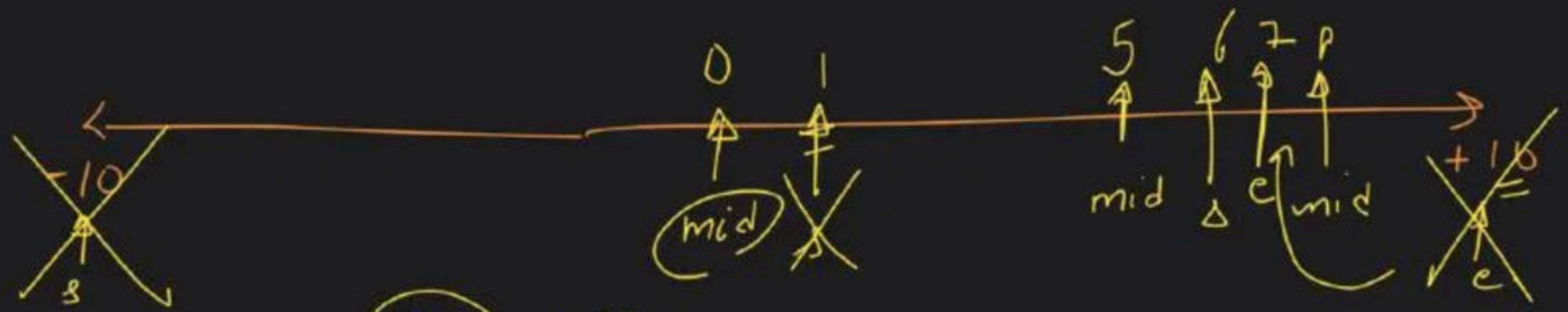
10

-1



dividend = 10 , divisor = 2

Quotient → mid



any

10

5

mid → 0

divisor * Quotient

2 * 0 <= dividend

0 <= dividend

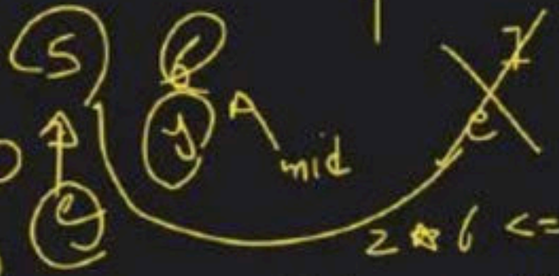
↳ 0

2 * 5 <= 10

10 <= 10

↳ T

5 > 2 so 5 <



<= Dividend

2 * 6 <= 10

12 <= 10

↳ F

2 * 8 <= 10

16 <= 10

↳ F



x + y = dividend

if y = 0

x = dividend

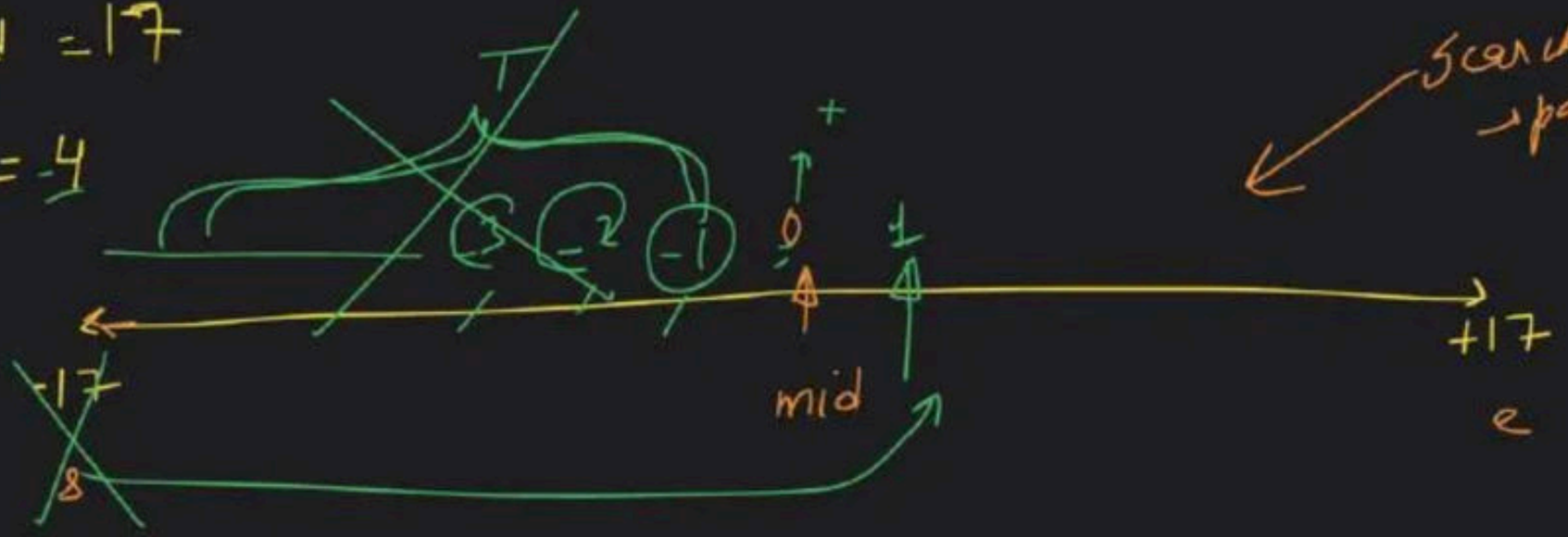
if y = +∞

x <= dividend

x < dividend

dividend = 17

divisor = -4



search space → any

Quotient

mid

any

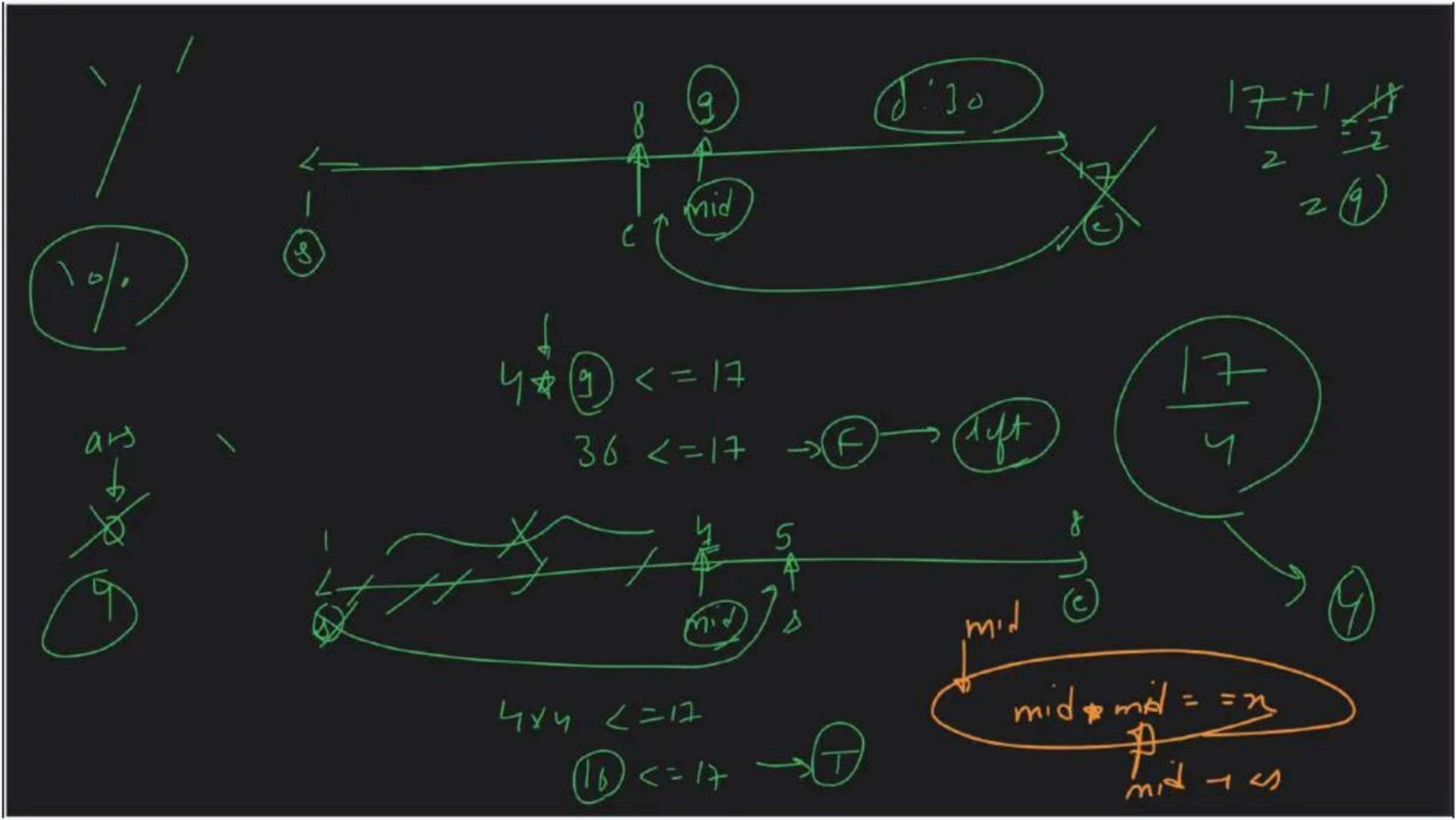
0

divisor * Quotient + ~~sum~~ = dividend

divisor * Quotient <= dividend

4 * 0 <= 17

0 <= 17 → T

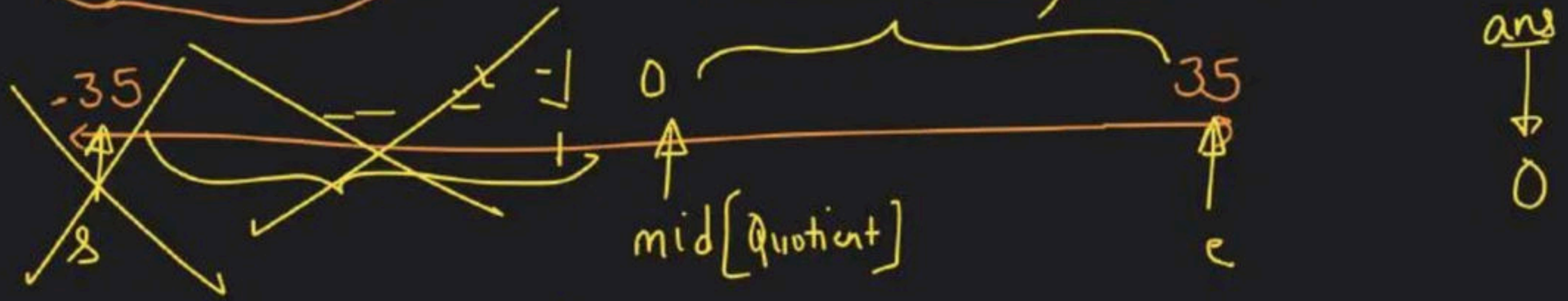

10/.
ans
9
1
8
9
d'10
mid
s
c
e
17+1/2 = 9
4 * 9 <= 17
36 <= 17 → F → left
17/4
4
1
4
5
8
mid
d
e
4x4 <= 17
16 <= 17 → T
mid
mid * mid == n
mid + 1

dividend = 35

divisor = 4

divisor * Quotient + ~~Remainder~~ (!0) = dividend

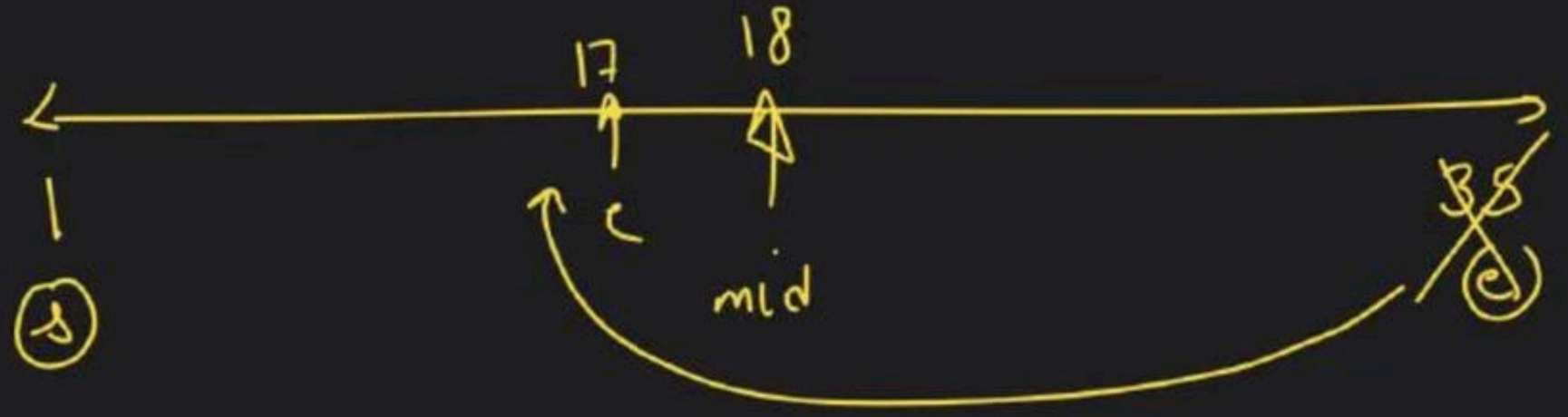
divisor * Quotient <= dividend



4 * 0 = 0 =< 35

0 < 35

↳ s = mid + 1



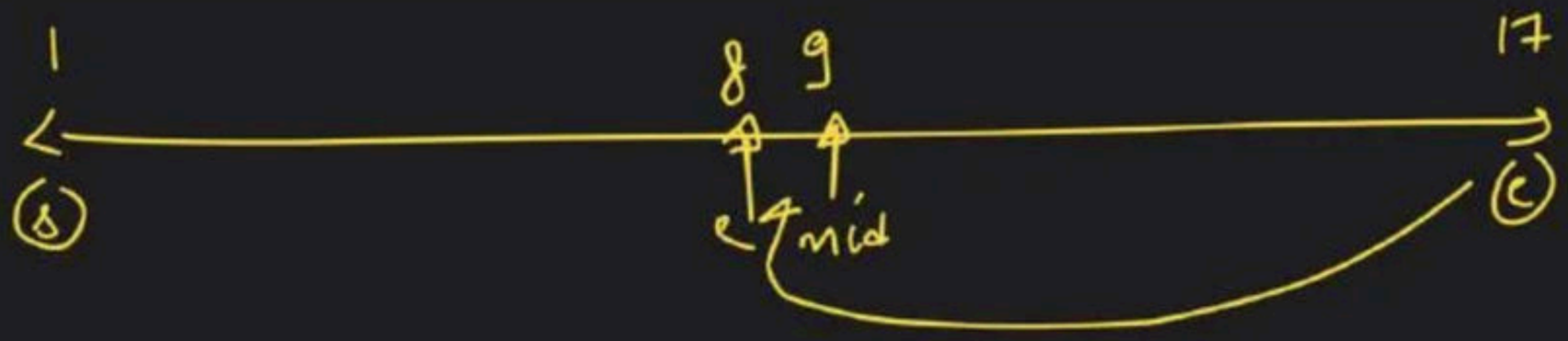
ans

0

$4 * 18 = 72 = \alpha = 35$

72 ⊃ 35

→ left



$4 * 9 = 36 = 35$

$\geq$ → left → e = mid - 1



$4 * 4 = 16 = 35$

S → s l c

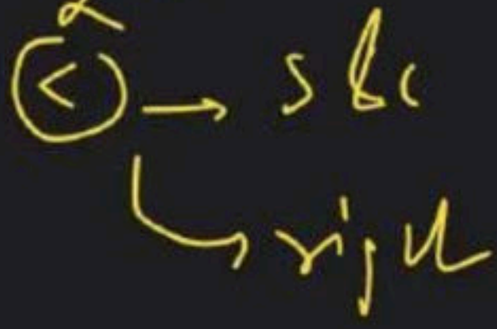
n | 4

s = mid + 1





4 * 6 = = 24 = = 35





4 * 7 = 28 <)35°

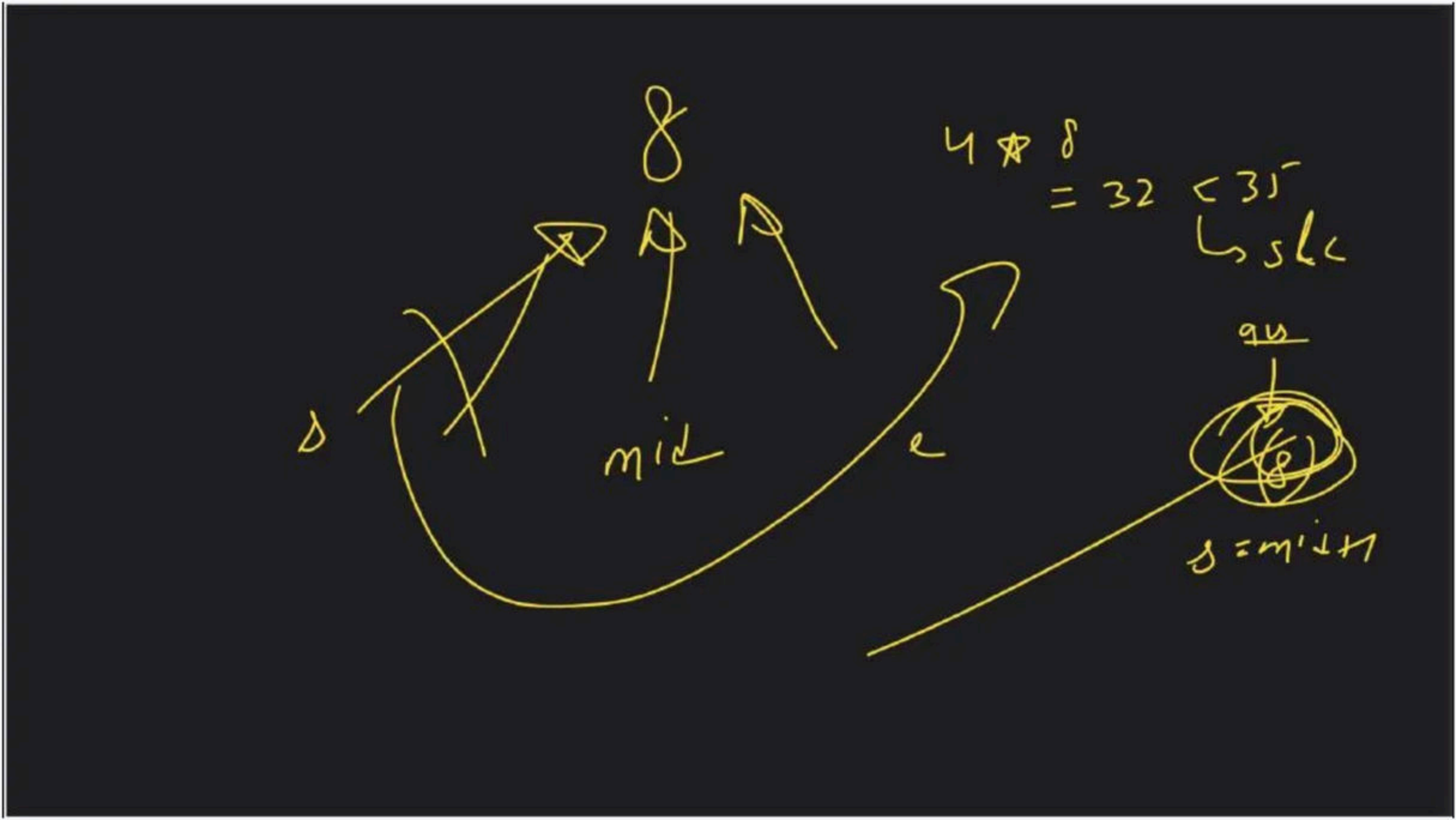
↳ s & c

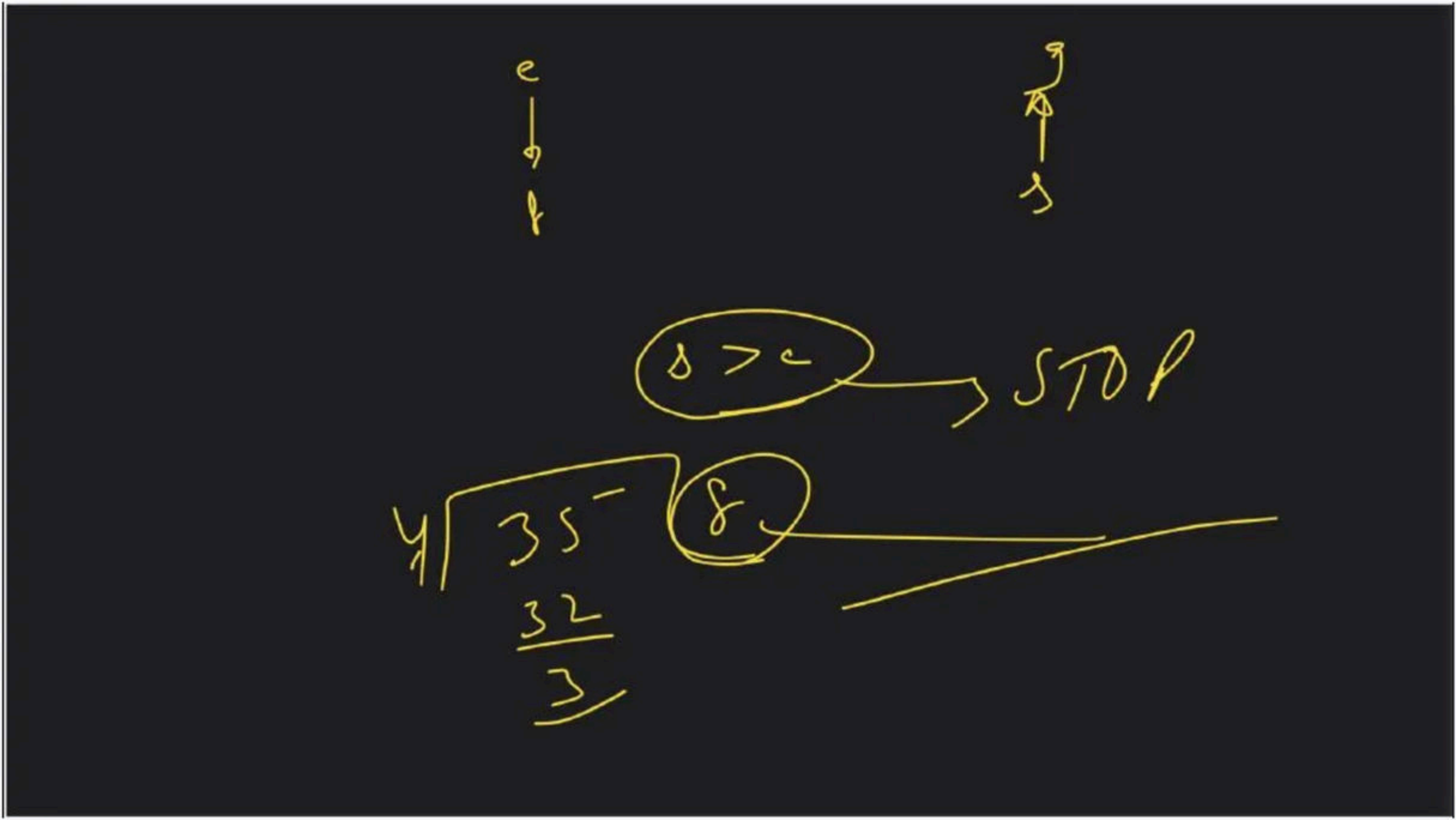
↳ s = mid th

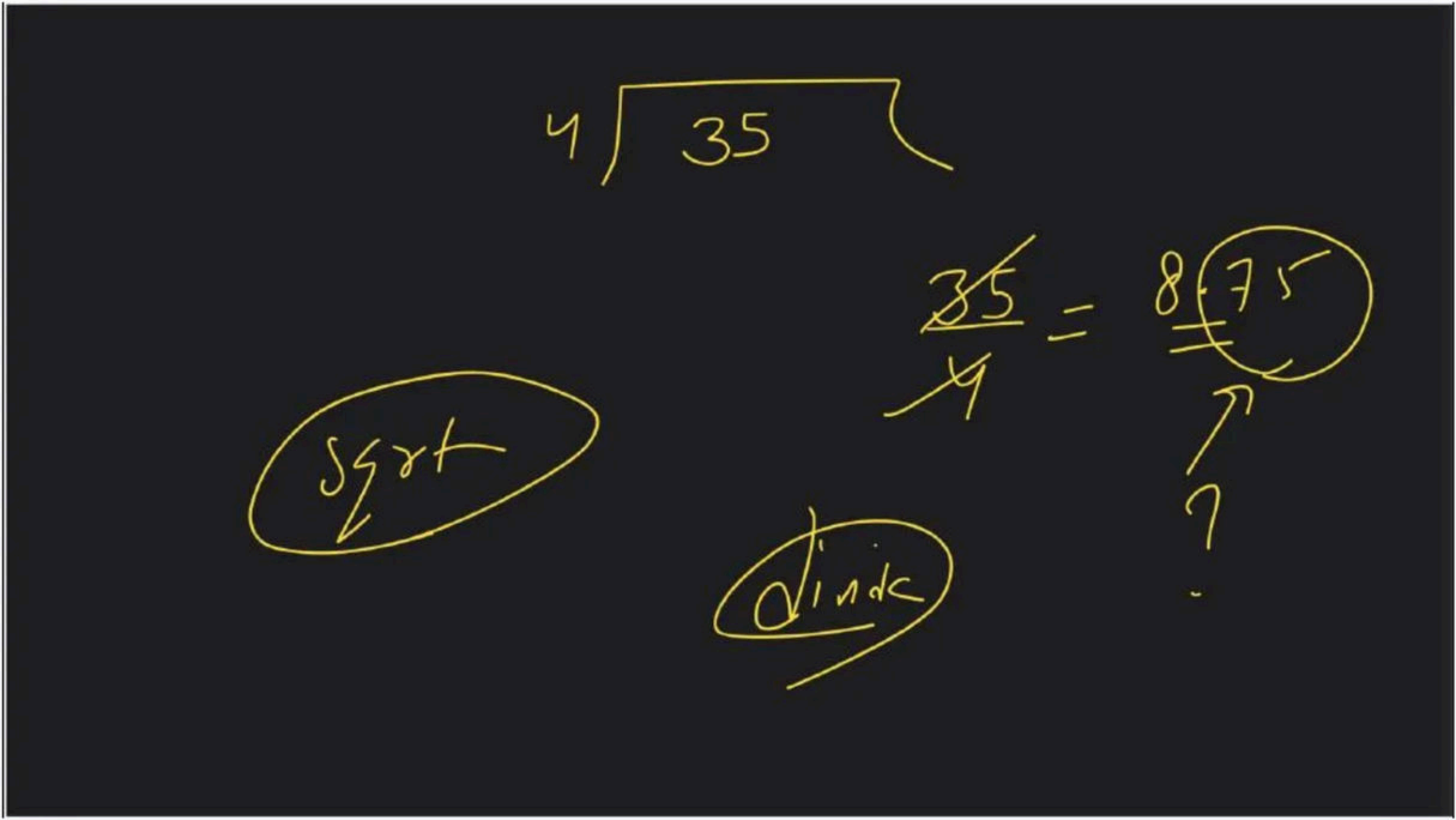
a/b

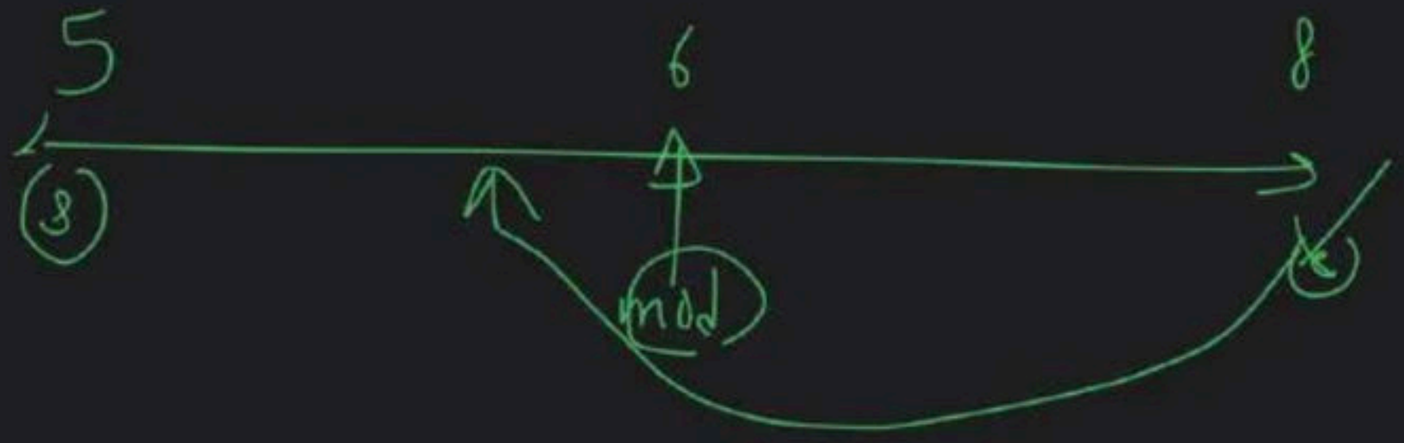
↓

7


8
4 * 8
= 32
s
mid
e
qs
s = mid


e
f
g
5
d > e
STOP
4
35
8
32
3


4 | 35
35/4 = 8.75
?
Sqrt
Find



$4 * (6) <= 17$

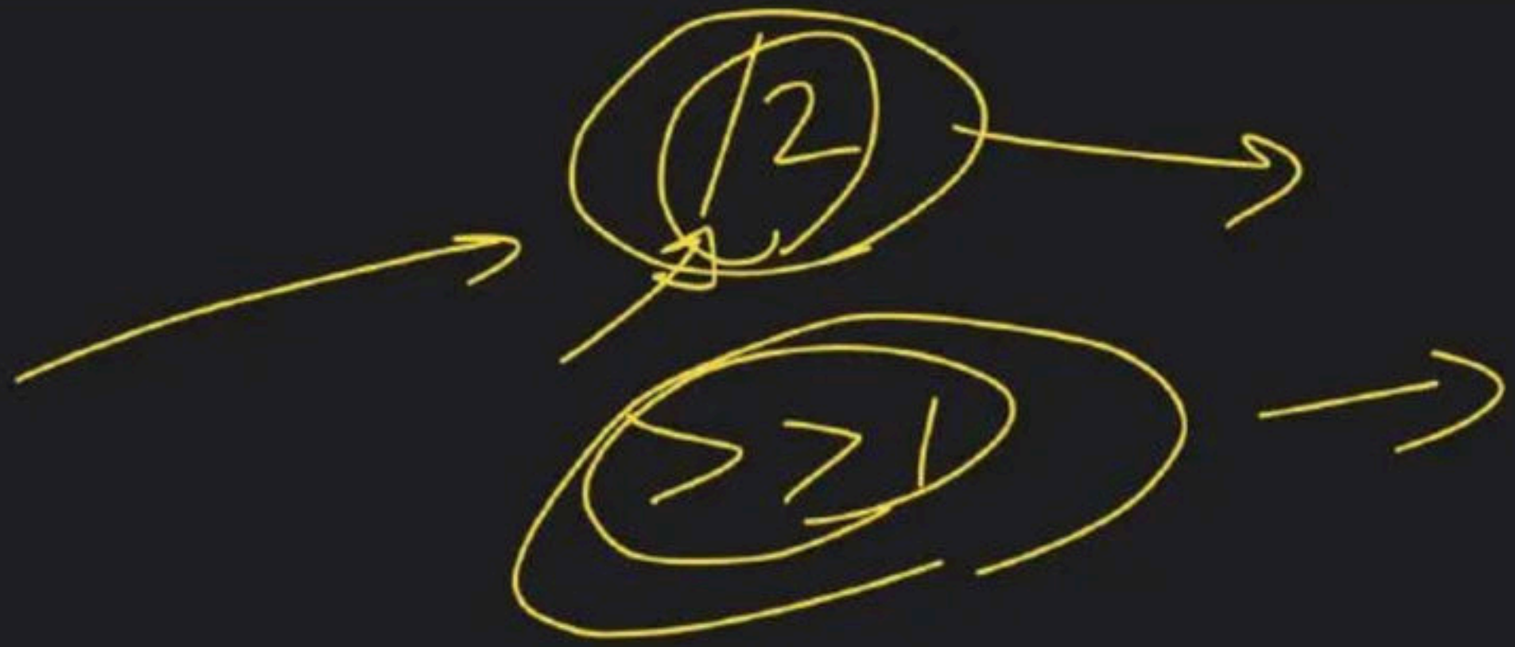
$24 <= 17 \rightarrow$ (F)



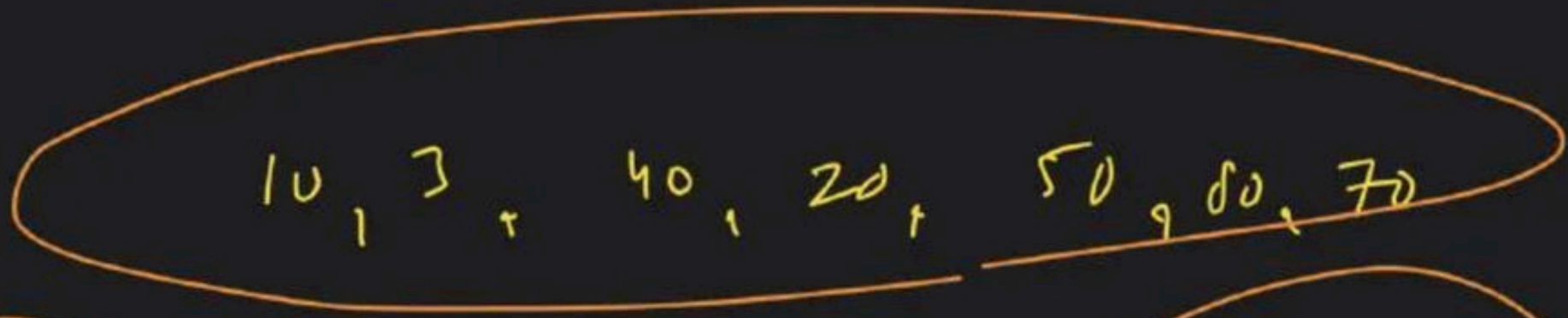
$4 * 5 <= 17$

$20 <= 17 \rightarrow$ (F) $\rightarrow$ 나옴

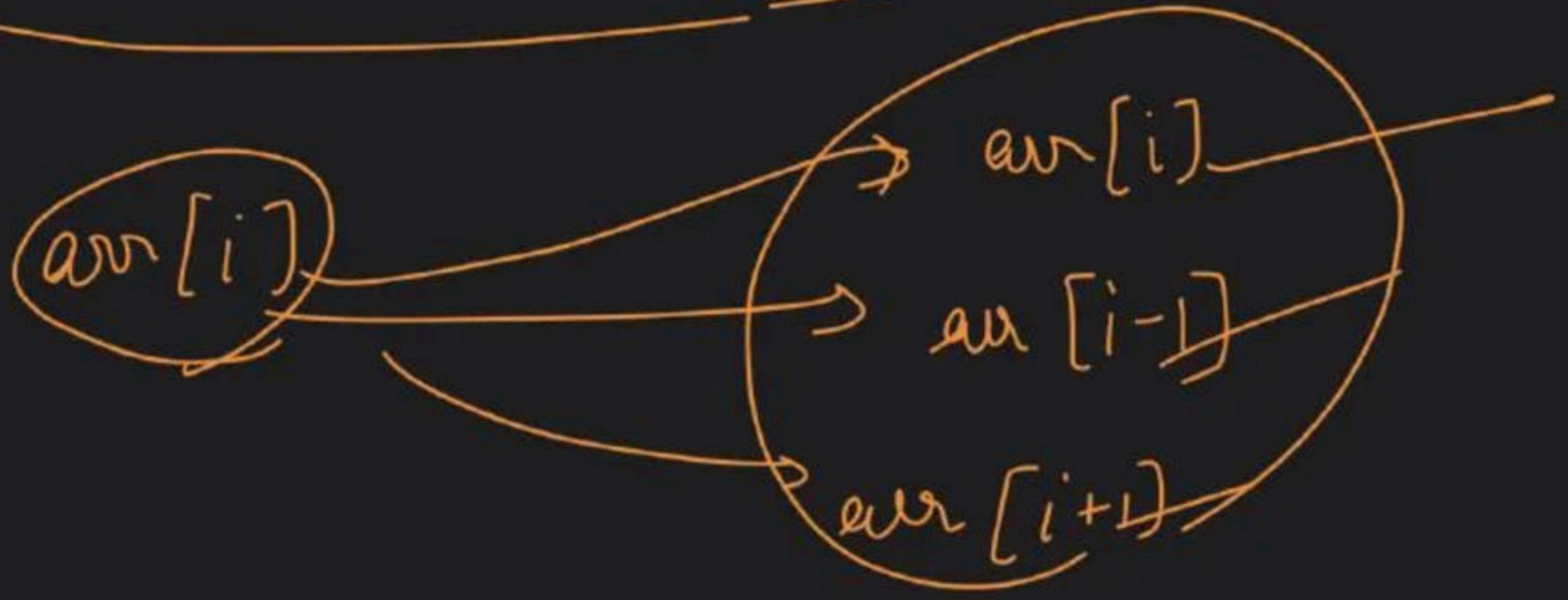
$c = 4 \quad s = 5 \rightarrow s > c \rightarrow$ 탈출

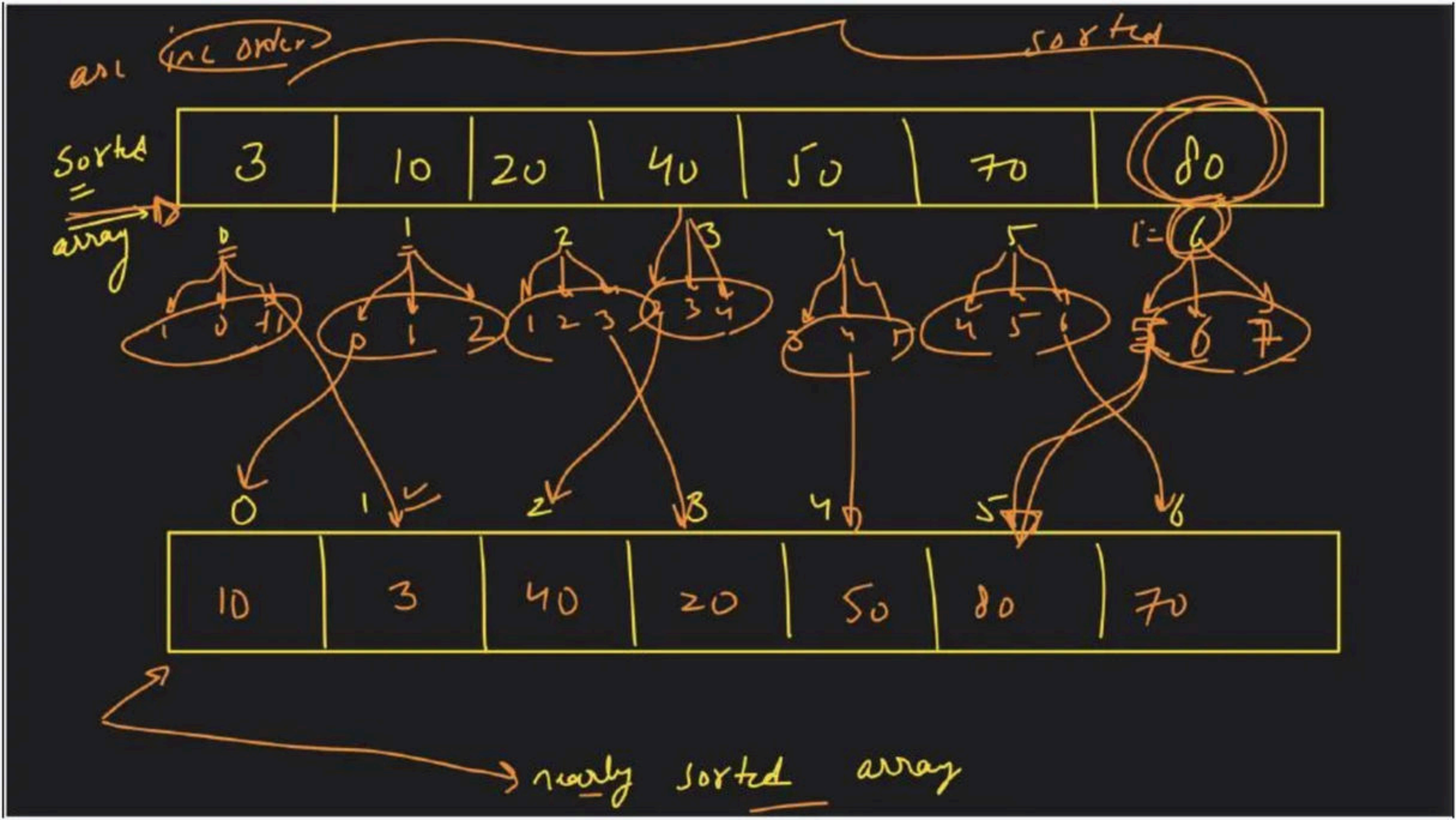

12
>21

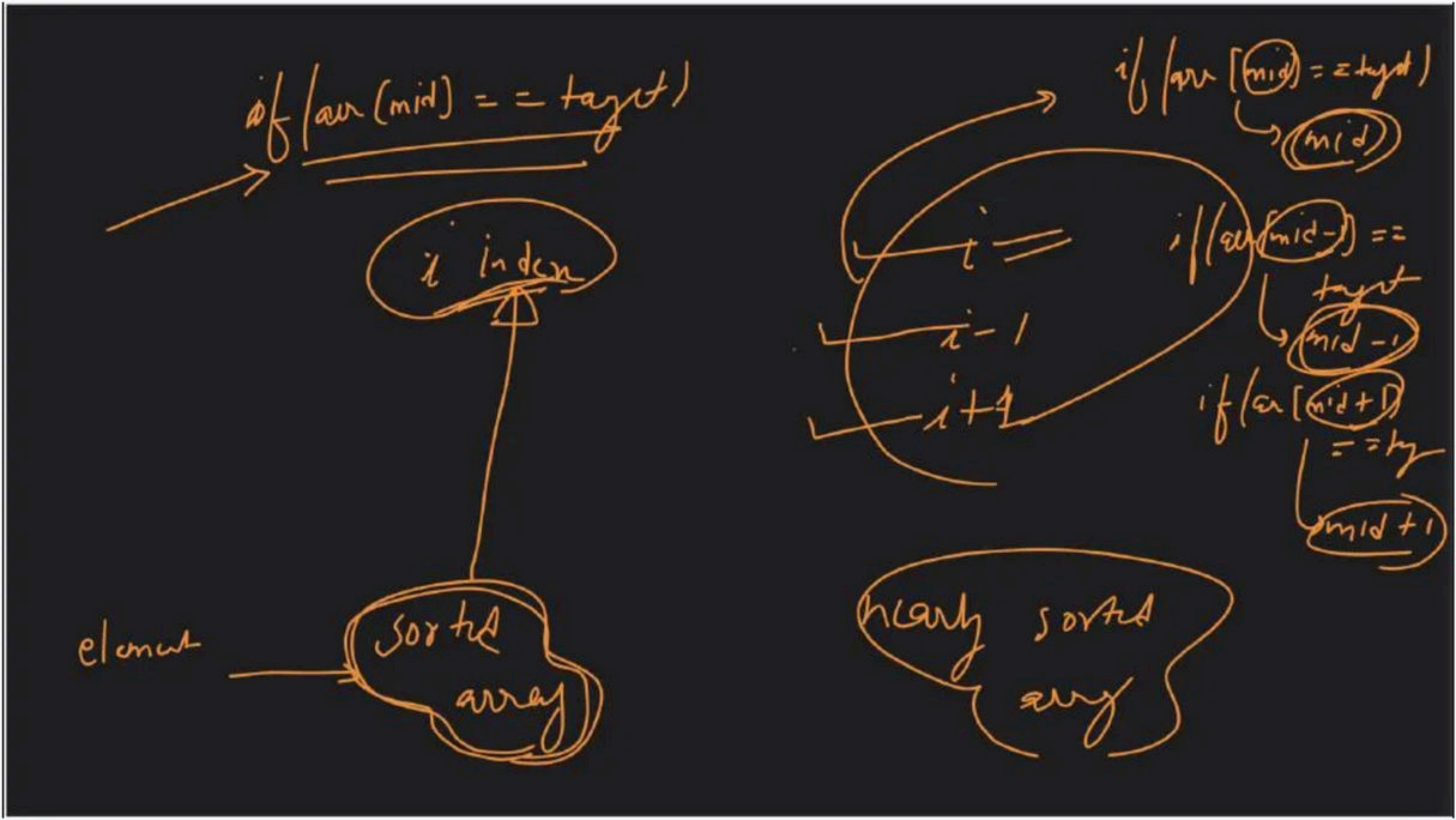
→ Search in a nearly / almost sorted array

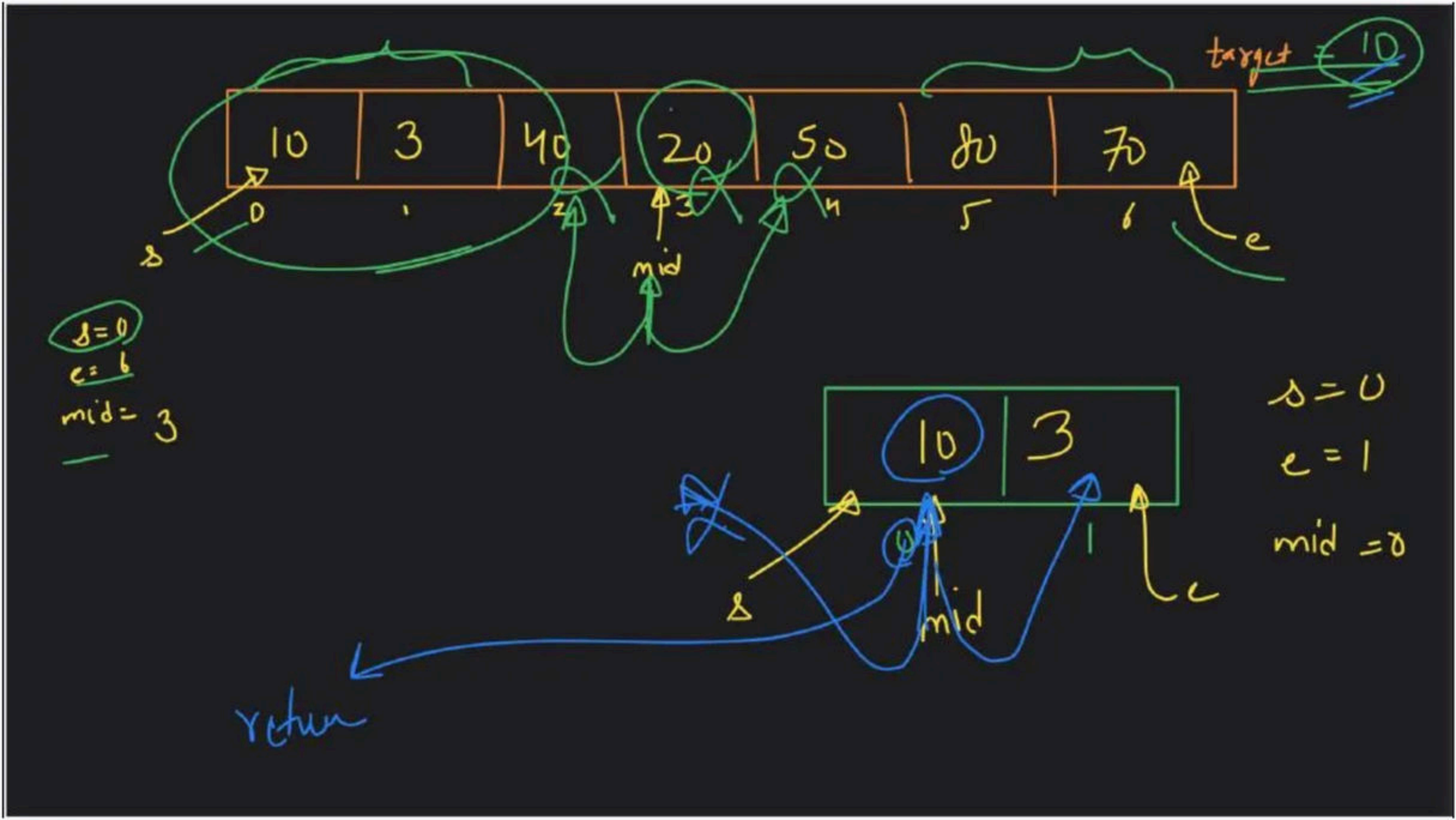




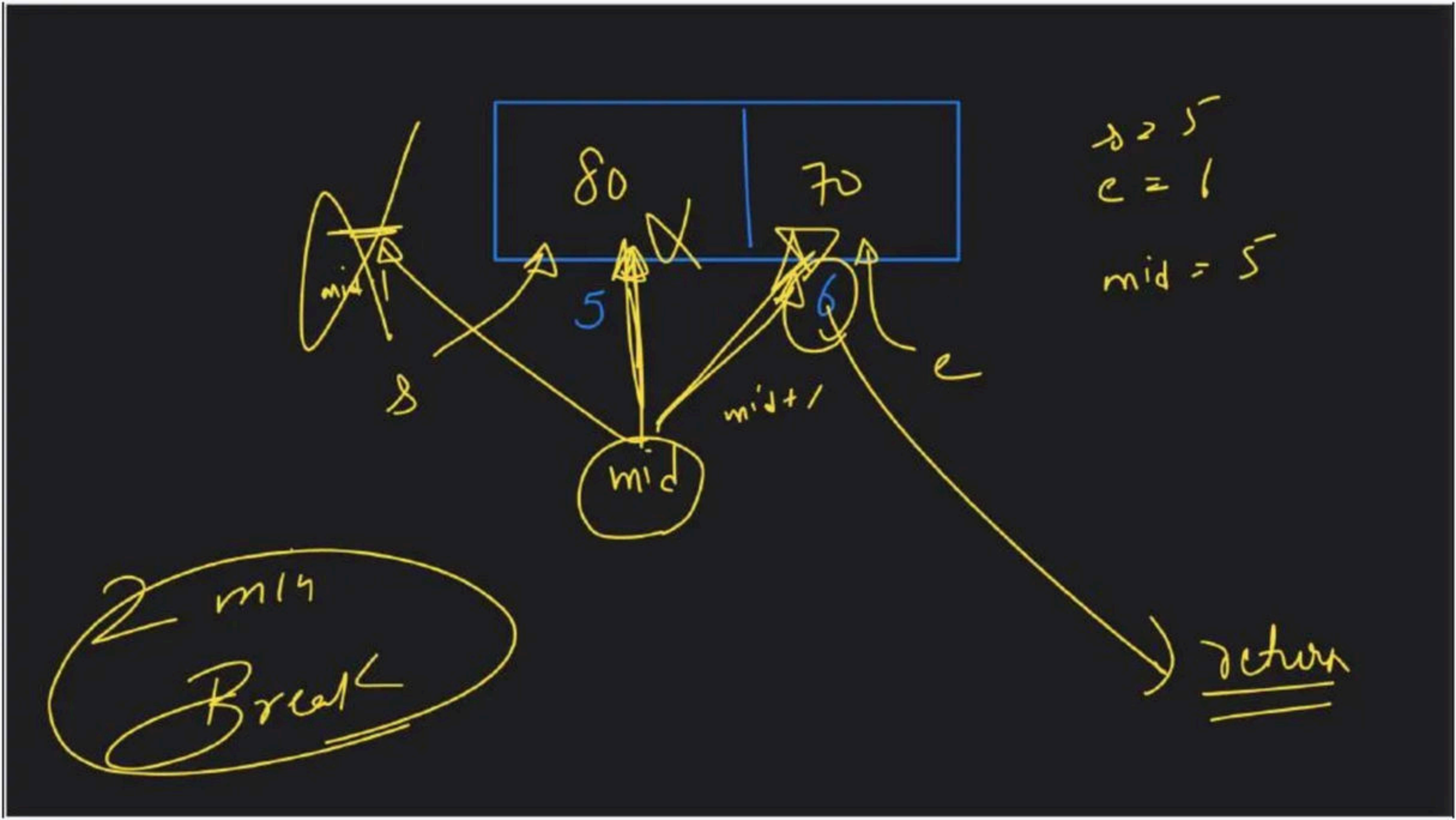


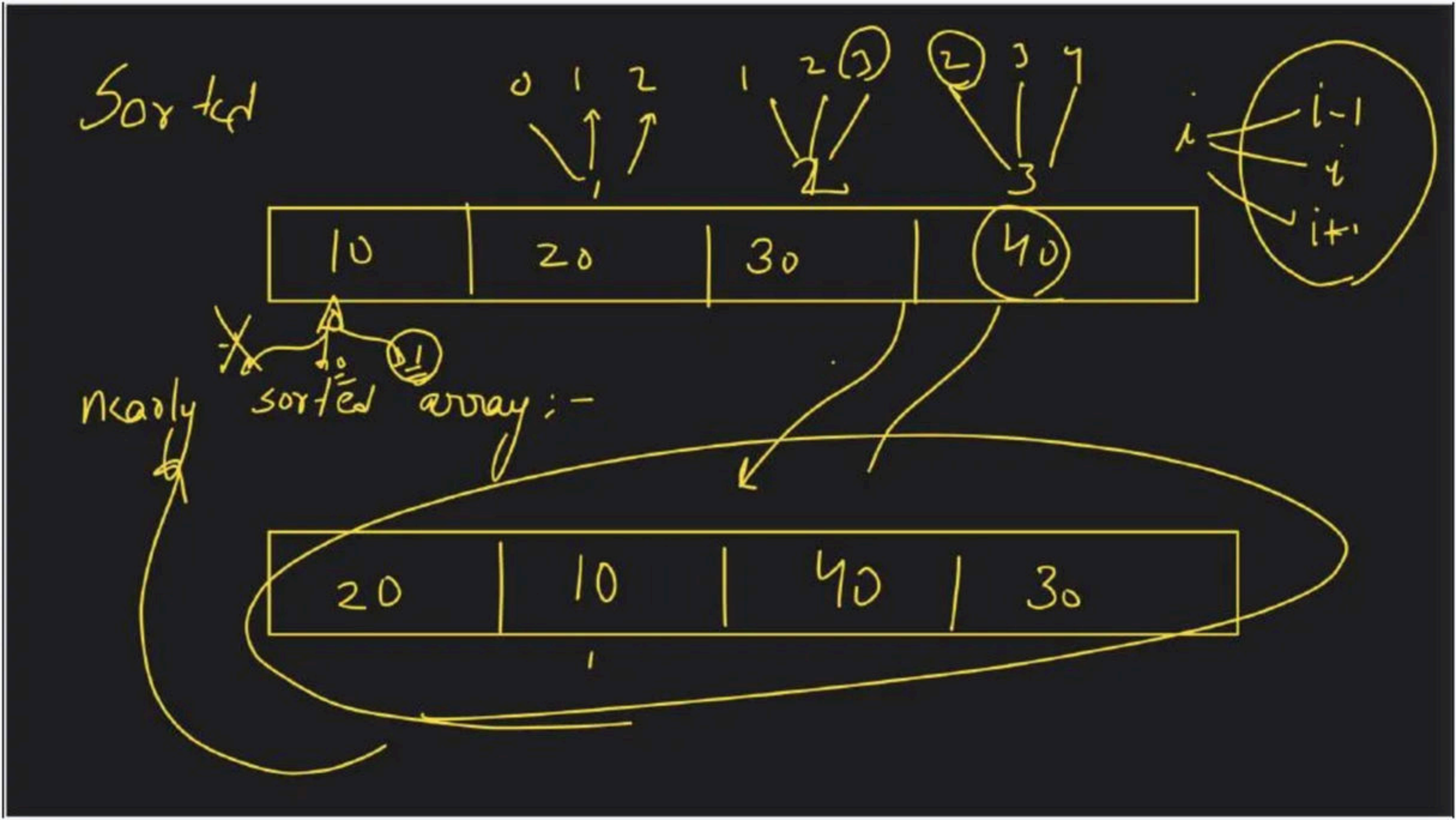

arr
inc order
sorted
sorted
=
array
3
10
20
40
50
70
80
i=6
0
1
2
3
4
5
6
7
10
3
40
20
50
80
70
nearly sorted array


if (arr[mid] == target)
index
element
sorted array
if (arr[mid] == target)
mid
target
mid - 1
mid + 1
nearly sorted array


target = 10
10
3
40
20
50
80
70
0
1
2
3
4
5
6
s
e
mid
s=0
e=6
mid= 3
10
3
s=0
e=1
mid =0
0
1
s
e
mid
return




80
70
5
6
mid-1
mid
mid+1
s
e
s = 5
e = 6
mid = 5
2 mid
Break
return


Sorted
0 1 2
1 2 3
2 3 4
i
i-1
i
i+1
10
20
30
40
nearly sorted array :-
20
10
40
30

Odd occuring element :-

sort() → O(n log n)

O(log n) → B.S

(A) all element occurs even number of times except one

(B) element repeats itself in pairs (1,1) (2,2) (3,7)

(C) no pair repeats itself, no number can occur more than 2 times in a single stretch

(1,1,1,3,1,1,1)

(d) find element that occur Odd time

- Single No. - 2

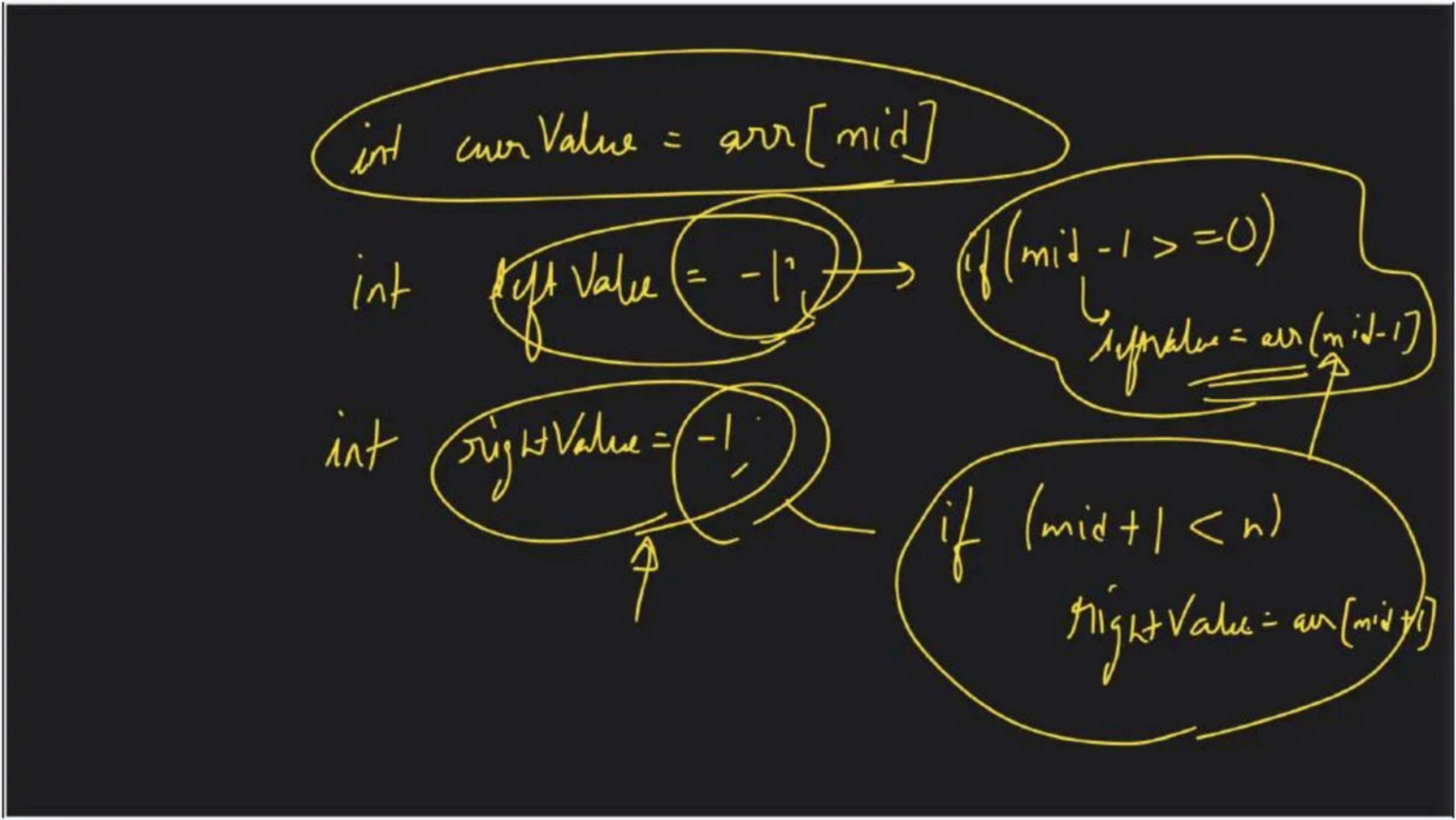
Sigle No - 3

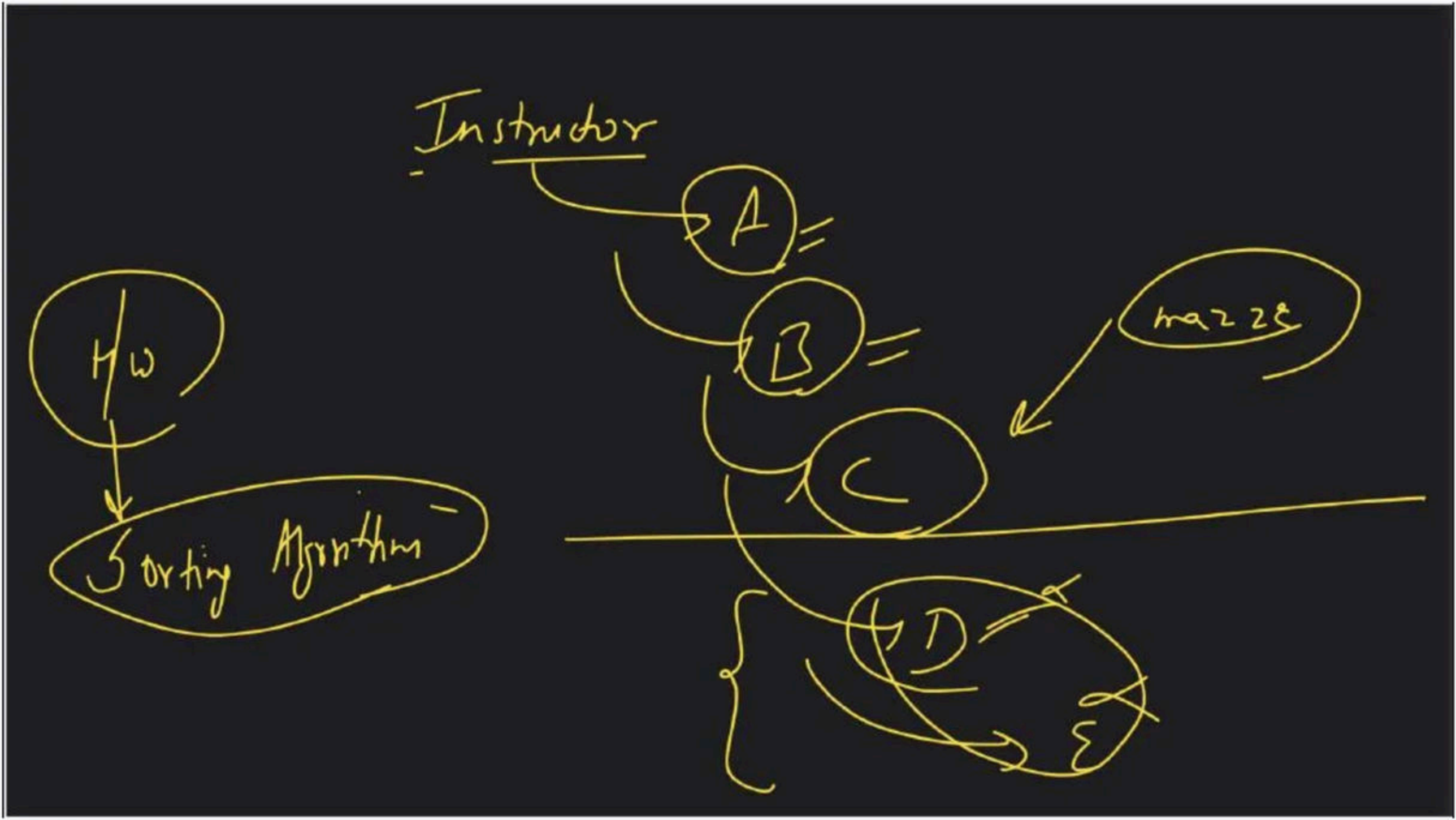
Row with Max - 1

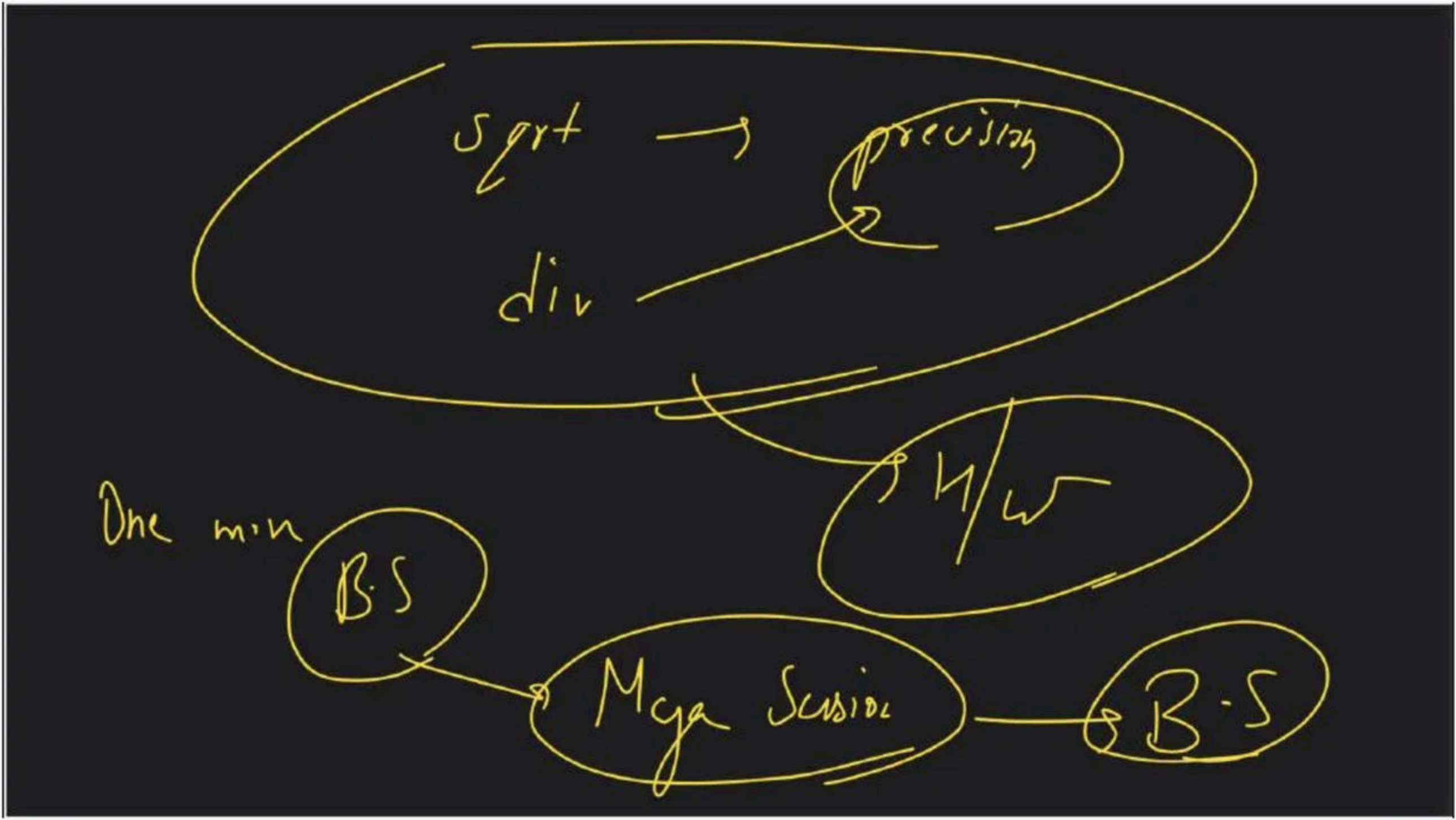
Majority Element] → Moore's Voting algorithm

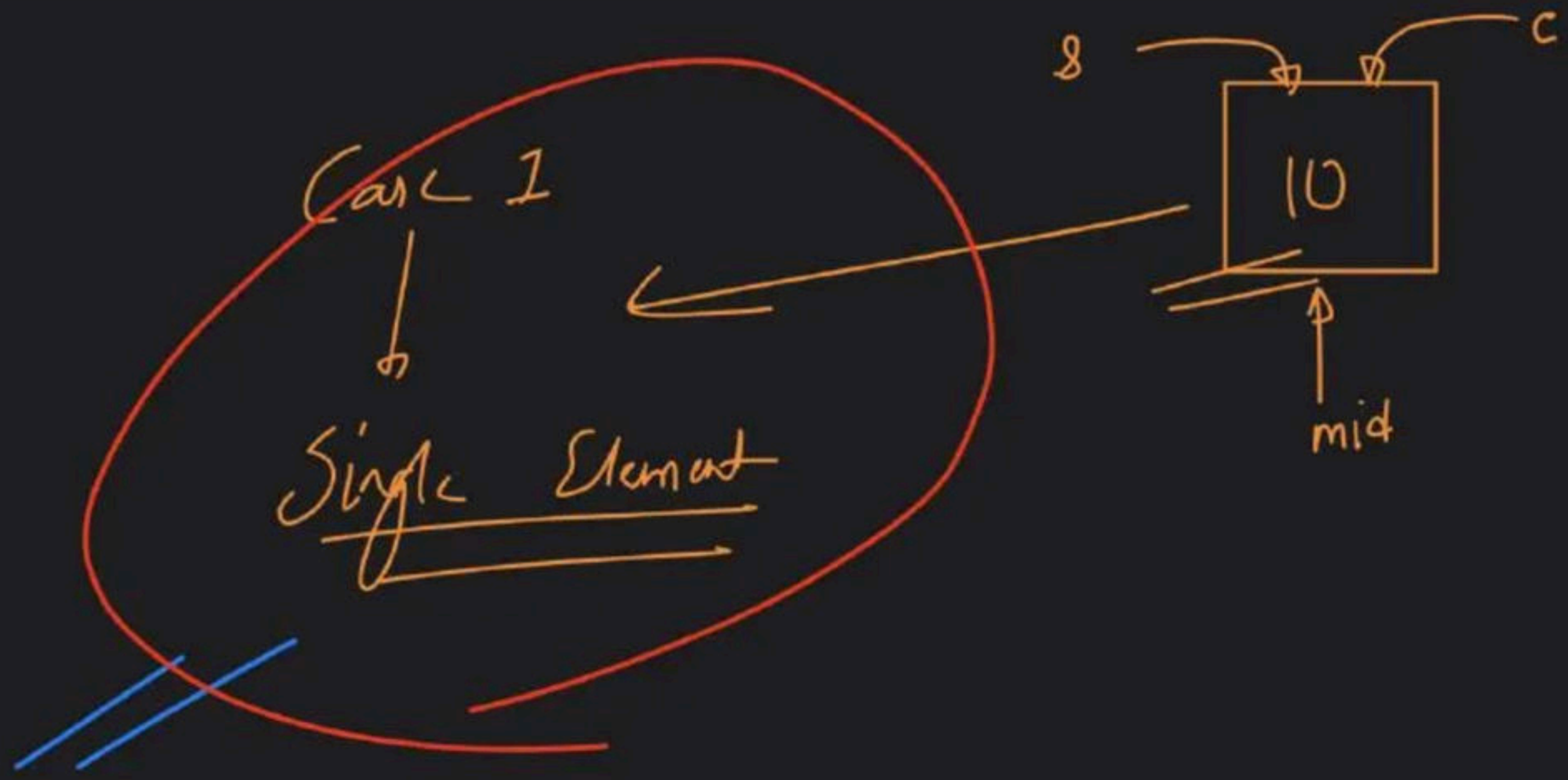
" " 2

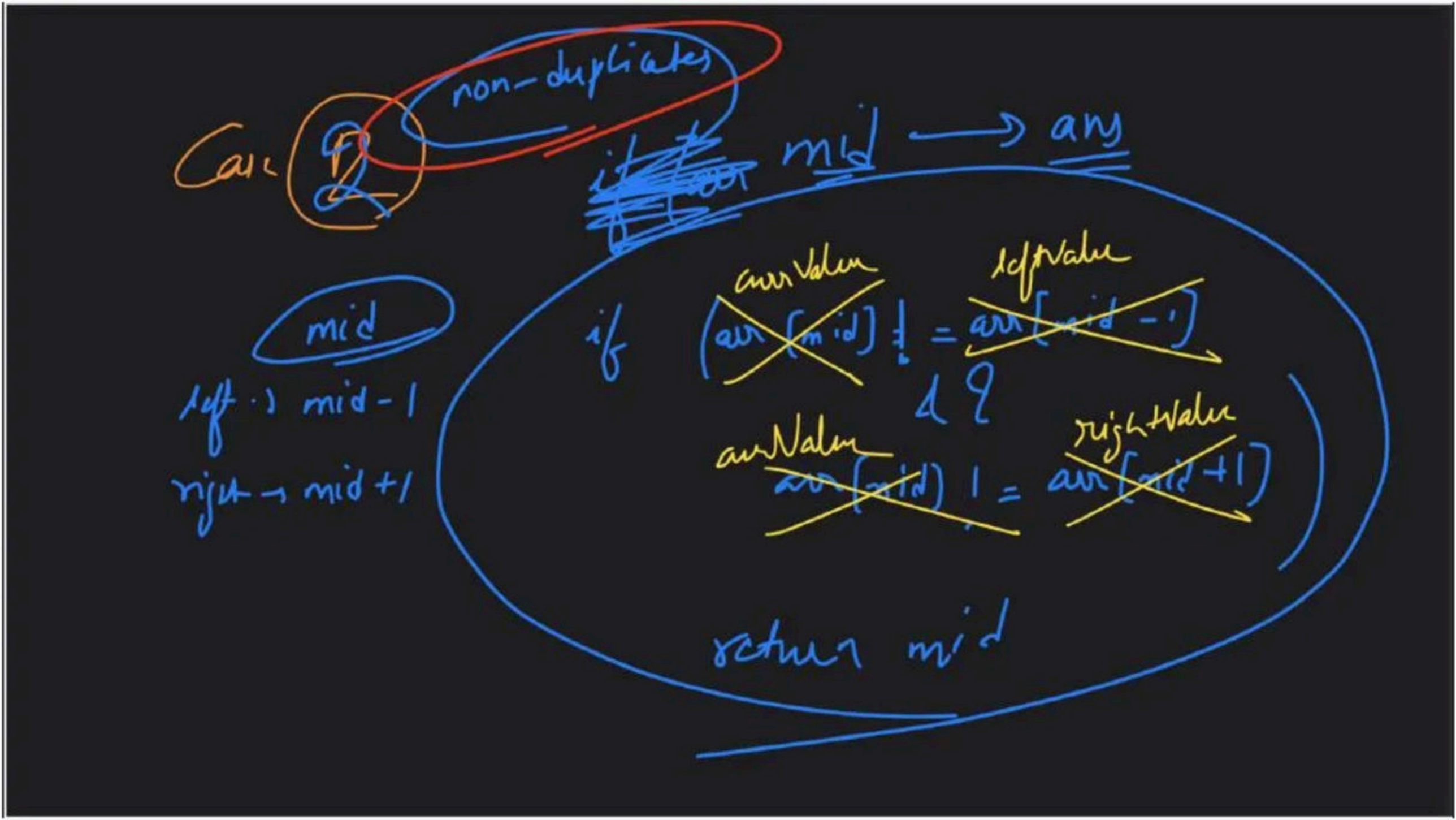

we need to handle this
1, 2, 2, 3, 3
4 2 2 3 3 5 5 2 2 6 6
1 2 3 4 5 6 7 8 9 10
we need to handle this
2 2 3 3 5 5 2 2 6 6 4
0 1 2 3 4 5 6 7 8 9 10
mid
left part → pair -> 1st -> Even
mid
mid
right part → pair -> 1st -> Odd

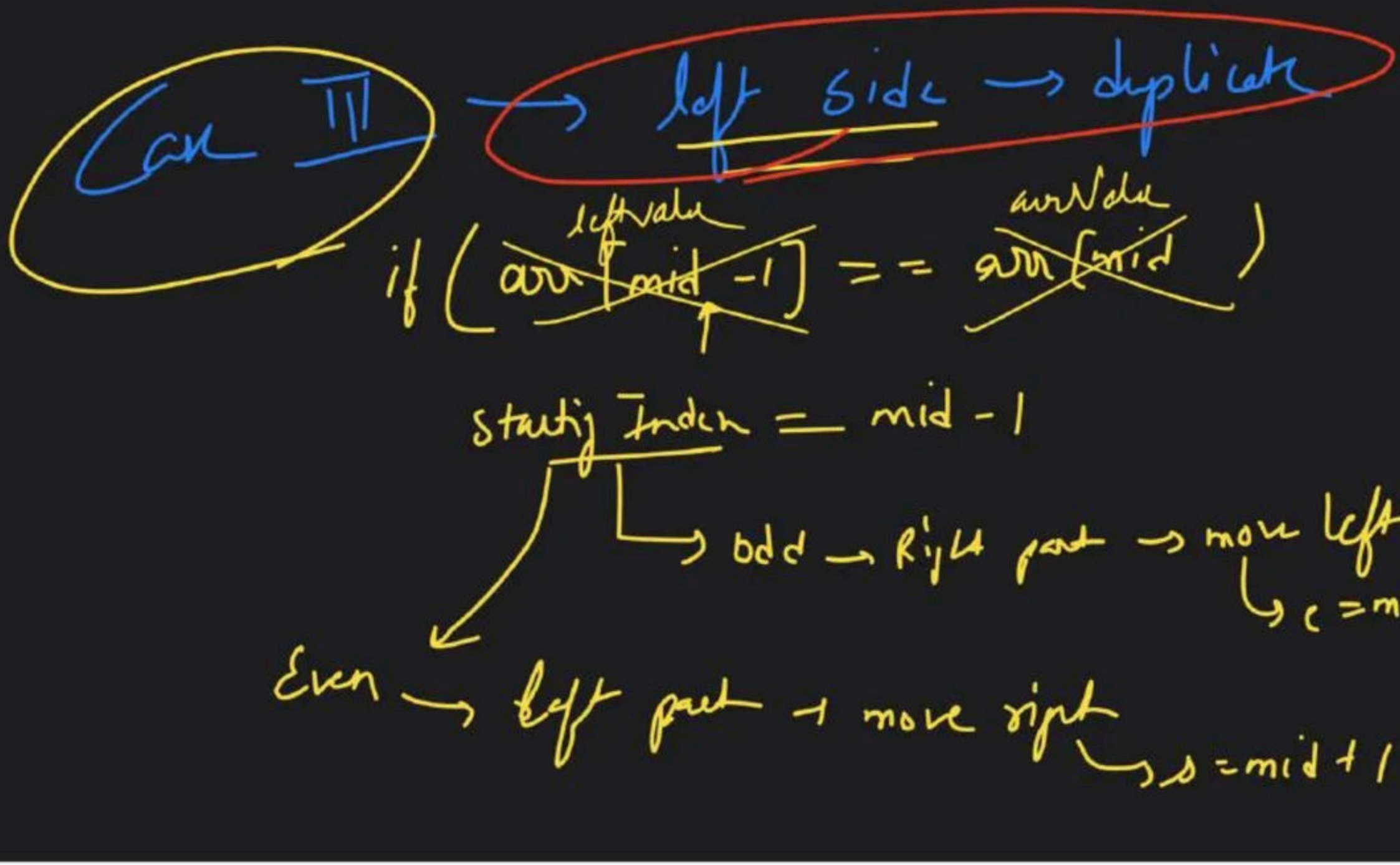

int currValue = arr[mid]
int leftValue = -1
int rightValue = -1
if (mid-1 >=0)
leftValue = arr[mid-1]
if (mid+1 < n)
rightValue = arr[mid+1]

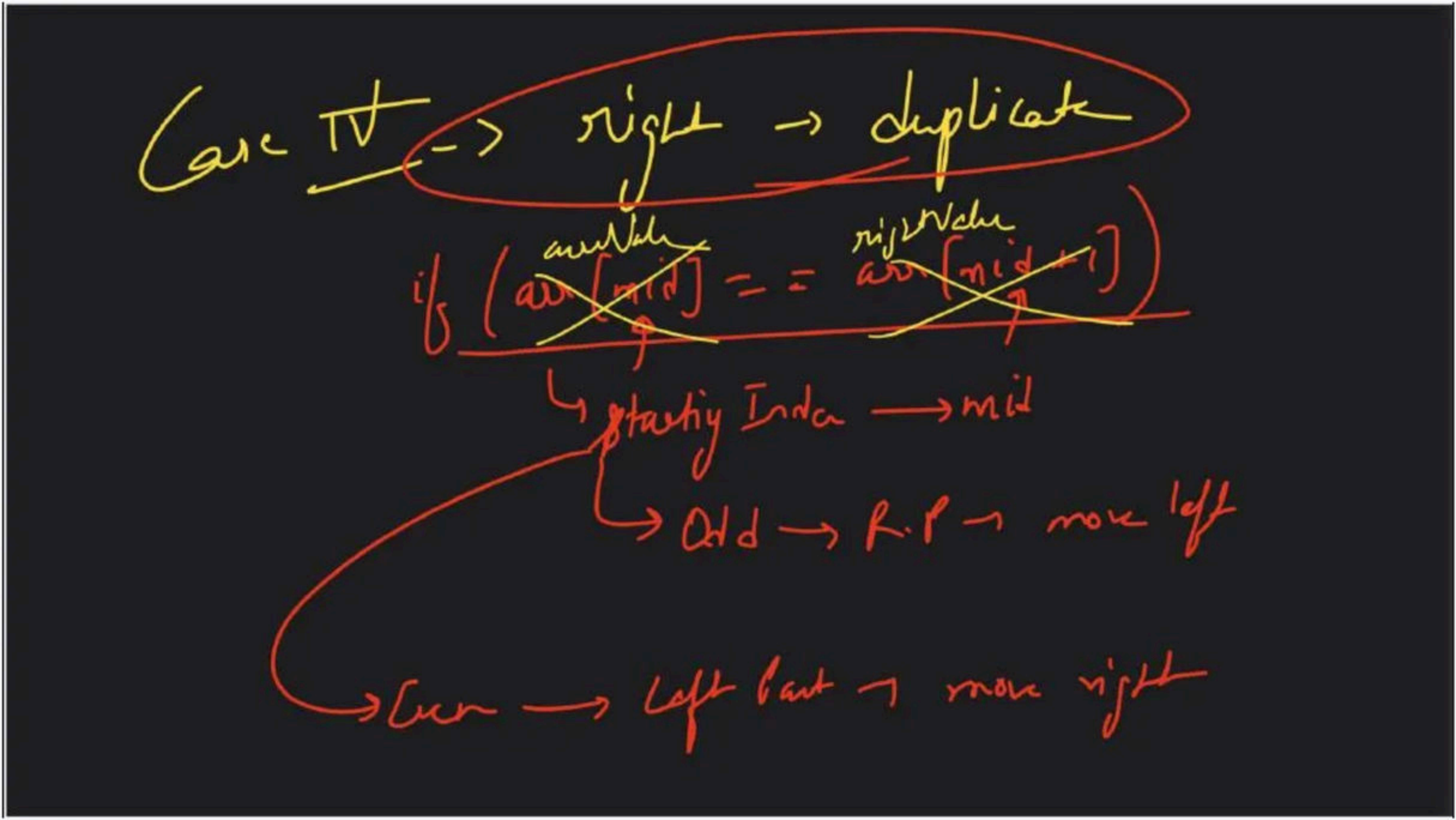

Instructor
A
B
C
D
E
maze
HW
Sorting Algorithm

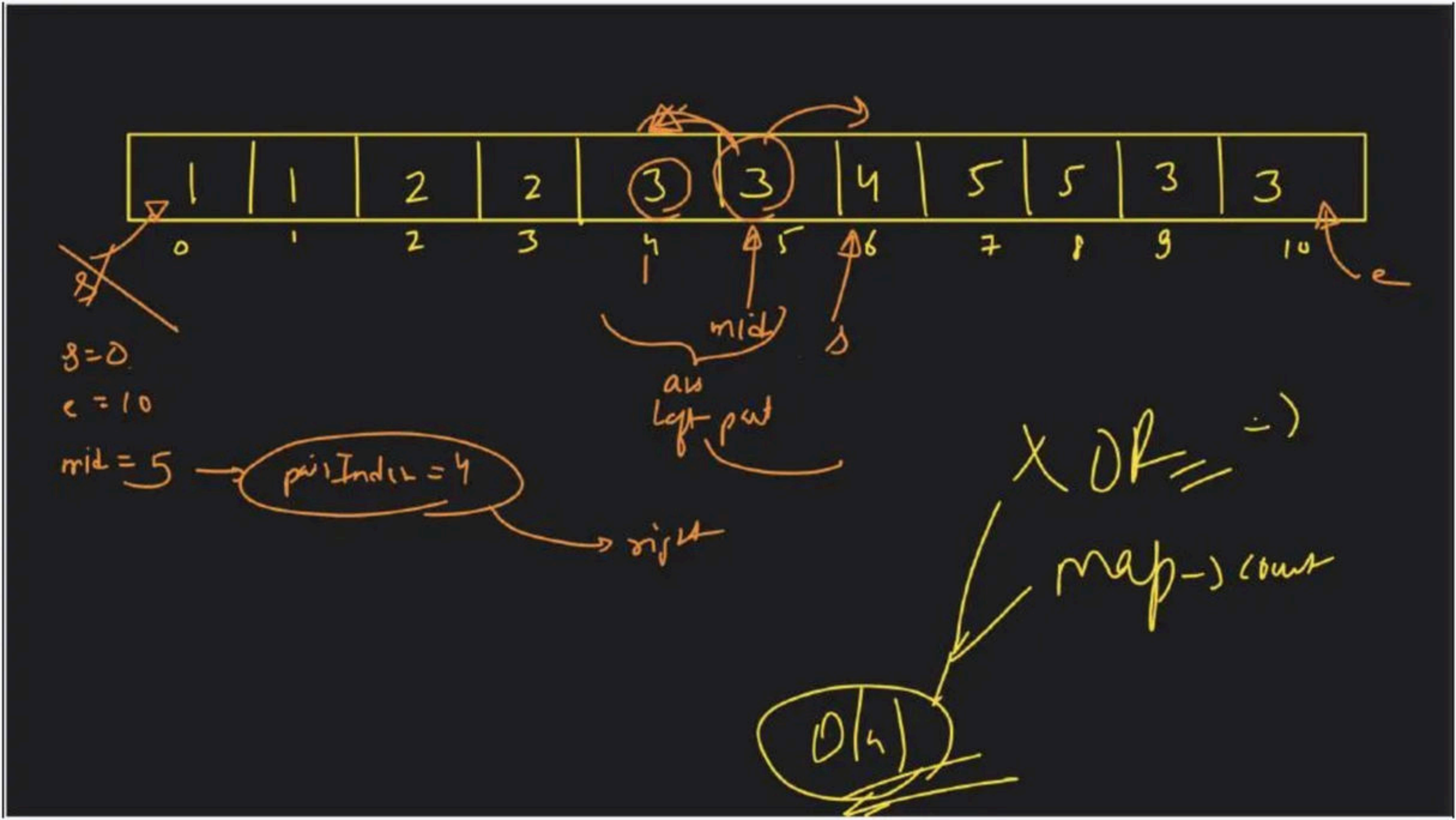

sqrt
precision
div
H/W
One min
B.S
Mega Sussion
B.S

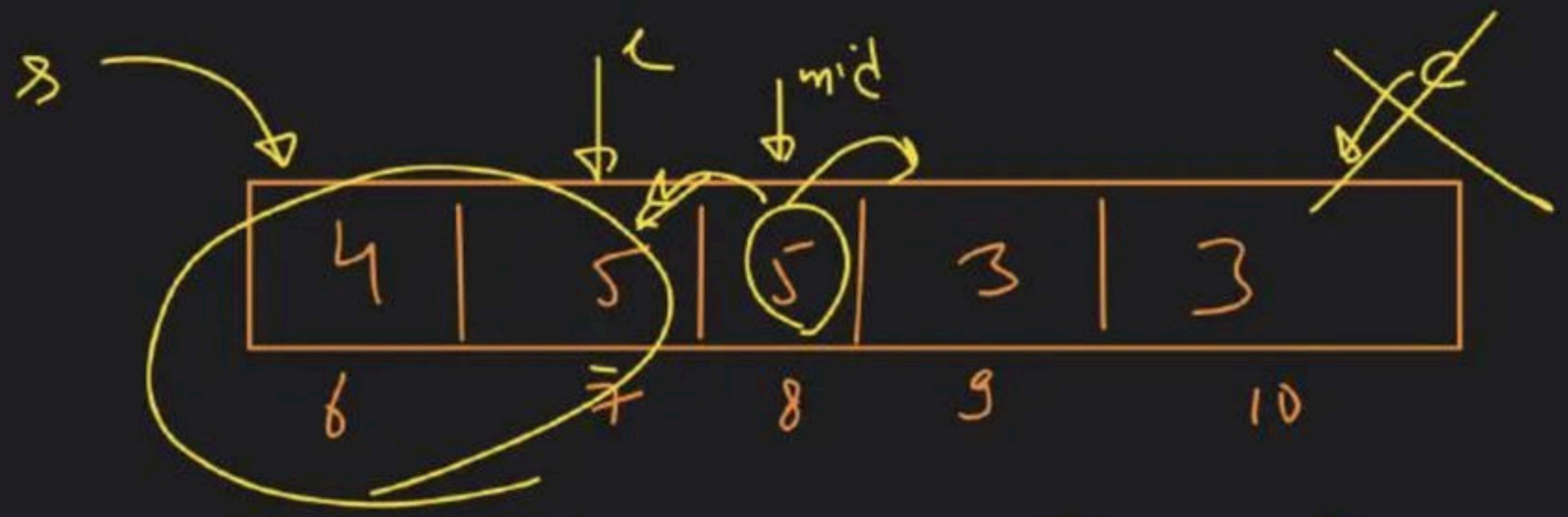

8
c
10
mid
Case 1
Single Element


non-duplicates
mid → any
mid
left → mid - 1
right → mid + 1
if
currValue
arr[mid] != arr[mid - 1]
leftValue
&&
currValue
arr[mid] != arr[mid + 1]
rightValue
return mid


can
left side → duplicate
leftValue
arrNalue
if (arr [mid -1] == arr [mid)
staring Index = mid - 1
odd → Right part → move left
c = mid - 1
Even → left part → move right
s = mid + 1


Case IV -> right -> duplicate
if (arr[mid] == arr[mid+1])
startIndex -> mid
Odd -> R.P -> move left
Even -> Left Part -> move right

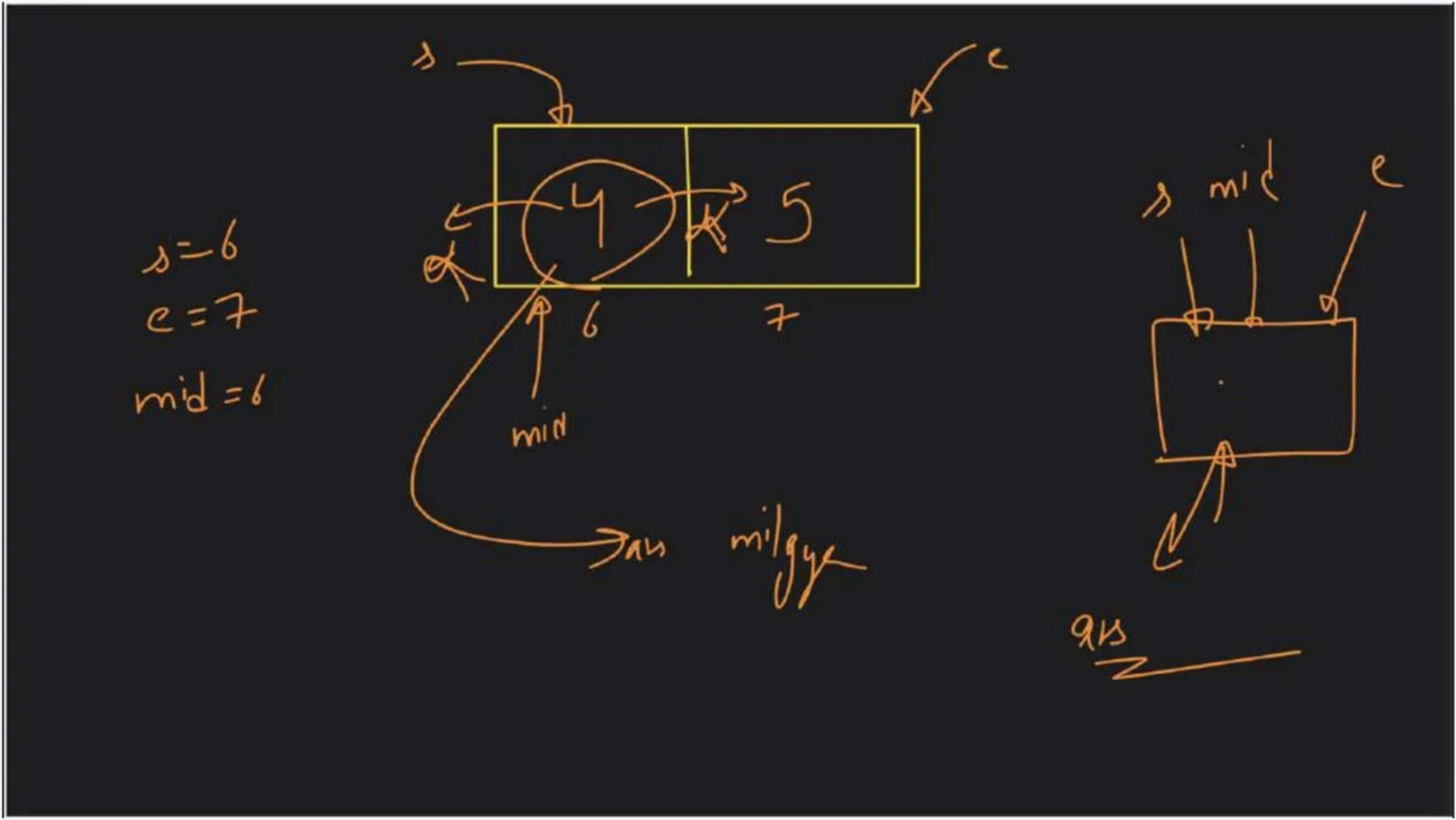

1 1 2 2 3 3 4 5 5 3 3
0 1 2 3 4 5 6 7 8 9 10
s
e
mid
s=0
e=10
mid=5
pairIndex=4
right
XOR
:)
map->count
0 4

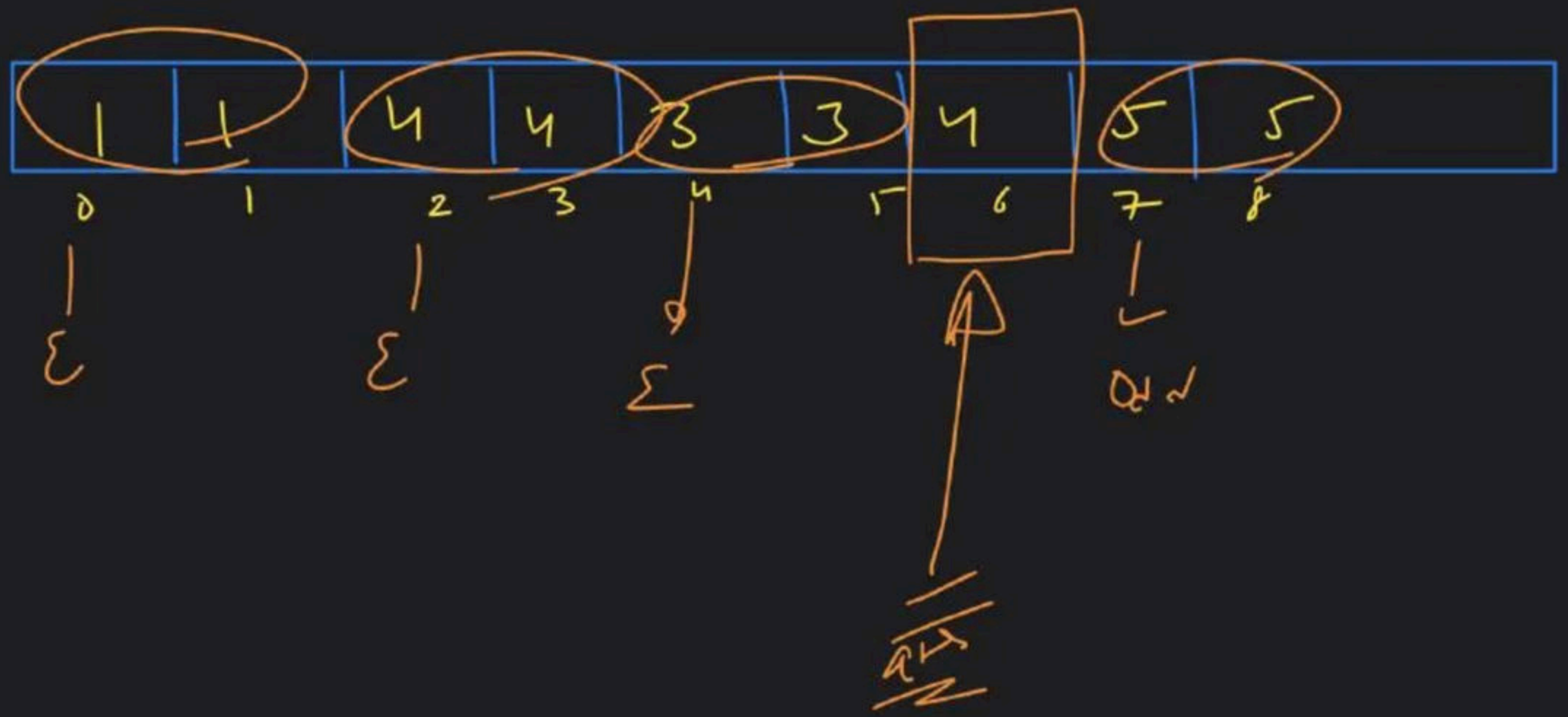


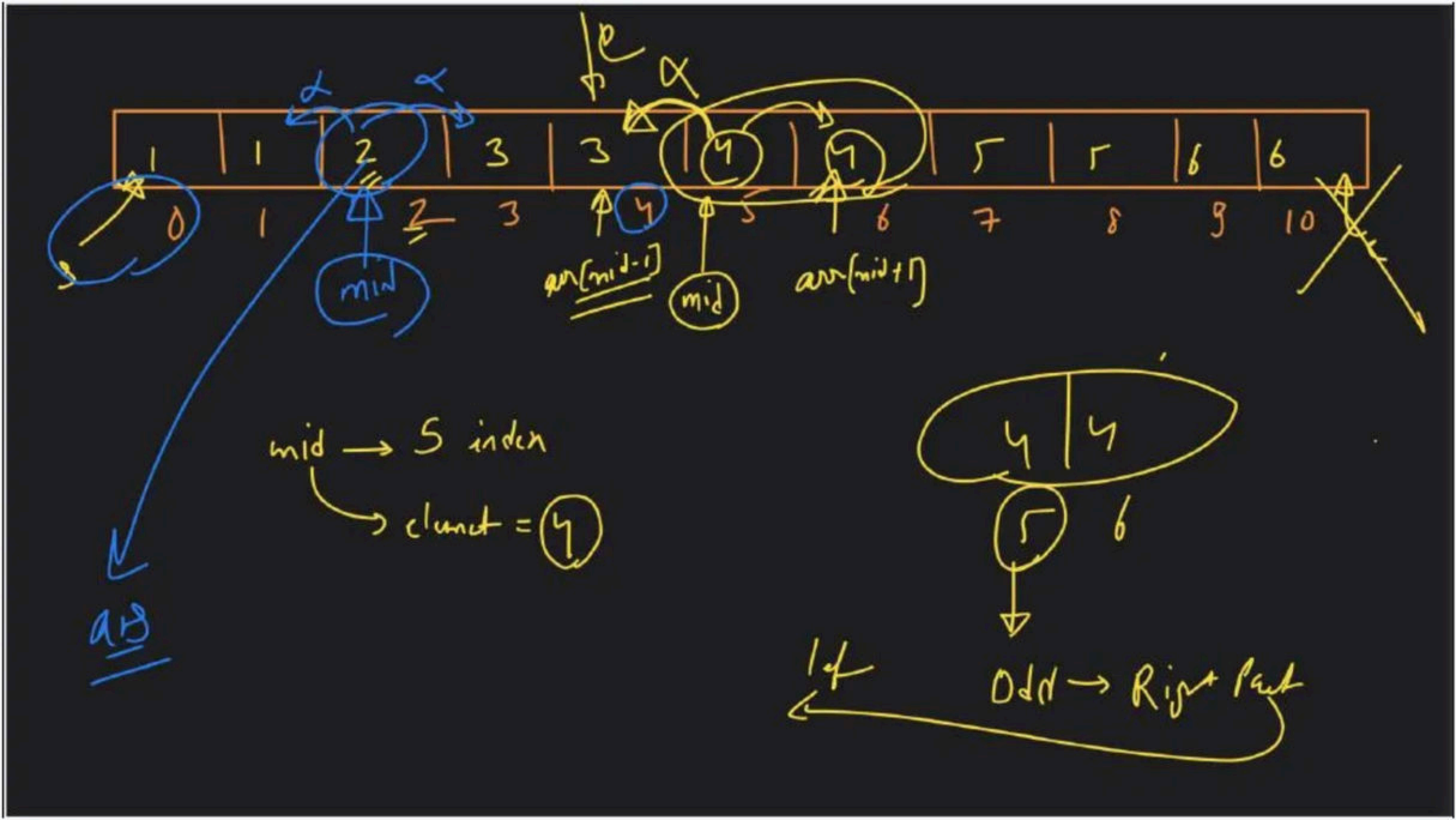
s = 6

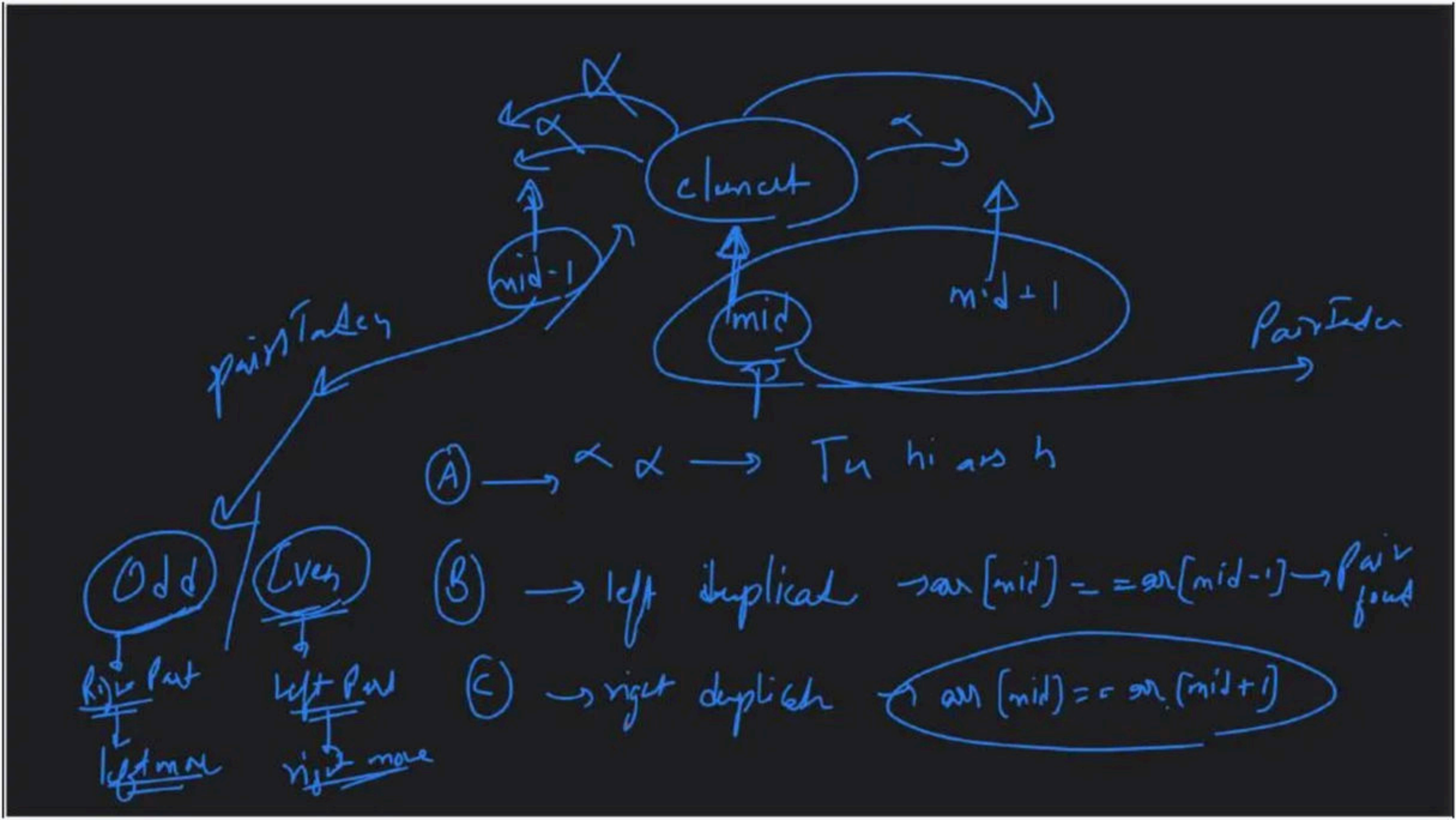
e = 10

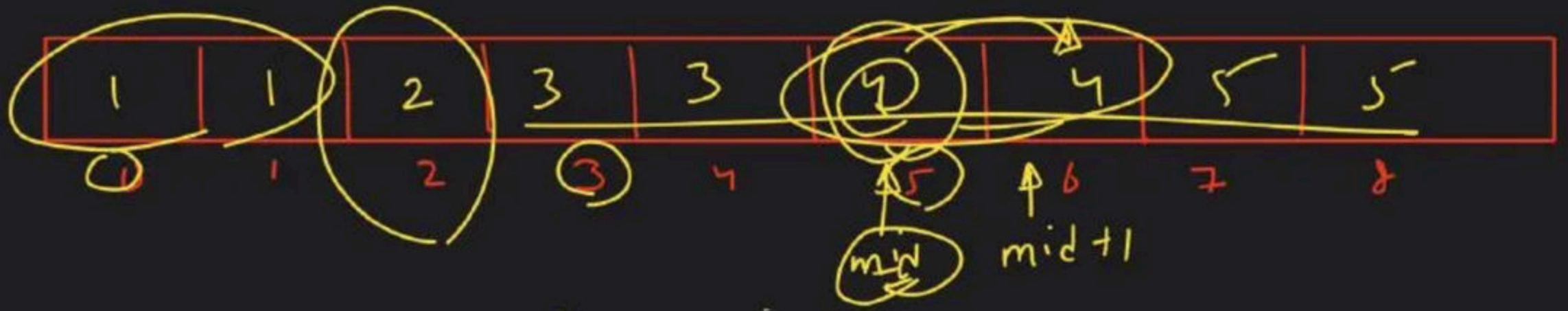
mid = 8 → pairIndex = 7 → Odd → Right part → left


s
e
4
5
6
7
mid
s=6
e=7
mid=6
mid
s
mid
e


1
1
4
4
3
3
4
5
5
0
1
2
3
4
5
6
7
8


1 1 2 3 3 4 4 5 5 6 6
0 1 2 3 4 5 6 7 8 9 10
min
arr[mid-1]
mid
arr[mid+1]
mid → 5 index
element = 4
AB
4 4
5 6
If
Odd → Right Part


clunCut
mid-1
mid
mid+1
pairIndex
Odd
Even
Right Part
Left Part
A → α α → Tu hi aas h
B → left duplicat → arr[mid] == arr[mid-1] → Pair found
C → right duplicate → arr[mid] == arr[mid+1]



Case IV ——> right duplicate

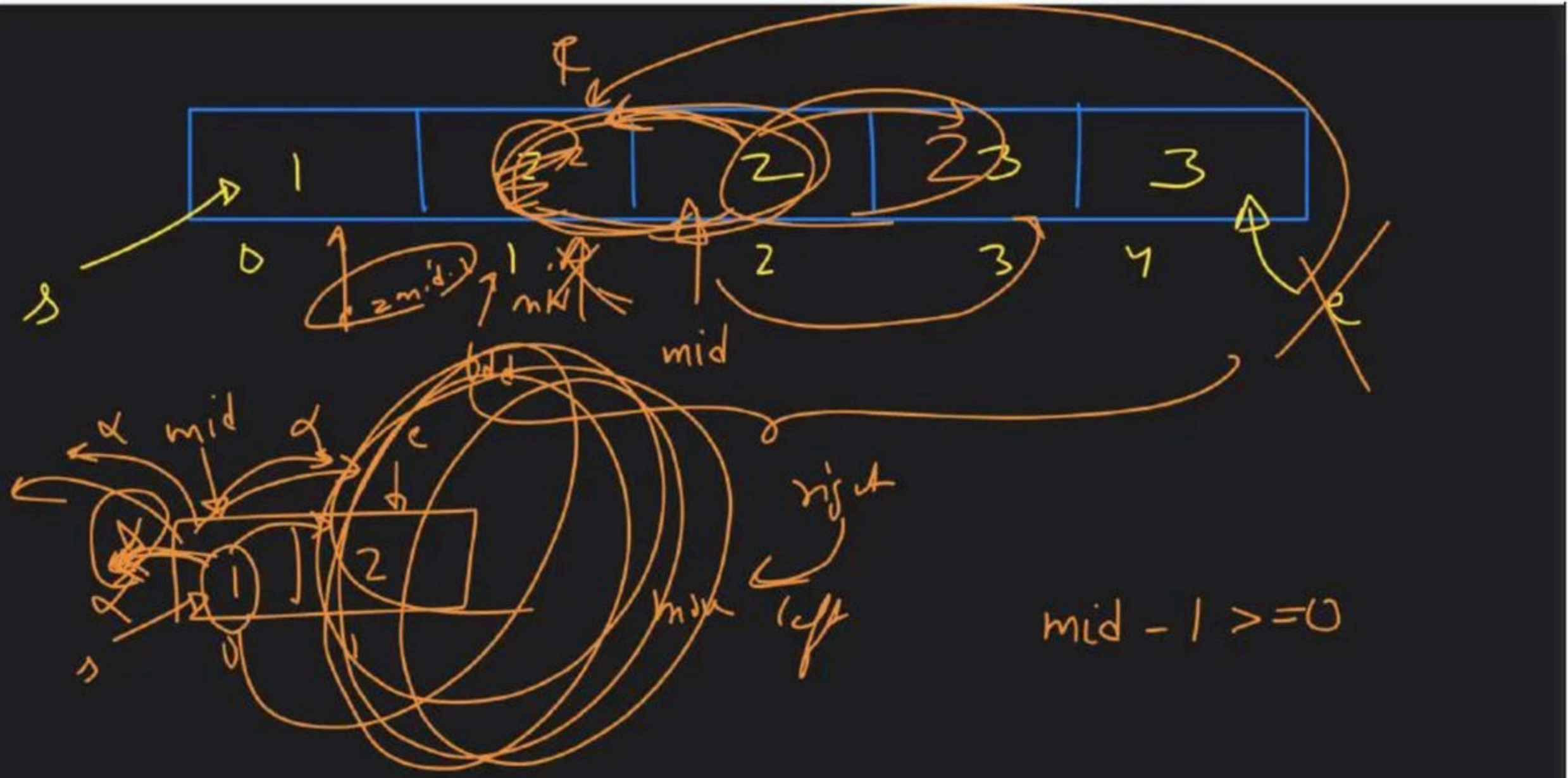
if (arr[mid] == arr[mid+1])

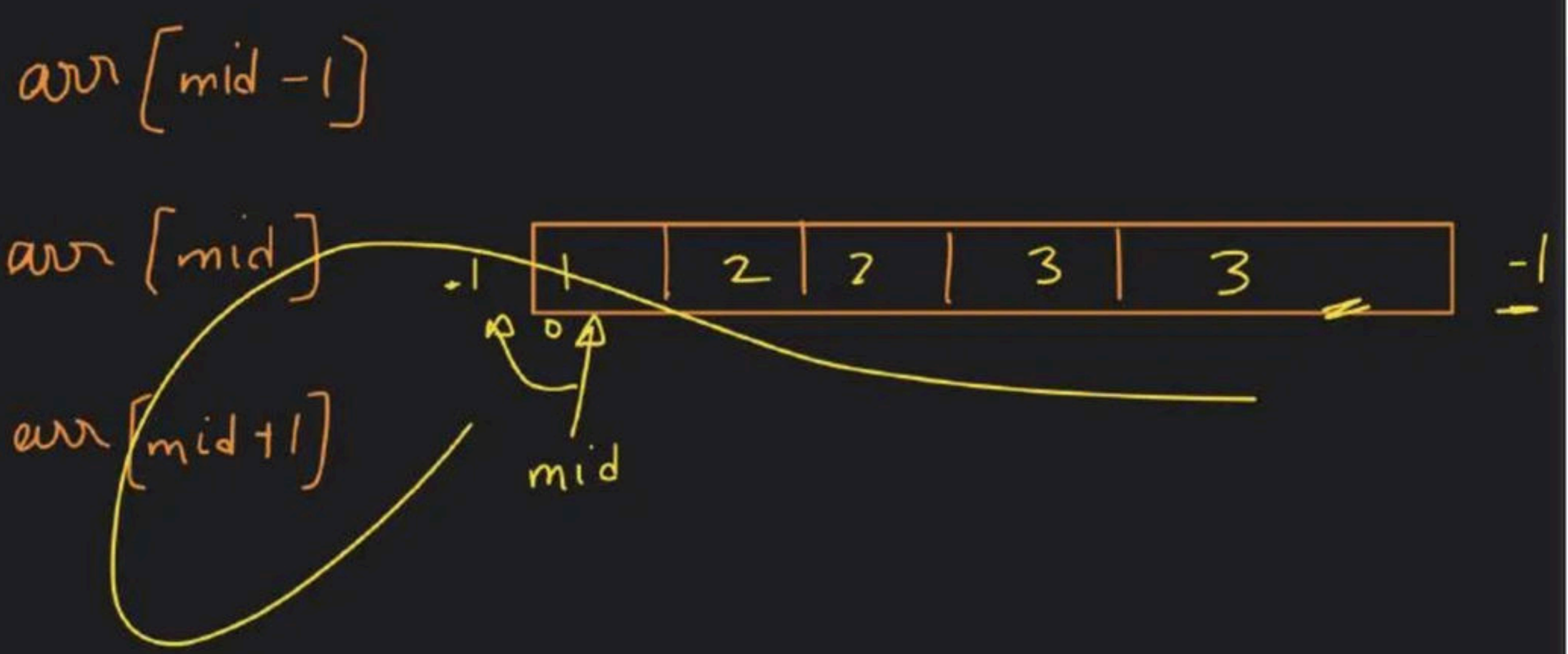
↳ pairStartIndex = mid

↳ Odd → R.P → move left → e = mid-1

↳ Even → L.P → move right → s = mid+1

90.1-


1
2
23
3
0
1
2
3
4
mid
mid
mid
right
mid - 1 >=0


arr[mid-1]
arr[mid]
arr[mid+1]
-1
1
2
2
3
3
-1
0
mid